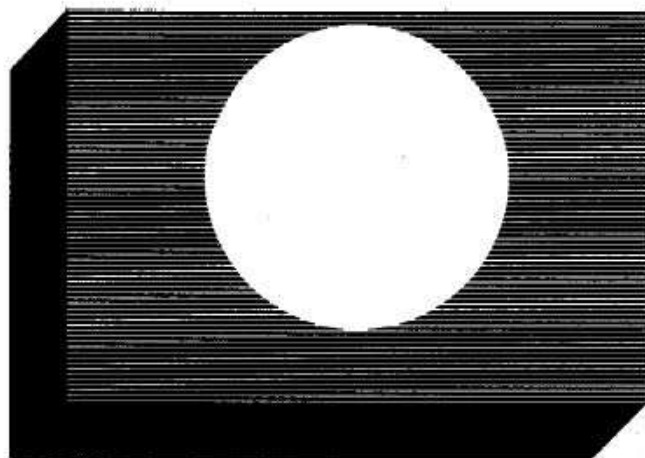


S E C O N D E D I T I O N

SIMULATION MODELING & ANALYSIS

Averill M. Law

W. David Kelton



McGRAW-HILL INTERNATIONAL EDITIONS

Industrial Engineering Series

McGraw-Hill Series in Industrial Engineering and Management Science

Consulting Editor

James L. Riggs, *Department of Industrial Engineering, Oregon State University*

Barish and Kaplan: *Economic Analysis: For Engineering and Managerial Decision Making*

Blank: *Statistical Procedures for Engineering, Management, and Science*

Cleland Kocaoglu: *Engineering Management*

Denton: *Safety Management: Improving Performance*

Dervitsiotis: *Operations Management*

Gillet: *Introduction to Operations Research: A Computer-oriented Algorithmic Approach*

Hicks: *Introduction to Industrial Engineering and Management Science*

Huchingson: *New Horizons for Human Factors in Design*

Law and Kelton: *Simulation Modeling and Analysis*

Leherer: *White-Collar Productivity*

Love: *Inventory Control*

Niebel, Draper and Wysk: *Modern Manufacturing Process Engineering*

Polk: *Methods Analysis and Work Measurement*

Riggs and West: *Engineering Economics*

Taguchi, Elsayed and Hsiang: *Quality Engineering in Production Systems*

Riggs and West: *Essentials of Engineering Economics*

Wu and Coppins: *Linear Programming and Extensions*

SIMULATION MODELING AND ANALYSIS

Second Edition

Averill M. Law

*President
Simulation Modeling and Analysis Company
Tucson, Arizona*

*Professor of Decision Sciences
University of Arizona*

W. David Kelton

*Associate Professor of Operations and Management Science
Curtis L. Carlson School of Management
University of Minnesota*

McGraw-Hill, Inc.

New York St. Louis San Francisco Auckland Bogotá Caracas Hamburg
Lisbon London Madrid Mexico Milan Montreal New Delhi Paris
San Juan São Paulo Singapore Sydney Tokyo Toronto

SIMULATION MODELING AND ANALYSIS
International Editions 1991

Exclusive rights by McGraw-Hill Book Co. - Singapore for manufacture and export. This book cannot be re-exported from the country to which it is consigned by McGraw-Hill.

Copyright © 1991, 1982 by McGraw-Hill, Inc. All rights reserved.
Except as permitted under the United States Copyright Act of 1976, no part of this publication may be reproduced or distributed in any form or by any means, or stored in a data base or retrieval system, without the prior written permission of the publisher.

8 9 0 CWP FC 9 8 7

This book was set in Times Roman.
The editors were Eric M. Munson and Margery Luhrs.
The production supervisor was Louise Karam.
The cover was designed by Ed Butler.

Library of Congress Cataloging-in-Publication Data

Law, Averill M.

Simulation modeling and analysis/Averill M. Law, W. David
Kelton. - 2nd ed.

p. cm. - (McGraw-Hill series in industrial engineering and
management science)

Includes bibliographical references and index.

ISBN 0-07-036698-5

1. Digital computer simulation. I. Kelton, W. David. II. Title
III. Series.

QA76.9.C65L38

1991

003'.3 - dc20

90-42969

When ordering this title, use ISBN 0-07-100803-9

Printed in Singapore

ABOUT THE AUTHORS

Averill M. Law is President of Simulation Modeling and Analysis Company (Tucson, Arizona), and Professor of Decision Sciences at the University of Arizona. He has been a simulation consultant to such organizations as General Motors, IBM, AT&T, General Electric, 3M, Nabisco, Xerox, Kimberly-Clark, NASA, the Army, the Navy, and the Air Force. He has presented more than 160 simulation seminars in 10 countries.

He is the author (or coauthor) of four books and numerous papers on simulation, manufacturing, operations research, and statistics. His article, "Statistical Analysis of Simulation Output Data," was the first invited feature paper on simulation to appear in a major research journal. He won the 1988 Institute of Industrial Engineers' best publication award for his series of papers on the simulation of manufacturing systems. He is the codeveloper of the UniFit II software package for fitting probability distributions to observed data, and he developed a four-hour videotape on simulation with the Society for Manufacturing Engineers. Dr. Law writes a regular column on simulation for *Industrial Engineering* magazine.

He was previously Associate Professor of Industrial Engineering at the University of Wisconsin. Dr. Law has a Ph.D. in Industrial Engineering and Operations Research from the University of California at Berkeley.

W. David Kelton is Associate Professor of Operations and Management Science in the Curtis L. Carlson School of Management at the University of Minnesota, in Minneapolis, where he teaches courses on simulation, stochastic processes, statistics, and computing. He received a B.A. in Mathematics from the University of Wisconsin—Madison, an M.S. in Mathematics from Ohio University, as well as M.S. and Ph.D. degrees in Industrial Engineering from the University of Wisconsin. His research interests include the design and analysis of simulation experiments, applied stochastic processes, and statistical quality control. He serves as Associate Editor for *Operations Research* and *IIE Transactions*, and is Simulation Area Editor for the *ORSA Journal on Comput-*

ing; he is also President of The Institute for Management Sciences College on Simulation. In 1987 he served as Program Chair for the Winter Simulation Conference, and is General Chair for this conference in 1991. He has consulted with private industry, government, and nonprofit organizations on simulation and related topics.

To my wife, Steffi, and children, Heather, Adam, and Brian,
for their encouragement and understanding
during the writing of this book.
Averill M. Law

For Christie, Molly, and Anna.
W. David Kelton

CONTENTS

List of Symbols	xvii
Preface to the Second Edition	xix
Preface to the First Edition	xxi

✓ Chapter 1	Basic Simulation Modeling	1
✓ 1.1	The Nature of Simulation	1
✓ 1.2	Systems, Models, and Simulation	3
✓ 1.3	Discrete-Event Simulation	7
	1.3.1 Time-Advance Mechanisms	8
	1.3.2 Components and Organization of a Discrete-Event Simulation Model	10
✓ 1.4	Simulation of a Single-Server Queueing System	13
	1.4.1 Problem Statement	13
	1.4.2 Intuitive Explanation	19
	1.4.3 Program Organization and Logic	29
	1.4.4 FORTRAN Program	34
	1.4.5 Pascal Program	44
	1.4.6 C Program	52
	1.4.7 Simulation Output and Discussion	60
	1.4.8 Alternative Stopping Rules	62
	1.4.9 Determining the Events and Variables	72
1.5	Simulation of an Inventory System	75
	1.5.1 Problem Statement	75
	1.5.2 Program Organization and Logic	77
	1.5.3 FORTRAN Program	82
	1.5.4 Pascal Program	89
	1.5.5 C Program	96
	1.5.6 Simulation Output and Discussion	102
1.6	Distributed Simulation	103
✓ 1.7	Steps in a Simulation Study	106
✓ 1.8	Other Types of Simulation	109

1.8.1	Continuous Simulation	109
1.8.2	Combined Discrete-Continuous Simulation	112
1.8.3	Monte Carlo Simulation	113
1.9	Advantages, Disadvantages, and Pitfalls of Simulation	114
	Appendix 1A: Fixed-Increment Time Advance	116
	Appendix 1B: A Primer on Queueing Systems	118
	1B.1 Components of a Queueing System	118
	1B.2 Notation for Queueing Systems	119
	1B.3 Measures of Performance for Queueing Systems	120
	Appendix 1C: Notes on the Computers and Compilers Used	122
	Problems	123
	References	130
Chapter 2	Modeling Complex Systems	133
2.1	Introduction	133
2.2	List Processing in Simulation	134
	2.2.1 Approaches to Storing Lists in a Computer	134
	2.2.2 Linked Storage Allocation	135
2.3	A Simple Simulation Language, SIMLIB	141
2.4	Single-Server Queueing Simulation with SIMLIB	150
	2.4.1 Problem Statement	150
	2.4.2 SIMLIB Program	150
	2.4.3 Simulation Output and Discussion	156
2.5	Time-Shared Computer Model	157
	2.5.1 Problem Statement	158
	2.5.2 SIMLIB Program	158
	2.5.3 Simulation Output and Discussion	169
2.6	Multiteller Bank with Jockeying	170
	2.6.1 Problem Statement	170
	2.6.2 SIMLIB Program	171
	2.6.3 Simulation Output and Discussion	183
2.7	Job-Shop Model	185
	2.7.1 Problem Statement	185
	2.7.2 SIMLIB Program	187
	2.7.3 Simulation Output and Discussion	199
2.8	Efficient Event-List Manipulation	200
	Appendix 2A: FORTRAN Code for SIMLIB	202
	Problems	215
	References	232
Chapter 3	Simulation Software	234
3.1	Introduction	234
3.2	Comparison of Simulation Languages with General-Purpose Languages	235

✓ 3.3	Classification of Simulation Software	236
	3.3.1 Simulation Languages vs. Simulators	236
	3.3.2 Modeling Approaches	237
	3.3.3 Common Modeling Elements	240
✓ 3.4	Desirable Software Features	240
	3.4.1 General Features	240
	3.4.2 Animation	241
	3.4.3 Statistical Capabilities	242
	3.4.4 Customer Support	243
	3.4.5 Output Reports	243
✓ 3.5	GPSS	243
	3.5.1 GPSS/H	244
	3.5.2 Simulation of the <i>M/M/1</i> Queue	244
	3.5.3 GPSS/PC	248
✓ 3.6	SIMAN/Cinema	248
	3.6.1 Simulation of the <i>M/M/1</i> Queue	249
3.7	SIMSCRIPT II.5	252
	3.7.1 Simulation of the <i>M/M/1</i> Queue	254
✓ 3.8	SLAM II and Related Software	258
	3.8.1 Simulation of the <i>M/M/1</i> Queue	259
3.9	Comparison of Simulation Languages	263
3.10	Additional Simulation Software	265
	References	266
Chapter 4	Review of Basic Probability and Statistics	267
4.1	Introduction	267
4.2	Random Variables and Their Properties	268
4.3	Simulation Output Data and Stochastic Processes	279
4.4	Estimation of Means, Variances, and Correlations	282
4.5	Confidence Intervals and Hypothesis Tests for the Mean	286
4.6	The Strong Law of Large Numbers	292
4.7	The Danger of Replacing a Probability Distribution by Its Mean	292
	Appendix 4A: Comments on Covariance-Stationary Processes	293
	Problems	294
	References	297
Chapter 5	Building Valid and Credible Simulation Models	298
5.1	Introduction and Definitions	298
5.2	Some Principles of Valid Simulation Modeling	300
5.3	Verification of Simulation Computer Programs	302
5.4	General Perspectives on Validation	306

5.5	A Three-Step Approach for Developing Valid and Credible Simulation Models	307
5.5.1	Develop a Model with High Face Validity	308
5.5.2	Test the Assumptions of the Model Empirically	310
5.5.3	Determine How Representative the Simulation Output Data Are	311
5.6	Statistical Procedures for Comparing Real-World Observations and Simulation Output Data	314
5.6.1	Inspection Approach	315
5.6.2	Confidence-Interval Approach Based on Independent Data	319
5.6.3	Time-Series Approaches	321
	Problems	322
	References	323
Chapter 6	Selecting Input Probability Distributions	325
6.1	Introduction	325
6.2	Useful Probability Distributions	329
6.2.1	Parameterization of Continuous Distributions	329
6.2.2	Continuous Distributions	329
6.2.3	Discrete Distributions	343
6.2.4	Empirical Distributions	350
6.3	Techniques for Assessing Sample Independence	353
6.4	Activity I: Hypothesizing Families of Distributions	356
6.4.1	Summary Statistics	358
6.4.2	Histograms and Line Graphs	360
6.4.3	Quantile Summaries and Box Plots	363
6.5	Activity II: Estimation of Parameters	367
6.6	Activity III: Determining How Representative the Fitted Distributions Are	372
6.6.1	Heuristic Procedures	372
6.6.2	Goodness-of-Fit Tests	380
6.7	An Extended Example	394
6.8	Shifted and Truncated Distributions	400
6.9	Selecting a Distribution in the Absence of Data	403
6.10	Models of Arrival Processes	405
6.10.1	Poisson Processes	405
6.10.2	Nonstationary Poisson Processes	406
6.10.3	Batch Arrivals	409
6.11	Assessing the Homogeneity of Different Data Sets	409
	Appendix 6A: Tables of MLEs for the Gamma and Beta Distributions	411
	Problems	413
	References	417
Chapter 7	Random-Number Generators	420
7.1	Introduction	420

7.2	Linear Congruential Generators	424
7.2.1	Mixed Generators	427
7.2.2	Multiplicative Generators	428
7.3	Other Kinds of Generators	431
7.3.1	More General Congruences	432
7.3.2	Composite Generators	433
7.3.3	Tausworthe and Related Generators	434
7.4	Testing Random-Number Generators	436
7.4.1	Empirical Tests	436
7.4.2	Theoretical Tests	442
7.4.3	Some General Observations on Testing	447
7.5	Random-Number Generation on Microcomputers	447
7.6	Generators Used by Simulation Languages	448
	Appendix 7A: Portable Computer Codes	449
	7A.1 FORTRAN	449
	7A.2 Pascal	451
	7A.3 C	454
	7A.4 Obtaining Initial Seeds for the Streams	456
	Problems	457
	References	459
Chapter 8	Generating Random Variates	462
8.1	Introduction	462
8.2	General Approaches to Generating Random Variates	465
8.2.1	Inverse Transform	465
8.2.2	Composition	474
8.2.3	Convolution	477
8.2.4	Acceptance-Rejection	478
8.2.5	Special Properties	484
8.3	Generating Continuous Random Variates	485
8.3.1	Uniform	485
8.3.2	Exponential	486
8.3.3	m -Erlang	486
8.3.4	Gamma	487
8.3.5	Weibull	490
8.3.6	Normal	490
8.3.7	Lognormal	492
8.3.8	Beta	492
8.3.9	Pearson Type V	493
8.3.10	Pearson Type VI	494
8.3.11	Triangular	494
8.3.12	Empirical Distributions	494
8.4	Generating Discrete Random Variates	496
8.4.1	Bernoulli	496
8.4.2	Discrete Uniform	497
8.4.3	Arbitrary Discrete Distribution	497

8.4.4	Binomial	502
8.4.5	Geometric	502
8.4.6	Negative Binomial	502
8.4.7	Poisson	503
8.5	Generating Correlated Random Variates	504
8.5.1	Using Conditional Distributions	504
8.5.2	Multivariate Normal and Multivariate Lognormal	505
8.5.3	Correlated Gamma Random Variates	506
8.6	Generating Arrival Processes	507
8.6.1	Poisson Processes	507
8.6.2	Nonstationary Poisson Processes	507
8.6.3	Batch Arrivals	510
Appendix 8A:	Validity of the Acceptance-Rejection Method	512
Appendix 8B:	Setup for the Alias Method	513
	Problems	514
	References	518
Chapter 9	Output Data Analysis for a Single System	522
9.1	Introduction	522
9.2	Transient and Steady-State Behavior of a Stochastic Process	525
9.3	Types of Simulations with Regard to Output Analysis	527
9.4	Statistical Analysis for Terminating Simulations	532
9.4.1	Estimating Means	532
9.4.2	Estimating Other Measures of Performance	540
9.4.3	Choosing Initial Conditions	543
9.5	Statistical Analysis for Steady-State Parameters	544
9.5.1	The Problem of the Initial Transient	545
9.5.2	Replication/Deletion Approach for Means	551
9.5.3	Other Approaches for Means	553
9.5.4	Estimating Other Measures of Performance	564
9.6	Statistical Analysis for Steady-State Cycle Parameters	565
9.7	Multiple Measures of Performance	568
9.8	Time Plots of Important Variables	572
Appendix 9A:	Ratios of Expectations and Jackknife Estimators	572
	Problems	575
	References	579
Chapter 10	Comparing Alternative System Configurations	582
10.1	Introduction	582
10.2	Confidence Intervals for the Difference between Performance Measures of Two Systems	586
10.2.1	A Paired- t Confidence Interval	587

10.2.2	A Modified Two-Sample- <i>t</i> Confidence Interval	588
10.2.3	Contrasting the Two Methods	589
10.2.4	Comparisons Based on Steady-State Measures of Performance	590
10.3	Confidence Intervals for Comparing More Than Two Systems	591
10.3.1	Comparisons with a Standard	592
10.3.2	All Pairwise Comparisons	594
10.4	Ranking and Selection	595
10.4.1	Selecting the Best of <i>k</i> Systems	596
10.4.2	Selecting a Subset of Size <i>m</i> Containing the Best of <i>k</i> Systems	598
10.4.3	Selecting the <i>m</i> Best of <i>k</i> Systems	600
10.4.4	Additional Problems and Methods	601
	Appendix 10A: Validity of the Selection Procedures	604
	Appendix 10B: Constants for the Selection Procedures	606
	Problems	607
	References	609
Chapter 11	Variance-Reduction Techniques	612
11.1	Introduction	612
11.2	Common Random Numbers	613
11.2.1	Rationale	613
11.2.2	Applicability	615
11.2.3	Synchronization	617
11.2.4	Some Examples	620
11.3	Antithetic Variates	628
11.4	Control Variates	634
11.5	Indirect Estimation	641
11.6	Conditioning	644
	Problems	648
	References	652
Chapter 12	Experimental Design and Optimization	656
12.1	Introduction	656
12.2	2^k Factorial Designs	659
12.3	Coping with Many Factors	670
12.3.1	2^{k-p} Fractional Factorial Designs	670
12.3.2	Factor-Screening Strategies	677
12.4	Response Surfaces and Metamodels	679
12.5	Gradient Estimation	689
	Problems	691
	References	693
Chapter 13	Simulation of Manufacturing Systems	696
13.1	Introduction	696
13.2	Objectives of Simulation in Manufacturing	697

13.3	Simulation Software for Manufacturing Applications	699
13.4	Modeling System Randomness	703
13.4.1	Sources of Randomness	703
13.4.2	Machine Downtimes	705
13.5	An Extended Example	713
13.5.1	Problem Description and Simulation Results	713
13.5.2	Statistical Calculations	723
13.6	A Simulation Case Study of a Metal-Parts Manufacturing Facility	725
13.6.1	Description of the System	725
13.6.2	Overall Objectives and Issues to Be Investigated	726
13.6.3	Development of the Model	726
13.6.4	Model Verification and Validation	727
13.6.5	Results of the Simulation Experiments	729
13.6.6	Conclusions and Benefits	732
	Problems	733
	References	735
	Appendix	737
	INDEXES	741
	Author Index	743
	Subject Index	749

LIST OF SYMBOLS

Notation or abbreviation	Page number of definition	Notation or abbreviation	Page number of definition
A_i	9	$F(x)$	268
AV	628	F^{-1}	363
A^T	504	$\text{gamma}(\alpha, \beta)$	332
Δb	360	$\text{geom}(p)$	347
Bernoulli(p)	343	GI/Gs	120
$\text{beta}(\alpha_1, \alpha_2)$	338	$h(x)$	360
$\text{bin}(t, p)$	345	H_0	291
$B(\alpha_1, \alpha_2)$	338	IID	13
$B(t)$	18	LIFO	119
C_{ij}	278	L	121
C_j	280	$L(t)$	120
Cor	279	$\text{LN}(\mu, \sigma^2)$	337
Cov	278	$M/E_2/1$	120
CPU	158	$M/G/1$	120
CRN	613	$M/M/1$	120
cv	359	$M/M/2$	583
CV	634	$M/M/s$	120
d	120	MLE	368
$d(n)$	14	$N(\mu, \sigma^2)$	335
df	288	$N(0, 1)$	336
D_i	9	$\text{negbin}(s, p)$	348
$\text{DU}(i, j)$	344	$p(x)$	269
$E()$	275	$p(x, y)$	273
Erlang	331	$P()$	268
$\text{expo}(\beta)$	330	$\text{Pareto}(c, \alpha_2)$	413
FIFO	13	$\text{Poisson}(\lambda)$	349
$f(x)$	270	$\text{PT5}(\alpha, \beta)$	339
$f(x, y)$	275	$\text{PT6}(\alpha, \beta)$	341

Notation or abbreviation	Page number of definition	Notation or abbreviation	Page number of definition
Q	120	$\chi_{k-1, 1-\alpha}$	383
$Q(t)$	15	$\Gamma(\alpha)$	332
(s, S)	75	λ	118, 406
S_i	9	$\lambda(t)$	406
$S^2(n)$	283	$\Lambda(t)$	407
t_i	9	μ	275
$t_{n-1, 1-\alpha/2}$	288	ν	290, 530
$T(n)$	15	$\Phi(z)$	287
triang(a, b, c)	341	$\Psi(\hat{\alpha})$	332
U	30	ρ	120
$U(a, b)$	113, 330	ρ_{ij}	279
$U(0, 1)$	30, 330	ρ_j	280
Var()	277	σ	278
VRT	613	σ^2	277
Weibull(α, β)	333	ω	119
w.p.	75	ζ	319, 587
w	120	$\wedge$	15
$w(n)$	61	$\approx$	287
$\tilde{w}(n)$	62, 642	$\in$	19
W_i	62	$\sim$	343
x_q	363	$\xrightarrow{\mathfrak{g}}$	370
$x_{0.5}$	276	$\binom{t}{x}$	345
$X_{(i)}$	350	$[x]$	344
$\bar{X}(n)$	282	$[x]$	597
$\bar{Y}_i(w)$	546	$\{ \}$	268
$z_{1-\alpha/2}$	287		

PREFACE TO THE SECOND EDITION

While the general philosophy and organization of the First Edition have been retained, the text has been almost completely rewritten. Our primary reasons for doing such a major revision were to bring the material up to date; to improve exposition and clarity, especially for the introductory material; and to emphasize the practical utility of the more advanced techniques treated in the later chapters. There is one completely new chapter on manufacturing systems (Chap. 13), and the material on validation (formerly Chap. 10) has been moved forward (now Chap. 5) to emphasize that this activity must begin early in a simulation project. The numbers of examples, figures, and problems have been greatly expanded. A comprehensive Solutions Manual is available from the publisher.

Several specific features of the Second Edition should be mentioned. At the beginning of each chapter we suggest particular sections that we feel are fundamental for all readers. A list of symbols and abbreviations has also been added. The computer programs in Chaps. 1 and 2 have been rewritten to use FORTRAN 77, and we have added complete Pascal and C versions of the simulations in Chap. 1. (Chapter numbers henceforth refer to the Second Edition.) The material on simulation languages in Chap. 3 has been updated and now includes a discussion of animation. The review material in Chap. 4 has been expanded to make it more accessible. Chapter 5 has been updated to reflect current thinking on validation, and emphasizes practical methods. Chapter 6 has many extended examples to illustrate the difficult task of input-distribution specification. Chapter 7, on random-number generators, has been updated, and includes in App. 7A highly portable codes (in FORTRAN, Pascal, and C) for a reliable generator. Chapter 8 contains expanded explanations of variate-generation methods, emphasizing graphical aids for enhancing insight. Chapter 9 gives an updated and practically oriented discussion of output analysis. Chapters 10 through 12 have been updated and rewritten to enhance development of intuition, and include many detailed examples of the use of statistical comparison and ranking procedures, variance-reduction techniques, and experimental-design methodology. The new chapter (Chap. 13)

discusses simulation applications to manufacturing systems, including relevant software and several comprehensive examples/case studies.

We have received valuable input from a large number of people and organizations in preparing this major revision. The second author received substantial support from the University of Minnesota, especially the Department of Operations and Management Science, the Carlson School of Management, and Academic Computing Services; he is also grateful to the Minnesota Supercomputer Institute for computational support. Special personal thanks go to Michael McComas and Stephen Vincent for numerous contributions throughout the book, and to Tom Schriber for his detailed reading of much of the manuscript. Knowing that we will almost surely commit grievous errors of omission, we would nonetheless like to thank the following individuals for their time and help: Joe Annino, Scott Baird, Diane Bischak, Glenn Browne, Tom Chan, John Charnes, Youngsoo Chun, Dave Goldsman, Jorge Haddock, Wali Haider, Jim Henriksen, Tom Hoffmann, Sheldon Jacobson, Walter Karplus, Pierre L'Ecuyer, Charlie Murgiano, Joe Murray, Chris Nachtsheim, Barry Nelson, Bill Nordgren, Jean O'Reilly, Dennis Pegden, Gene Polley, Steve Roberts, Ed Russell, Paul Sanchez, Bob Sargent, Bruce Schmeiser, Lee Schruben, Aarti Shanker, Murali Shanker, Marlene Smith, Mike Sullivan, Mike Thompson, Brian Unger, and Jim Wilson.

McGraw-Hill and the authors would like to thank the following reviewers for their many helpful comments and suggestions: Osman Balci, Virginia Polytechnic Institute and State University; Wafik H. Iskander, West Virginia University; Barry L. Nelson, Ohio State University; James L. Riggs, deceased; Pirooz Vakilli, Boston University; and Frank K. Wolf, Western Michigan University.

Averill M. Law
W. David Kelton

PREFACE TO THE FIRST EDITION

The goal of *Simulation Modeling and Analysis* is to give an up-to-date treatment of all the important aspects of a simulation study, including modeling, simulation languages, validation, and output data analysis. In addition, we have tried to present the material in a manner understandable to a person having only a basic familiarity with probability, statistics, and computer programming. The book does not sacrifice statistical correctness for expository convenience, but contains virtually no theorems or proofs. Technically difficult topics are placed in starred (*) sections or in an appendix to an appropriate chapter, and left for the advanced reader. (More difficult problems are also starred.) The book strives to motivate intuition about difficult topics and contains a large number of examples, figures, problems, and references for further study. There is also a solutions manual for instructors.

We feel that two of the book's major strengths are its treatment of modeling and of output data analysis. Chapters 1 and 2 show in complete detail how to build simulation models in FORTRAN of a simple queueing system, an inventory system, a time-shared computer model, a multiteller bank with jockeying, and a job-shop model. Chapter 8 contains what we believe is a complete and practical treatment of statistical analysis of simulation output data. Since lack of definitive output data analyses appears to have been a major shortcoming of most simulation studies, we feel that this chapter should enhance the practice of simulation.

We believe that *Simulation Modeling and Analysis* could serve as a textbook for the following types of courses:

1. A beginning course in simulation at the junior, senior, or first-year graduate level for engineering, business, or computer science students (Chaps. 1 through 4 and parts of Chaps. 5 through 8, 10, and 11).
2. A second, advanced course in simulation (most of Chaps. 7 through 12).
3. An introduction to simulation as part of a general course on operations research or management science (Chaps. 1 through 3).

The book should also be of interest to simulation practitioners. As a matter of fact, a large number of such practitioners from industry, government, and the military have used preliminary drafts of the manuscript while attending a seminar on simulation which has been given by the first author for the last four years.

There are a number of people and organizations that have contributed considerably to the writing of this book. Foremost among them are Dr. Thomas Varley and the Office of Naval Research, without whose research support during the past five years this book simply would not have been possible. We would also like to thank the Army Research Office for its research funding to the Mathematics Research Center at the University of Wisconsin. This support in 1980 allowed for the expeditious completion of the book. Most of the development of the simulation language SIMLIB which is discussed in Chap. 2, and almost all of the research of the statistical methods in Chap. 5 was done by Stephen Vincent, a graduate student at Wisconsin. The organization and content of Chap. 7 benefitted greatly from our having in-depth discussions with Professor Bruce Schmeiser of Purdue University. In addition, conversations with the following people positively influenced our thinking on particular chapters of the book: William Biles, Penn State University; Edward Dudewicz, Ohio State University; James Henriksen, Wolverine Software; Stephen Lavenberg, IBM; Richard Nance, Virginia Tech University; Alan Pritsker, Purdue University; Edward Russell, CACI; Robert Sargent, Syracuse University; Thomas Schriber, The University of Michigan; Edward Silver, Waterloo University; and Glenn Thomas, Kent State University. Finally, we acknowledge the following graduate students at Wisconsin who read the entire manuscript and made many valuable suggestions: Steven Kimbrough, Lloyd Koenig, Insup Lee, and Muslim Yildiz.

Averill M. Law
W. David Kelton

SIMULATION MODELING AND ANALYSIS

CHAPTER 1

BASIC SIMULATION MODELING

Recommended sections for a first reading: 1.1 through 1.4, 1.7, 1.9

1.1 THE NATURE OF SIMULATION

This is a book about techniques for using computers to imitate, or *simulate*, the operations of various kinds of real-world facilities or processes. The facility or process of interest is usually called a *system*, and in order to study it scientifically we often have to make a set of assumptions about how it works. These assumptions, which usually take the form of mathematical or logical relationships, constitute a *model* that is used to try to gain some understanding of how the corresponding system behaves.

If the relationships that compose the model are simple enough, it may be possible to use mathematical methods (such as algebra, calculus, or probability theory) to obtain *exact* information on questions of interest; this is called an *analytic* solution. However, most real-world systems are too complex to allow realistic models to be evaluated analytically, and these models must be studied by means of simulation. In a *simulation* we use a computer to evaluate a model *numerically*, and data are gathered in order to *estimate* the desired true characteristics of the model.

As an example of the use of simulation, consider a manufacturing firm that is contemplating building a large extension onto one of its plants but is not sure if the potential gain in productivity would justify the construction cost. It certainly would not be cost-effective to build the extension and then remove it later if it does not work out. However, a careful simulation study could shed some light on the question by simulating the operation of the plant as it currently exists and as it *would be if* the plant were expanded.

Application areas for simulation are numerous and diverse. Below is a list of some particular kinds of problems for which simulation has been found to be a useful and powerful tool:

- Designing and analyzing manufacturing systems
- Evaluating hardware and software requirements for a computer system
- Evaluating a new military weapons system or tactic
- Determining ordering policies for an inventory system
- Designing communications systems and message protocols for them
- Designing and operating transportation facilities such as freeways, airports, subways, or ports
- Evaluating designs for service organizations such as hospitals, post offices, or fast-food restaurants
- Analyzing financial or economic systems

As a technique, simulation is one of the most widely used in operations research and management science. In a survey of graduates of the Department of Operations Research at Case Western Reserve University (one of the first departments of this type), Rasmussen and George (1978) found that among M.S. graduates, simulation ranked fifth among some fifteen subject areas in terms of its value after graduation (behind what they called "statistical methods," "forecasting," "systems analysis," and "information systems," all of which may arguably be outside the realm of operations research and management science). Among Ph.D. graduates, simulation tied (with linear programming) for second (behind "statistical methods"). Thomas and DaCosta (1979), in a survey of a different type, asked some 137 large firms to indicate which of fourteen techniques they used, and simulation came in second, with 84 percent of the firms responding that they used it (what they termed "statistical analysis" came in first in this survey, with 93 percent). The members of the Operations Research Division of the American Institute of Industrial Engineers were surveyed by Shannon, Long, and Buckles (1980), who reported that simulation ranked second in "familiarity" (just behind linear programming), but first in terms of utility and interest, among some twelve methodologies. Forgie (1983) and Harpell, Lane, and Mansour (1989) also reported that simulation ranked second in utilization (again behind "statistical analysis" only) among eight tools in a survey of large corporations. All of these

surveys are by now several years old, and we can assume that simulation's value and usage have since increased, due to improvements in computing power and in simulation software, as discussed below.

There have been, however, several impediments to even wider acceptance and usefulness of simulation. First, models used to study large-scale systems tend to be very complex, and writing computer programs to execute them can be an arduous task indeed. This task has been eased in recent years by the development of excellent software products that automatically provide many of the features needed to code a simulation model. A second problem with simulation of complex systems is that a large amount of computer time is often required. However, this difficulty is becoming less severe as the cost of computing continues to fall. Finally, there appears to be an unfortunate impression that simulation is just an exercise in computer programming, albeit a complicated one. Consequently, many simulation "studies" have been composed of heuristic model building, coding, and a single run of the program to obtain "the answer." We fear that this attitude, which neglects the important issue of how a properly coded model should be used to make inferences about the system of interest, has doubtless led to erroneous conclusions being drawn from many simulation studies. These questions of simulation methodology, which are largely independent of the software and hardware used, form an integral part of the latter chapters of this book.

In the remainder of this chapter (as well as in Chap. 2) we discuss systems and models in considerably more detail and then show how to write computer programs to simulate systems of varying degrees of complexity.

1.2 SYSTEMS, MODELS, AND SIMULATION

A *system* is defined to be a collection of entities, e.g., people or machines, that act and interact together toward the accomplishment of some logical end. [This definition was proposed by Schmidt and Taylor (1970).] In practice, what is meant by "the system" depends on the objectives of a particular study. The collection of entities that compose a system for one study might be only a subset of the overall system for another. For example, if one wants to study a bank to determine the number of tellers needed to provide adequate service for customers who want just to cash a check or make a savings deposit, the system can be defined to be that portion of the bank consisting of the tellers and the customers waiting in line or being served. If, on the other hand, the loan officer and the safety deposit boxes are to be included, the definition of the system must be expanded in an obvious way. [See also Fishman (1978, p. 3).] We define the *state* of a system to be that collection of variables necessary to describe a system at a particular time, relative to the objectives of a study. In a study of a bank, examples of possible state variables are the number of busy tellers, the number of customers in the bank, and the time of arrival of each customer in the bank.

We categorize systems to be of two types, discrete and continuous. A *discrete* system is one for which the state variables change instantaneously at separated points in time. A bank is an example of a discrete system, since state variables—e.g., the number of customers in the bank—change only when a customer arrives or when a customer finishes being served and departs. A *continuous* system is one for which the state variables change continuously with respect to time. An airplane moving through the air is an example of a continuous system, since state variables such as position and velocity can change continuously with respect to time. Few systems in practice are wholly discrete or wholly continuous, but since one type of change predominates for most systems, it will usually be possible to classify a system as being either discrete or continuous.

At some point in the lives of most systems, there is a need to study them to try to gain some insight into the relationships among various components, or to predict performance under some new conditions being considered. Figure 1.1 maps out different ways in which a system might be studied.

- *Experiment with the Actual System vs. Experiment with a Model of the System:* If it is possible (and cost-effective) to alter the system physically and then let it operate under the new conditions, it is probably desirable to do so, for in this case there is no question about whether what we study is

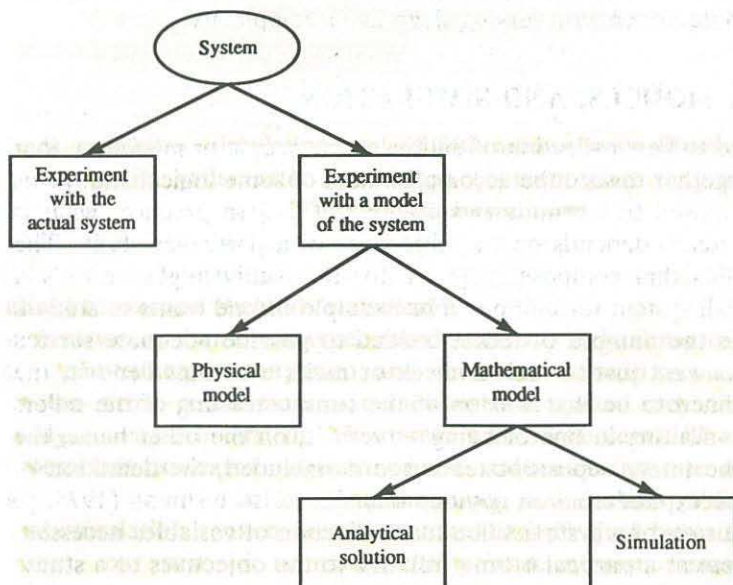


FIGURE 1.1
Ways to study a system.

relevant. However, it is rarely feasible to do this, because such an experiment would often be too costly or too disruptive to the system. For example, a bank may be contemplating reducing the number of tellers to decrease costs, but actually trying this could lead to long customer delays and alienation. More graphically, the "system" might not even exist, but we nevertheless want to study it in its various proposed alternative configurations to see how it should be built in the first place; examples of this situation might be modern flexible manufacturing facilities, or strategic nuclear weapons systems. For these reasons, it is usually necessary to build a model as a representation of the system and study it as a surrogate for the actual system. When using a model, there is always the question of whether it accurately reflects the system for the purposes of the decisions to be made; this question of model *validity* is taken up in detail in Chap. 5.

- *Physical Model vs. Mathematical Model:* To most people, the word "model" evokes images of clay cars in wind tunnels, cockpits disconnected from their airplanes to be used in pilot training, or miniature supertankers scurrying about in a swimming pool. These are examples of *physical* models (also called *iconic* models), and are not typical of the kinds of models that are usually of interest in operations research and systems analysis. Occasionally, however, it has been found useful to build physical models to study engineering or management systems; examples include tabletop scale models of material-handling systems, and in at least one case a full-scale physical model of a fast-food restaurant inside a warehouse, complete with full-scale, real (and presumably hungry) humans [see Swart and Donno (1981)]. But the vast majority of models built for such purposes are *mathematical*, representing a system in terms of logical and quantitative relationships that are then manipulated and changed to see how the model reacts, and thus how the system *would* react—if the mathematical model is a valid one. Perhaps the simplest example of a mathematical model is the familiar relation $d = rt$, where r is the rate of travel, t is the time spent traveling, and d is the distance traveled. This might provide a valid model in one instance (e.g., a space probe to another planet after it has attained its flight velocity) but a very poor model for other purposes (e.g., rush-hour commuting on congested urban freeways).
- *Analytical Solution vs. Simulation:* Once we have built a mathematical model, it must then be examined to see how it can be used to answer the questions of interest about the system it is supposed to represent. If the model is simple enough, it may be possible to work with its relationships and quantities to get an exact, *analytical* solution. In the $d = rt$ example, if we know the distance to be traveled and the velocity, then we can work with the model to get $t = d/r$ as the time that will be required. This is a very simple, closed-form solution obtainable with just paper and pencil, but some analytical solutions can become extraordinarily complex, requiring vast computing

resources; inverting a large nonsparse matrix is a well-known example of a situation in which there is an analytical formula known in principle, but obtaining it numerically in a given instance is far from trivial. If an analytical solution to a mathematical model is available and is computationally efficient, it is usually desirable to study the model in this way rather than via a simulation. However, many systems are highly complex, so that valid mathematical models of them are themselves complex, precluding any possibility of an analytical solution. In this case, the model must be studied by means of *simulation*, i.e., numerically exercising the model for the inputs in question to see how they affect the output measures of performance.

While there may be an element of truth to pejorative old saws such as “method of last resort” sometimes used to describe simulation, the fact is that we are very quickly led to simulation in many situations, due to the sheer complexity of the systems of interest and of the models necessary to represent them in a valid way.

Given, then, that we have a mathematical model to be studied by means of simulation (henceforth referred to as a *simulation model*), we must then look for particular tools to do this. It is useful for this purpose to classify simulation models along three different dimensions:

- *Static vs. Dynamic Simulation Models*: A *static* simulation model is a representation of a system at a particular time, or one that may be used to represent a system in which time simply plays no role; examples of static simulations are Monte Carlo models, discussed in Sec. 1.8.3. On the other hand, a *dynamic* simulation model represents a system as it evolves over time, such as a conveyor system in a factory.
- *Deterministic vs. Stochastic Simulation Models*: If a simulation model does not contain any probabilistic (i.e., random) components, it is called *deterministic*; a complicated (and analytically intractable) system of differential equations describing a chemical reaction might be such a model. In deterministic models, the output is “determined” once the set of input quantities and relationships in the model have been specified, even though it might take a lot of computer time to evaluate what it is. Many systems, however, must be modeled as having at least some random input components, and these give rise to *stochastic* simulation models. (For an example of the danger of ignoring randomness in modeling a system, see Sec. 4.7.) Most queueing and inventory systems are modeled stochastically. Stochastic simulation models produce output that is itself random, and must therefore be treated as only an estimate of the true characteristics of the model; this is one of the main disadvantages of simulation (see Sec. 1.9) and is dealt with in Chaps. 9 through 12 of this book.
- *Continuous vs. Discrete Simulation Models*: Loosely speaking, we define *discrete* and *continuous* simulation models analogously to the way discrete

and continuous systems were defined above. More precise definitions of discrete (event) simulation and continuous simulation are given in Secs. 1.3 and 1.8, respectively. It should be mentioned that a discrete model is not always used to model a discrete system and vice versa. The decision whether to use a discrete or a continuous model for a particular system depends on the specific objectives of the study. For example, a model of traffic flow on a freeway would be discrete if the characteristics and movement of individual cars are important. Alternatively, if the cars can be treated "in the aggregate," the flow of traffic can be described by differential equations in a continuous model. More discussion on this issue can be found in Sec. 5.2, and in particular in Example 5.1.

The simulation models we consider in the remainder of this book, except for those in Sec. 1.8, will be discrete, dynamic, and stochastic and will henceforth be called *discrete-event simulation models*. (Since deterministic models are a special case of stochastic models, the restriction to stochastic models involves no loss of generality.)

1.3 DISCRETE-EVENT SIMULATION

Discrete-event simulation concerns the modeling of a system as it evolves over time by a representation in which the state variables change instantaneously at separate points in time. (In more mathematical terms, we might say that the system can change at only a *countable* number of points in time.) These points in time are the ones at which an event occurs, where an *event* is defined as an instantaneous occurrence that may change the state of the system. Although discrete-event simulation could conceptually be done by hand calculations, the amount of data that must be stored and manipulated for most real-world systems dictates that discrete-event simulations be done on a digital computer. (In Sec. 1.4.2 we carry out a small hand simulation, merely to illustrate the logic involved.)

Example 1.1. Consider a service facility with a single server—e.g., a one-operator barbershop or an information desk at an airport—for which we would like to estimate the (expected) average delay in queue (line) of arriving customers, where the delay in queue of a customer is the length of the time interval from the instant of his arrival at the facility to the instant he begins being served. For the objective of estimating the average delay of a customer, the state variables for a discrete-event simulation model of the facility would be the status of the server, i.e., either idle or busy, the number of customers waiting in queue to be served (if any), and the time of arrival of each person waiting in queue. The status of the server is needed to determine, upon a customer's arrival, whether the customer can be served immediately or must join the end of the queue. When the server completes serving a customer, the number of customers in the queue is used to determine whether the server will become idle or begin serving the first customer in the queue. The time of arrival of a customer is needed to compute his delay in

queue, which is the time he begins being served (which will be known) minus his time of arrival. There are two types of events for this system: the arrival of a customer and the completion of service for a customer, which results in the customer's departure. An arrival is an event since it causes the (state variable) server status to change from idle to busy or the (state variable) number of customers in the queue to increase by 1. Correspondingly, a departure is an event because it causes the server status to change from busy to idle or the number of customers in the queue to decrease by 1. We show in detail how to build a discrete-event simulation model of this single-server queueing system in Sec. 1.4.

In the above example both types of events actually changed the state of the system, but in some discrete-event simulation models events are used for purposes that do not actually effect such a change. For example, an event might be used to schedule the end of a simulation run at a particular time (see Sec. 1.4.8) or to schedule a decision about a system's operation at a particular time (see Sec. 1.5) and might not actually result in a change in the state of the system. This is why we originally said that an event *may* change the state of a system.

1.3.1 Time-Advance Mechanisms

Because of the dynamic nature of discrete-event simulation models, we must keep track of the current value of simulated time as the simulation proceeds, and we also need a mechanism to advance simulated time from one value to another. We call the variable in a simulation model that gives the current value of simulated time the *simulation clock*. The unit of time for the simulation clock is never stated explicitly when a model is written in a general-purpose language such as FORTRAN, Pascal, or C, and it is assumed to be in the same units as the input parameters. Also, there is generally no relationship between simulated time and the time needed to run a simulation on the computer.

Historically, two principal approaches have been suggested for advancing the simulation clock: *next-event time advance* and *fixed-increment time advance*. Since the first approach is used by all major simulation languages and by most people coding their model in a general-purpose language, and since the second is a special case of the first, we shall use the next-event time-advance approach for all discrete-event simulation models discussed in this book. A brief discussion of fixed-increment time advance is given in App. 1A (at the end of this chapter).

With the next-event time-advance approach, the simulation clock is initialized to zero and the times of occurrence of future events are determined. The simulation clock is then advanced to the time of occurrence of the *most imminent* (first) of these future events, at which point the state of the system is updated to account for the fact that an event has occurred, and our knowledge of the times of occurrence of future events is also updated. Then the simulation clock is advanced to the time of the (new) most imminent event, the state of

the system is updated, and future event times are determined, etc. This process of advancing the simulation clock from one event time to another is continued until eventually some prespecified stopping condition is satisfied. Since all state changes occur only at event times for a discrete-event simulation model, periods of inactivity are skipped over by jumping the clock from event time to event time. (Fixed-increment time advance does not skip over these inactive periods, which can eat up a lot of computer time; see App. 1A.) It should be noted that the successive jumps of the simulation clock are generally variable (or unequal) in size.

Example 1.2 We now illustrate in detail the next-event time-advance approach for the single-server queueing system of Example 1.1. We need the following notation:

- t_i = time of arrival of the i th customer ($t_0 = 0$)
- $A_i = t_i - t_{i-1}$ = interarrival time between $(i-1)$ st and i th arrivals of customers
- S_i = time that server actually spends serving i th customer (exclusive of customer's delay in queue)
- D_i = delay in queue of i th customer
- $c_i = t_i + D_i + S_i$ = time that i th customer completes service and departs
- e_i = time of occurrence of i th event of any type (i th value the simulation clock takes on, excluding the value $e_0 = 0$)

Each of these defined quantities will generally be a random variable. Assume that the probability distributions of the interarrival times $A_1, A_2, \dots$ and the service times $S_1, S_2, \dots$ are known and have cumulative distribution functions (see Sec. 4.2) denoted by F_A and F_S , respectively. (In general, F_A and F_S would be determined by collecting data from the system of interest and then fitting distributions to these data using the techniques of Chap. 6.) At time $e_0 = 0$ the status of the server is idle, and the time t_1 of the first arrival is determined by generating A_1 from F_A (techniques for generating random observations from a specified distribution are discussed in Chap. 8) and adding it to 0. The simulation clock is then advanced from e_0 to the time of the next (first) event, $e_1 = t_1$. (See Fig. 1.2, where the curved arrows represent advancing the simulation clock.) Since the customer arriving at time t_1 finds the server idle, she immediately enters service and has a delay in queue of $D_1 = 0$ and the status of the server is changed from idle to busy. The time, c_1 , when the arriving customer will complete service is computed by generating S_1 from F_S and adding it to t_1 . Finally, the time of the second arrival, t_2 , is computed as $t_2 = t_1 + A_2$, where A_2 is generated from F_A . If $t_2 < c_1$, as depicted in Fig. 1.2, the simulation clock is advanced from e_1 to the time of the next event, $e_2 = t_2$. (If c_1 were less than t_2 , the clock would be advanced from e_1 to c_1 .) Since the customer arriving at time t_2 finds the server already busy, the number of customers in the queue is increased from 0 to 1 and the time of arrival of this customer is recorded; however, his service time S_2 is not generated at this time. Also, the time of the third arrival, t_3 , is computed as $t_3 = t_2 + A_3$. If $c_1 < t_3$, as depicted in the figure, the simulation clock is advanced from e_2 to the time of the next event, $e_3 = c_1$, where the customer completing service departs, the customer in the queue (i.e., the one who arrived at time t_2)

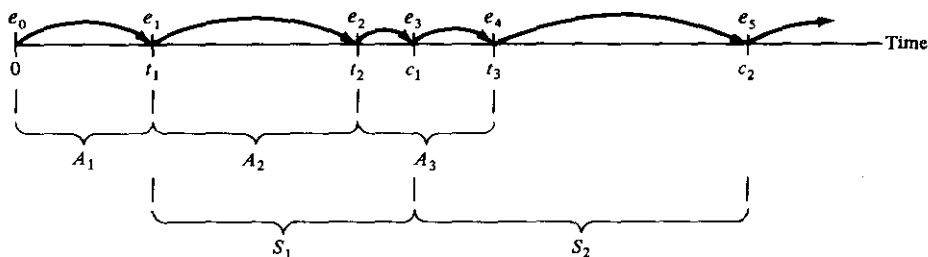


FIGURE 1.2

The next-event time-advance approach illustrated for the single-server queueing system.

begins service and his delay in queue and service-completion time are computed as $D_2 = c_1 - t_2$ and $c_2 = c_1 + S_2$ (S_2 is now generated from F_S), and the number of customers in the queue is decreased from 1 to 0. If $t_3 < c_2$, the simulation clock is advanced from e_3 to the time of the next event, $e_4 = t_3$, etc. The simulation might eventually be terminated when, say, the number of customers whose delays have been observed reaches some specified value.

1.3.2 Components and Organization of a Discrete-Event Simulation Model

Although simulation has been applied to a great diversity of real-world systems, discrete-event simulation models all share a number of common components and there is a logical organization for these components that promotes the coding, debugging, and future changing of a simulation model's computer program. In particular, the following components will be found in most discrete-event simulation models using the next-event time-advance approach:

System state: The collection of state variables necessary to describe the system at a particular time.

Simulation clock: A variable giving the current value of simulated time.

Event list: A list containing the next time when each type of event will occur.

Statistical counters: Variables used for storing statistical information about system performance.

Initialization routine: A subprogram to initialize the simulation model at time zero.

Timing routine: A subprogram that determines the next event from the event list and then advances the simulation clock to the time when that event is to occur.

Event routine: A subprogram that updates the system state when a particular type of event occurs (there is one event routine for each event type).

Library routines: A set of subprograms used to generate random observations

from probability distributions that were determined as part of the simulation model.

Report generator: A subprogram that computes estimates (from the statistical counters) of the desired measures of performance and produces a report when the simulation ends.

Main program: A subprogram that invokes the timing routine to determine the next event and then transfers control to the corresponding event routine to update the system state appropriately. The main program may also check for termination and invoke the report generator when the simulation is over. ┘

┐ The logical relationships (flow of control) among these components is shown in Fig. 1.3. ┘ The simulation begins at time 0 with the main program invoking the initialization routine, where the simulation clock is set to zero, the system state and the statistical counters are initialized, and the event list is initialized. After control has been returned to the main program, it invokes the timing routine to determine which type of event is most imminent. If an event of type i is the next to occur, the simulation clock is advanced to the time that event type i will occur and control is returned to the main program. ┐ Then the main program invokes event routine i , where typically three types of activities occur: (1) the system state is updated to account for the fact that an event of type i has occurred; (2) information about system performance is gathered by updating the statistical counters; and (3) the times of occurrence of future events are generated and this information is added to the event list. ┘ Often it is necessary to generate random observations from probability distributions in order to determine these future event times; we will refer to such a generated observation as a *random variate*. After all processing has been completed, either in event routine i or in the main program, a check is typically made to determine (relative to some stopping condition) if the simulation should now be terminated. If it is time to terminate the simulation, the report generator is invoked from the main program to compute estimates (from the statistical counters) of the desired measures of performance and to produce a report. If it is not time for termination, control is passed back to the main program and the main program—timing routine—main program—event routine—termination check cycle is repeated until the stopping condition is eventually satisfied.

Before concluding this section, a few additional words about the system state may be in order. As mentioned in Sec. 1.2, a system is a well-defined collection of *entities*. Entities are characterized by data values called *attributes*, and these attributes are part of the system state for a discrete-event simulation model. Furthermore, entities with some common property are often grouped together in *lists* (or *files* or *sets*). For each entity there is a *record* in the list consisting of the entity's attributes, and the order in which the records are placed in the list depends on some specified rule. (See Chap. 2 for a discussion of efficient approaches for storing lists of records.) For the single-server queuing facility of Examples 1.1 and 1.2, the entities are the server and the

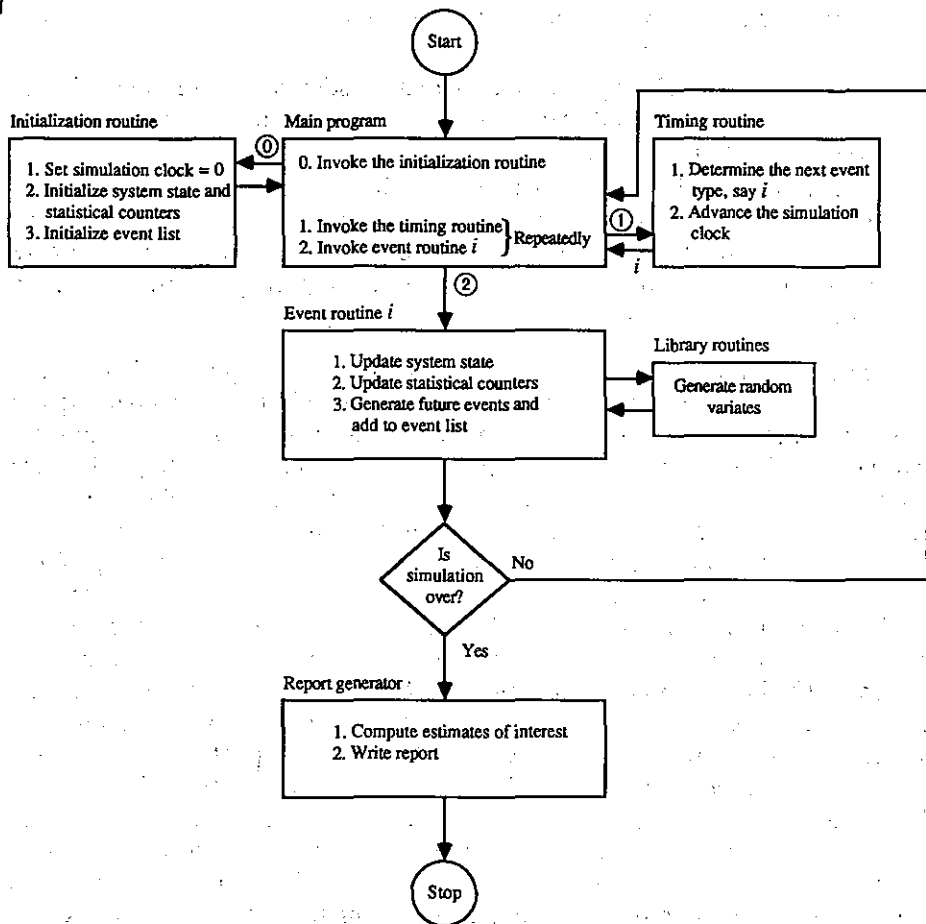


FIGURE 1.3

Flow of control for the next-event time-advance approach.

customers in the facility. The server has the attribute “server status” (busy or idle), and the customers waiting in queue have the attribute “time of arrival.” (The number of customers in the queue might also be considered an attribute of the server.) Furthermore, as we shall see in Sec. 1.4, these customers in queue will be grouped together in a list.

The organization and action of a discrete-event simulation program using the next-event time-advance mechanism as depicted above is fairly typical when coding such simulations in a general-purpose programming language such as FORTRAN, Pascal, or C; it is called the *event-scheduling approach* to simulation modeling, since the times of future events are explicitly coded into the model and are scheduled to occur in the simulated future. It should be

mentioned here that there is an alternative approach to simulation modeling, called the *process approach*, that instead views the simulation in terms of the individual entities involved, and the code written describes the "experience" of a "typical" entity as it "flows" through the system; coding simulations modeled from the process point of view usually requires the use of special-purpose simulation software, as discussed in Chap. 3. Even when taking the process approach, however, the simulation is actually executed behind the scenes in the event-scheduling logic as described above.

1.4 SIMULATION OF A SINGLE-SERVER QUEUEING SYSTEM

This section shows in detail how to simulate a single-server queueing system such as a one-operator barbershop. Although this system seems very simple compared with those usually of real interest, how it is simulated is actually quite representative of the operation of simulations of great complexity.

In Sec. 1.4.1 we describe the system of interest and state our objectives more precisely. We explain intuitively how to simulate this system in Sec. 1.4.2 by showing a "snapshot" of the simulated system just after each event occurs. Section 1.4.3 describes the language-independent organization and logic of the FORTRAN, Pascal, and C codes given in Secs. 1.4.4, 1.4.5, and 1.4.6. The simulation's results are discussed in Sec. 1.4.7, and Sec. 1.4.8 alters the stopping rule to another common way to end simulations. Finally, Sec. 1.4.9 briefly describes a technique for identifying and simplifying the event and variable structure of a simulation.

1.4.1 Problem Statement

Consider a single-server queueing system (see Fig. 1.4) for which the interarrival times $A_1, A_2, \dots$ are independent, identically distributed (IID) random variables. ("Identically distributed" means that the interarrival times have the same probability distribution.) A customer who arrives and finds the server idle enters service immediately, and the service times $S_1, S_2, \dots$ of the successive customers are IID random variables that are independent of the interarrival times. A customer who arrives and finds the server busy joins the end of a single queue. Upon completing service for a customer, the server chooses a customer from the queue (if any) in a first-in, first-out (FIFO) manner. (For a discussion of other queue disciplines and queueing systems in general, see App. 1B.)

The simulation will begin in the "empty-and-idle" state; i.e., no customers are present and the server is idle. At time 0, we will begin waiting for the arrival of the first customer, which will occur after the first interarrival time, A_1 , rather than at time 0 (which would be a possibly valid, but different, modeling assumption). We wish to simulate this system until a fixed number

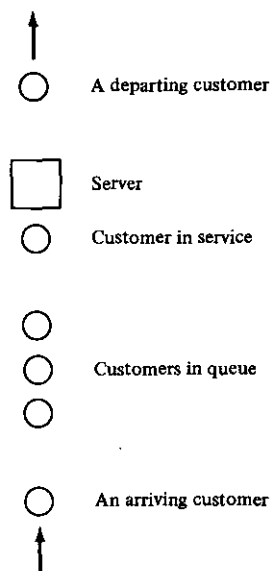


FIGURE 1.4
A single-server queueing system.

(n) of customers have completed their delays in queue; i.e., the simulation will stop when the n th customer enters service. Note that the *time* the simulation ends is thus a random variable, depending on the observed values for the interarrival and service-time random variables.

To measure the performance of this system, we will look at estimates of three quantities. First, we will estimate the expected average delay in queue of the n customers completing their delays during the simulation; we denote this quantity by $d(n)$. The word “expected” in the definition of $d(n)$ means this: On a given run of the simulation (or, for that matter, on a given run of the actual system the simulation model represents), the actual average delay observed of the n customers depends on the interarrival and service-time random variable observations that happen to have been obtained. On another run of the simulation (or on a different day for the real system) there would probably be arrivals at different times, and the service times required would also be different; this would give rise to a different value for the average of the n delays. Thus, the average delay on a given run of the simulation is properly regarded as a random variable itself. What we want to estimate, $d(n)$, is the *expected value* of this random variable. One interpretation of this is that $d(n)$ is the average of a large (actually, infinite) number of n -customer average delays. From a single run of the simulation resulting in customer delays $D_1, D_2, \dots, D_n$, an obvious estimator of $d(n)$ is

$$\hat{d}(n) = \frac{\sum_{i=1}^n D_i}{n}$$

which is just the average of the n D_i 's that were observed in the simulation [so that $\hat{d}(n)$ could also be denoted by $\bar{D}(n)$]. (Throughout this book, a hat (^) above a symbol denotes an estimator.) It is important to note that by "delay" we do not exclude the possibility that a customer could have a delay of zero in the case of an arrival finding the system empty and idle (with this model, we know for sure that $D_1 = 0$); delays with a value of zero *are* counted in the average, since if many delays were zero this would represent a system providing very good service, and our output measure should reflect this. One reason for taking the average of the D_i 's, as opposed to just looking at them individually, is that they will not have the same distribution (e.g., $D_1 = 0$, but D_2 could be positive), and the average gives us a single composite measure of all the customers' delays; in this sense, this is not the usual "average" taken in basic statistics, as the individual terms are not random observations from the same distribution. Note also that by itself, $\hat{d}(n)$ is an estimator based on a "sample" (here, a set of *complete* simulation runs) of size 1, since we are making only a single simulation run. From elementary statistics, we know that a sample of size 1 is not worth much; we return to this issue in Chaps. 9 through 12.

While an estimate of $d(n)$ gives information about system performance from the customers' point of view, the management of such a system may want different information; indeed, since most real simulations are quite complex and may be costly to run, we usually collect many output measures of performance, describing different aspects of system behavior. One such measure for our simple model here is the expected average number of customers in the queue (but not being served), denoted by $q(n)$, where the n is necessary in the notation to indicate that this average is taken over the time period needed to observe the n delays defining our stopping rule. This is a different kind of "average" than the average delay in queue, because it is taken over (continuous) time, rather than over customers (being discrete). Thus, we need to define what is meant by this *time-average* number of customers in queue. To do this, let $Q(t)$ denote the number of customers in queue at time t , for any real number $t \geq 0$, and let $T(n)$ be the time required to observe our n delays in queue. Then for any time t between 0 and $T(n)$, $Q(t)$ is a nonnegative integer. Further, if we let p_i be the expected *proportion* (which will be between 0 and 1) of the time that $Q(t)$ is equal to i , then a reasonable definition of $q(n)$ would be

$$q(n) = \sum_{i=0}^{\infty} i p_i$$

Thus, $q(n)$ is a weighted average of the possible values i for the queue length $Q(t)$, with the weights being the expected proportion of time the queue spends at each of its possible lengths. To estimate $q(n)$ from a simulation, we simply replace the p_i 's with estimates of them, and get

$$\hat{q}(n) = \sum_{i=0}^{\infty} i \hat{p}_i \quad (1.1)$$

where $\hat{p}_i$ is the *observed* (rather than expected) proportion of the time *during the simulation* that there were i customers in the queue. Computationally, however, it is easier to rewrite $\hat{q}(n)$ using some geometric considerations. If we let T_i be the *total* time during the simulation that the queue is of length i , then $T(n) = T_0 + T_1 + T_2 + \cdots$ and $\hat{p}_i = T_i/T(n)$, so that we can rewrite Eq. (1.1) above as

$$\hat{q}(n) = \frac{\sum_{i=0}^{\infty} iT_i}{T(n)} \quad (1.2)$$

Figure 1.5 illustrates a possible time path, or *realization*, of $Q(t)$ for this system in the case of $n = 6$; ignore the shading for now. Arrivals occur at times 0.4, 1.6, 2.1, 3.8, 4.0, 5.6, 5.8, and 7.2. Departures (service completions) occur at times 2.4, 3.1, 3.3, 4.9, and 8.6, and the simulation ends at time $T(6) = 8.6$. Remember in looking at Fig. 1.5 that $Q(t)$ does not count the customer in service (if any), so between times 0.4 and 1.6 there is one customer in the system being served, even though the queue is empty [$Q(t) = 0$]; the same is true between times 3.1 and 3.3, between times 3.8 and 4.0, and between times 4.9 and 5.6. Between times 3.3 and 3.8, however, the system is empty of customers and the server is idle, as is obviously the case between times 0 and 0.4. To compute $\hat{q}(n)$, we must first compute the T_i 's, which can be read off Fig. 1.5 as the (sometimes separated) intervals over which $Q(t)$ is equal to 0, 1,

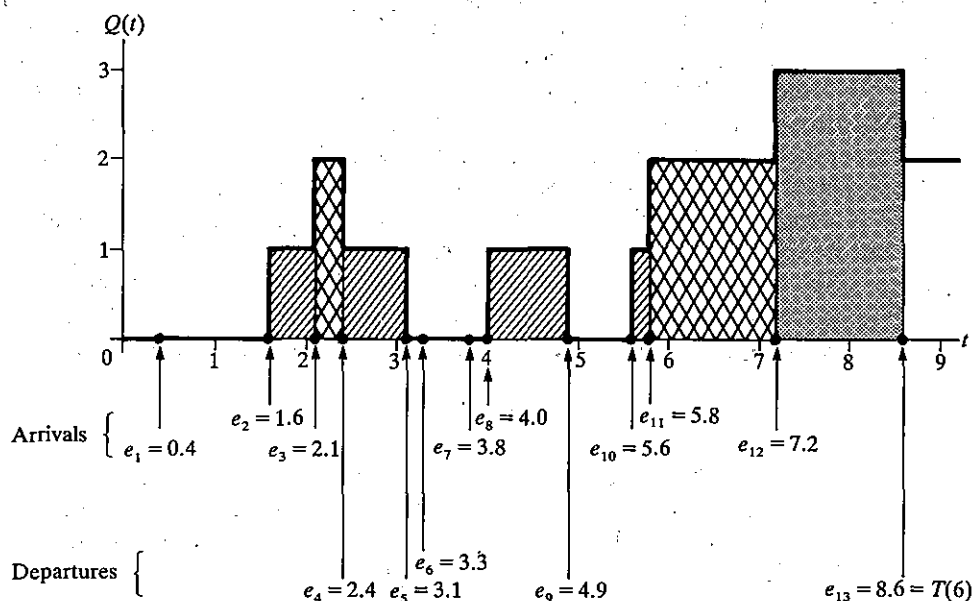


FIGURE 1.5

$Q(t)$, arrival times, and departure times for a realization of a single-server queueing system.

2, and so on:

$$T_0 = (1.6 - 0.0) + (4.0 - 3.1) + (5.6 - 4.9) = 3.2$$

$$T_1 = (2.1 - 1.6) + (3.1 - 2.4) + (4.9 - 4.0) + (5.8 - 5.6) = 2.3$$

$$T_2 = (2.4 - 2.1) + (7.2 - 5.8) = 1.7$$

$$T_3 = (8.6 - 7.2) = 1.4$$

($T_i = 0$ for $i \geq 4$, since the queue never grew to those lengths in this realization.) The numerator in Eq. (1.2) is thus

$$\sum_{i=0}^{\infty} iT_i = (0 \times 3.2) + (1 \times 2.3) + (2 \times 1.7) + (3 \times 1.4) = 9.9 \quad (1.3)$$

and so our estimate of the time-average number in queue from this particular simulation run is $\hat{q}(6) = 9.9/8.6 = 1.15$. Now, note that each of the nonzero terms on the right-hand side of Eq. (1.3) corresponds to one of the shaded areas in Fig. 1.5: 1×2.3 is the diagonally shaded area (in four pieces), 2×1.7 is the cross-hatched area (in two pieces), and 3×1.4 is the screened area (in a single piece). In other words, the summation in the numerator of Eq. (1.2) is just the area under the $Q(t)$ curve between the beginning and the end of the simulation. Remembering that "area under a curve" is an integral, we can thus write

$$\sum_{i=0}^{\infty} iT_i = \int_0^{T(n)} Q(t) dt$$

and the estimator of $q(n)$ can then be expressed as

$$\hat{q}(n) = \frac{\int_0^{T(n)} Q(t) dt}{T(n)} \quad (1.4)$$

While Eqs. (1.4) and (1.2) are equivalent expressions for $\hat{q}(n)$, Eq. (1.4) is preferable since the integral in this equation can be accumulated as simple areas of rectangles as the simulation progresses through time. It is less convenient to carry out the computations to get the summation in Eq. (1.2) explicitly. Moreover, the appearance of Eq. (1.4) suggests a continuous average of $Q(t)$, since in a rough sense, an integral can be regarded as a continuous summation.

The third and final output measure of performance for this system is a measure of how busy the server is. The expected utilization of the server is the expected proportion of time during the simulation [from time 0 to time $T(n)$] that the server is busy (i.e., not idle), and is thus a number between 0 and 1; denote it by $u(n)$. From a single simulation, then, our estimate of $u(n)$ is $\hat{u}(n)$ = the observed proportion of time during the simulation that the server is busy. Now $\hat{u}(n)$ could be computed directly from the simulation by noting the times at which the server changes status (idle to busy or vice versa) and then

doing the appropriate subtractions and division. However, it is easier to look at this quantity as a continuous-time average, similar to the average queue length, by defining the “busy function”

$$B(t) = \begin{cases} 1 & \text{if the server is busy at time } t \\ 0 & \text{if the server is idle at time } t \end{cases}$$

and so $\hat{u}(n)$ could be expressed as the proportion of time that $B(t)$ is equal to 1. Figure 1.6 plots $B(t)$ for the same simulation realization as used in Fig. 1.5 for $Q(t)$. In this case, we get

$$\hat{u}(n) = \frac{(3.3 - 0.4) + (8.6 - 3.8)}{8.6} = \frac{7.7}{8.6} = 0.90 \quad (1.5)$$

indicating that the server was busy about 90 percent of the time during this simulation. Again, however, the numerator in Eq. (1.5) can be viewed as the area under the $B(t)$ function over the course of the simulation, since the height of $B(t)$ is always either 0 or 1. Thus,

$$\hat{u}(n) = \frac{\int_0^{T(n)} B(t) dt}{T(n)} \quad (1.6)$$

and we see again that $\hat{u}(n)$ is the continuous average of the $B(t)$ function corresponding to our notion of utilization. As was the case for $\hat{q}(n)$, the reason for writing $\hat{u}(n)$ in the integral form of Eq. (1.6) is that computationally, as the simulation progresses, the integral of $B(t)$ can easily be accumulated by adding up areas of rectangles. For many simulations involving “servers” of some sort, utilization statistics are quite informative in identifying bottlenecks (utilizations

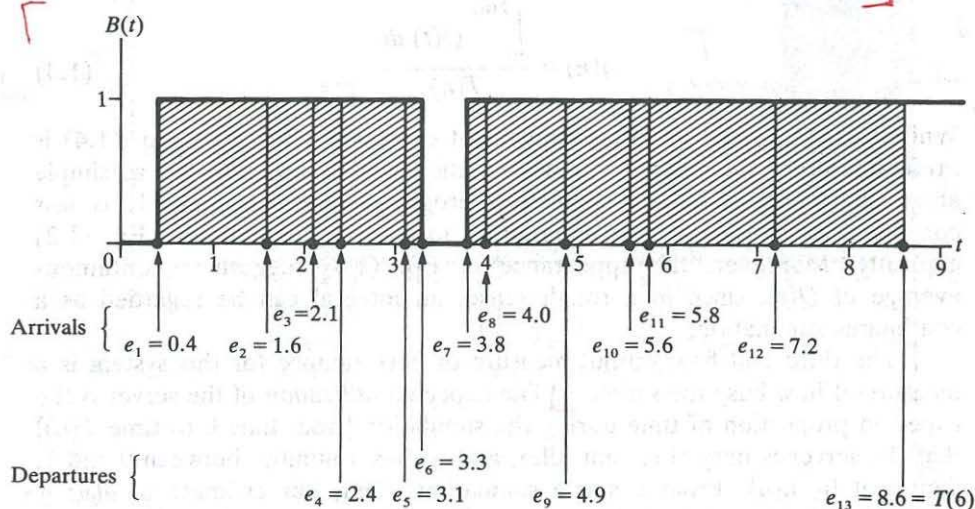


FIGURE 1.6

$B(t)$, arrival times, and departure times for a realization of a single-server queueing system (same realization as in Fig. 1.5).

near 100 percent, coupled with heavy congestion measures for the queue leading in) or excess capacity (low utilizations); this is particularly true if the "servers" are expensive items such as robots in a manufacturing system or large mainframe computers in a data-processing operation.

To recap, the three measures of performance are: the average delay in queue $\hat{d}(n)$, the time-average number of customers in queue $\hat{q}(n)$, and the proportion of time the server is busy $\hat{u}(n)$. The average delay in queue is an example of a *discrete-time statistic*, since it is defined relative to the collection of random variables $\{D_i\}$ that have a discrete "time" index, $i = 1, 2, \dots$. The time-average number in queue and the proportion of time the server is busy are examples of *continuous-time statistics*, since they are defined on the collection of random variables $\{Q(t)\}$ and $\{B(t)\}$, respectively, each of which is indexed on the continuous time parameter $t \in [0, \infty)$. (The symbol $\in$ means "contained in." Thus, in this case, t can be any nonnegative real number.) Both discrete-time and continuous-time statistics are common in simulation, and they furthermore can be other than averages. For example, we might be interested in the *maximum* of all the delays in queue observed (a discrete-time statistic), or the *proportion* of time during the simulation that the queue contained at least five customers (a continuous-time statistic).

The events for this system are the arrival of a customer and the departure of a customer (after a service completion); the state variables necessary to estimate $d(n)$, $q(n)$, and $u(n)$ are the status of the server (0 for idle and 1 for busy), the number of customers in the queue, the time of arrival of each customer currently in the queue (represented as a list), and the time of the last (i.e., most recent) event. The time of the last event, defined to be e_{i-1} if $e_{i-1} \leq t < e_i$ (where t is the current time in the simulation), is needed to compute the width of the rectangles for the area accumulations in the estimates of $q(n)$ and $u(n)$.

1.4.2 Intuitive Explanation

We begin our explanation of how to simulate a single-server queueing system by showing how its simulation model would be represented inside the computer at time $e_0 = 0$ and the times $e_1, e_2, \dots, e_{13}$ at which the 13 successive events occur that are needed to observe the desired number, $n = 6$, of delays in queue. For expository convenience, we assume that the interarrival and service times of customers are

$$A_1 = 0.4, A_2 = 1.2, A_3 = 0.5, A_4 = 1.7, A_5 = 0.2, A_6 = 1.6, A_7 = 0.2, A_8 = 1.4, A_9 = 1.9, \dots$$

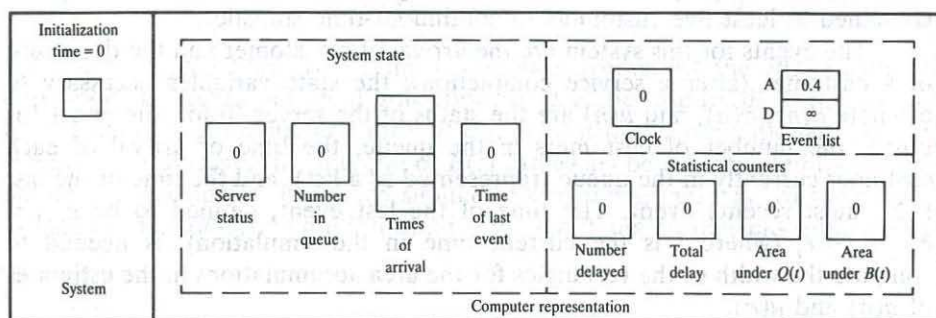
$$S_1 = 2.0, S_2 = 0.7, S_3 = 0.2, S_4 = 1.1, S_5 = 3.7, S_6 = 0.6, \dots$$

Thus, between time 0 and the time of the first arrival there is 0.4 time unit, between the arrivals of the first and second customers there are 1.2 time units, etc., and the service time required for the first customer is 2.0 time units, etc. Note that it is not necessary to declare what the time units are (minutes, hours, etc.), but only to be sure that all time quantities are expressed in the same

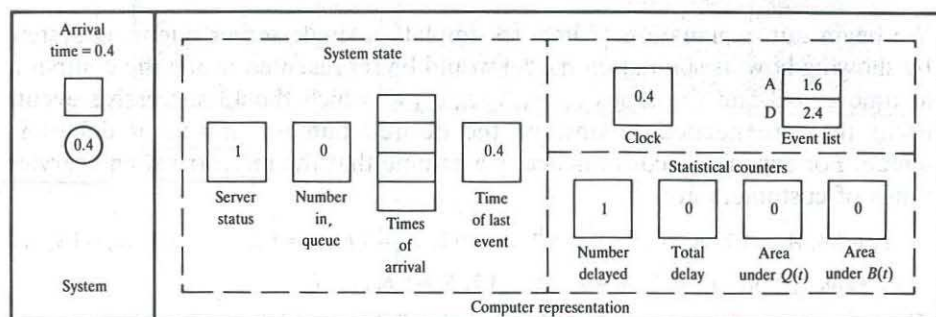
units. In an actual simulation (see Secs. 1.4.4 through 1.4.6), the A_i 's and the S_i 's would be generated from their corresponding probability distributions, as needed, during the course of the simulation. The numerical values for the A_i 's and the S_i 's given above have been artificially chosen so as to generate the same simulation realization as depicted in Figs. 1.5 and 1.6 illustrating the $Q(t)$ and $B(t)$ processes.

Figure 1.7 gives a snapshot of the system itself and of a computer representation of the system at each of the times $e_0 = 0, e_1 = 0.4, \dots, e_{13} = 8.6$. In the "system" pictures, the square represents the server, and circles represent customers; the numbers inside the customer circles are the times of their arrivals. In the "computer representation" pictures, the values of the variables shown are after all processing has been completed at that event. Our discussion will focus on how the computer representation changes at the event times.

$t = 0$: *Initialization.* The simulation begins with the main program invoking the initialization routine. Our modeling assumption was that



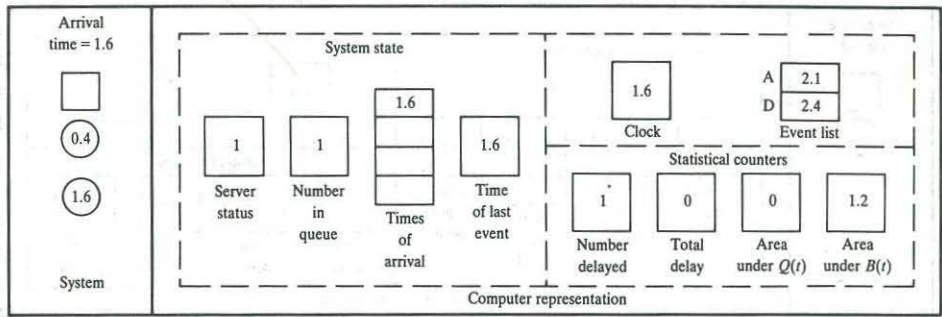
(a)



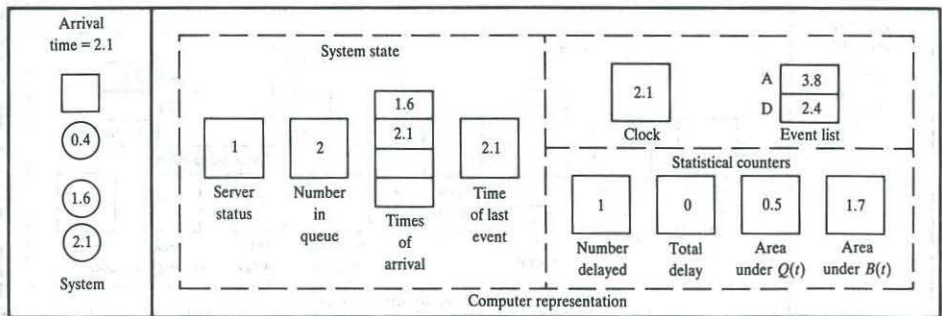
(b)

FIGURE 1.7

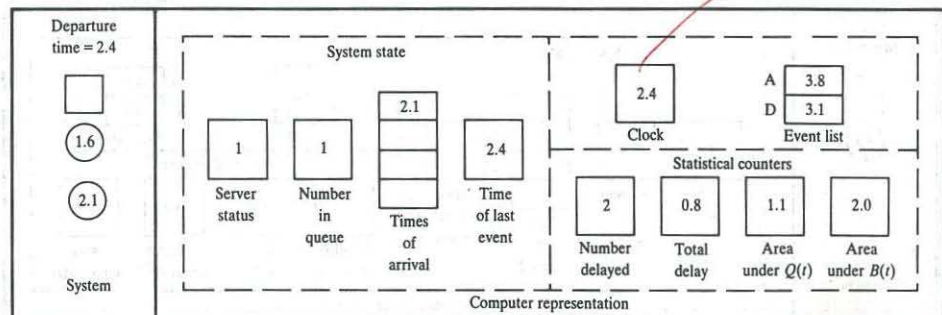
Snapshots of the system and of its computer representation at time 0 and at each of the thirteen succeeding event times.



(c)

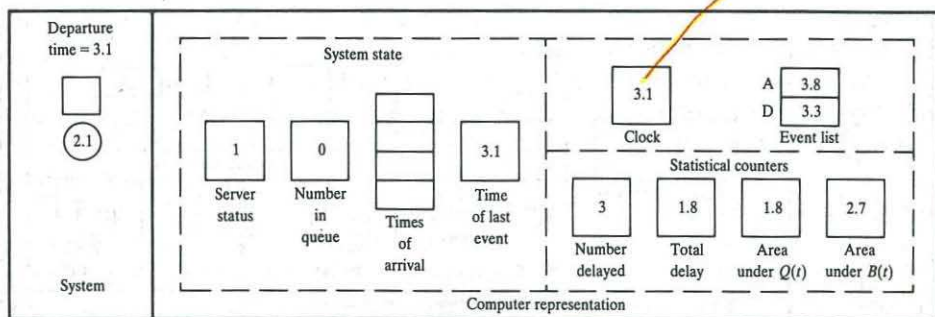


(d)

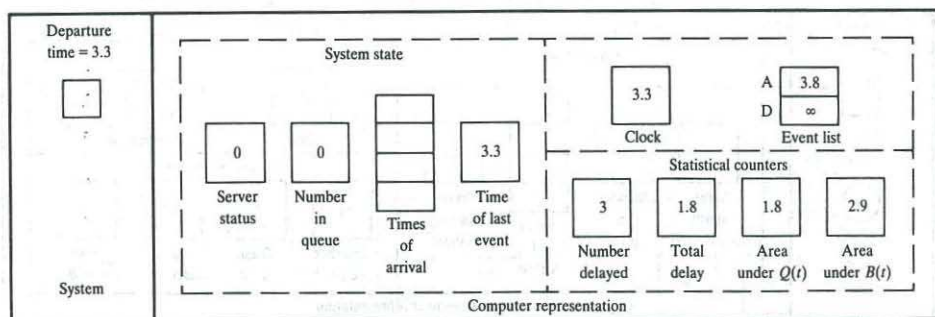


(e)

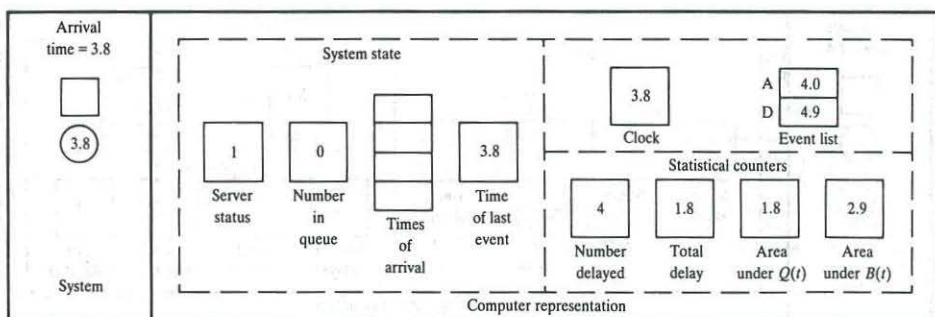
FIGURE 1.7
(Continued.)



(f)

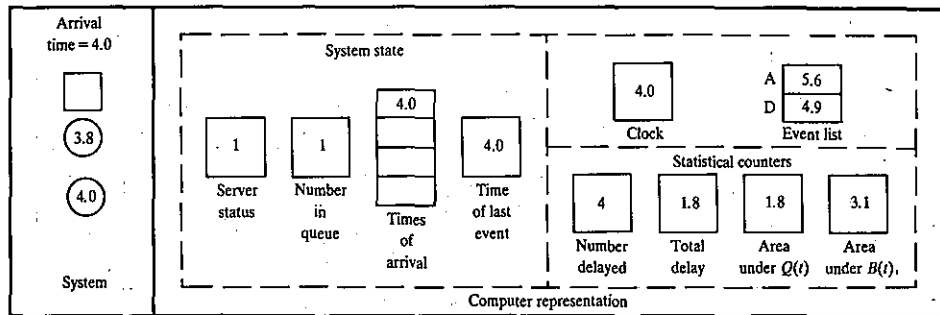


(g)

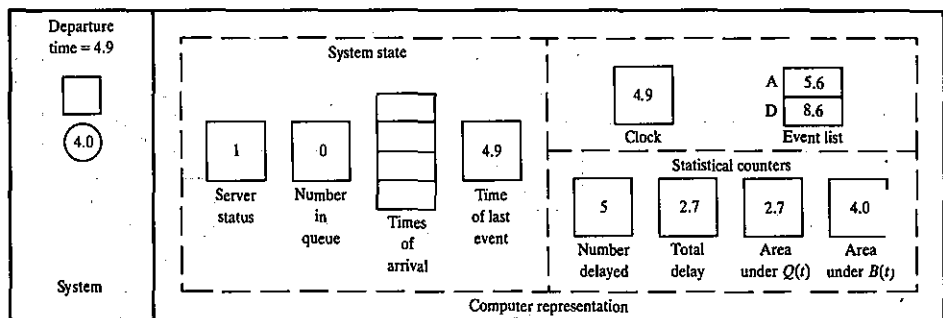


(h)

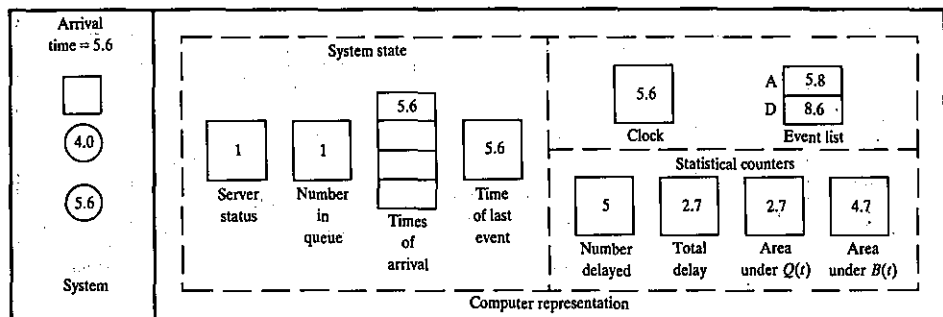
FIGURE 1.7
(Continued.)



(i)

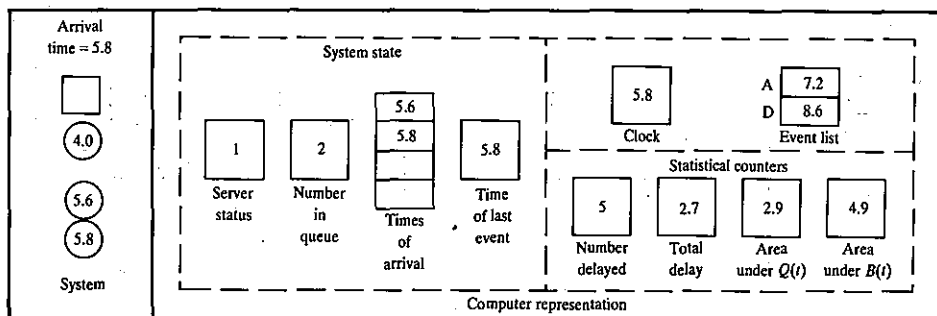


(j)

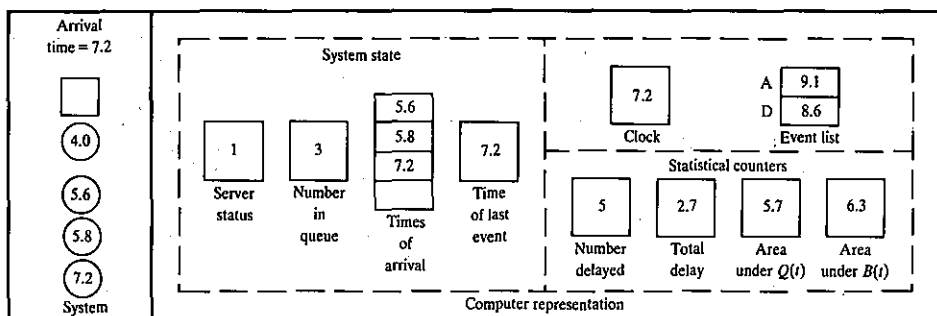


(k)

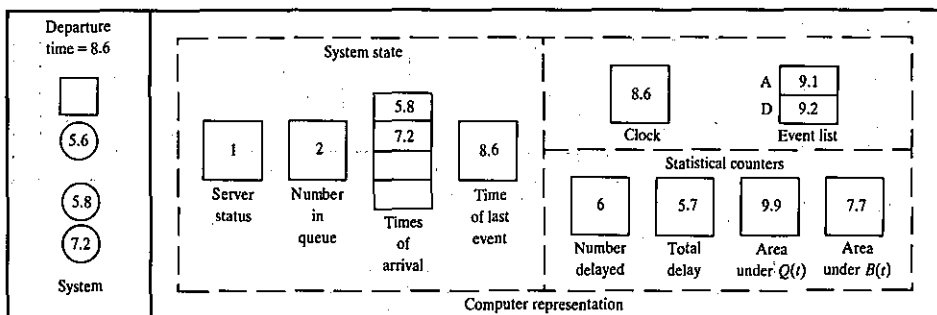
FIGURE 1.7
(Continued.)



(l)



(m)



(n)

FIGURE 1.7
(Continued.)

initially the system is empty of customers and the server is idle, as depicted in the "system" picture of Fig. 1.7a. The model state variables are initialized to represent this: server status is zero [we use 0 to represent an idle server and 1 to represent a busy server, similar to the definition of the $B(t)$ function], and the number of customers in the queue is zero. There is a one-dimensional array to store the times of arrival of customers currently in the queue; this array is initially empty, and as the simulation progresses its length will grow and shrink. The time of the last (most recent) event is initialized to zero, so that at the time of the first event (when it is used), it will have its correct value. The simulation clock is set to zero, and the event list, giving the times of the next occurrence of each of the event types, is initialized as follows. The time of the first arrival is $0 + A_1 = 0.4$, and is denoted by "A" next to the event list. Since there is no customer in service, it does not even make sense to talk about the time of the next departure ("D" by the event list), and we know that the first event will be the initial customer arrival at time 0.4. However, the simulation progresses in general by looking at the event list and picking the smallest value from it to determine what the next event will be, so by scheduling the next departure to occur at time ∞ (or a very large number in a computer program), we effectively eliminate the departure event from consideration and force the next event to be an arrival. (This is sometimes called poisoning the departure event.) Finally, the four statistical counters are initialized to zero. When all initialization is done, control is returned to the main program, which then calls the timing routine to determine the next event. Since $0.4 < \infty$, the next event will be an arrival at time 0.4, and the timing routine advances the clock to this time, then passes control back to the main program with the information that the next event is to be an arrival.

$t = 0.4$: *Arrival of customer 1.* At time 0.4, the main program passes control to the arrival routine to process the arrival of the first customer. Figure 1.7b shows the system and its computer representation after all changes have been made to process this arrival. Since this customer arrived to find the server idle (status equal to 0), he begins service immediately and has a delay in queue of $D_1 = 0$ (which does count as a delay). The server status is set to 1 to represent that the server is now busy, but the queue itself is still empty. The clock has been advanced to the current time, 0.4, and the event list is updated to reflect this customer's arrival: The next arrival will be $A_2 = 1.2$ time units from now, at time $0.4 + 1.2 = 1.6$, and the next departure (the service completion of the customer now arriving) will be $S_1 = 2.0$ time units from now, at time $0.4 + 2.0 = 2.4$. The number delayed is incremented to 1 (when this reaches $n = 6$, the simulation will end), and $D_1 = 0$ is added into the total delay (still at zero). The

area under $Q(t)$ is updated by adding in the product of the *previous* value (i.e., the level it had between the last event and now) of $Q(t)$ (0 in this case) times the width of the interval of time from the last event to now, $t - (\text{time of last event}) = 0.4 - 0$ in this case. Note that the time of the last event used here is its *old* value (0), before it is updated to its new value (0.4) in this event routine. Similarly, the area under $B(t)$ is updated by adding in the product of its previous value (0) times the width of the interval of time since the last event. [Look back at Figs. 1.5 and 1.6 to trace the accumulation of the areas under $Q(t)$ and $B(t)$.] Finally, the time of the last event is brought up to the current time, 0.4, and control is passed back to the main program. It invokes the timing routine, which scans the event list for the smallest value, and determines that the next event will be another arrival at time 1.6; it updates the clock to this value and passes control back to the main program with the information that the next event is an arrival.

$t = 1.6$: *Arrival of customer 2.* At this time we again enter the arrival routine, and Fig. 1.7c shows the system and its computer representation after all changes have been made to process this event. Since this customer arrives to find the server busy (status equal to 1 upon her arrival), she must queue up in the first location in the queue, her time of arrival is stored in the first location in the array, and the number-in-queue variable rises to 1. The time of the next arrival in the event list is updated to $A_3 = 0.5$ time units from now, $1.6 + 0.5 = 2.1$; the time of the next departure is not changed, since its value of 2.4 is the departure time of customer 1, who is still in service at this time. Since we are not observing the end of anyone's delay in queue, the number-delayed and total-delay variables are unchanged. The area under $Q(t)$ is increased by 0 [the previous value of $Q(t)$] times the time since the last event, $1.6 - 0.4 = 1.2$. The area under $B(t)$ is increased by 1 [the previous value of $B(t)$] times this same interval of time, 1.2. After updating the time of the last event to now, control is passed back to the main program and then to the timing routine, which determines that the next event will be an arrival at time 2.1.

$t = 2.1$: *Arrival of customer 3.* Once again the arrival routine is invoked, as depicted in Fig. 1.7d. The server stays busy, and the queue grows by one customer, whose time of arrival is stored in the queue array's second location. The next arrival is updated to $t + A_4 = 2.1 + 1.7 = 3.8$, and the next departure is still the same, as we are still waiting for the service completion of customer 1. The delay counters are unchanged, since this is not the end of anyone's delay in queue, and the two area accumulators are updated by adding in 1 [the previous values of both $Q(t)$ and $B(t)$] times the time since the last event, $2.1 - 1.6 = 0.5$. After bringing the time of the last event up to the present, we go back to the main program and invoke the timing

routine, which looks at the event list to determine that the next event will be a departure at time 2.4, and updates the clock to that time.

t = 2.4: Departure of customer 1. Now the main program invokes the departure routine, and Fig. 1.7e shows the system and its representation after this occurs. The server will maintain its busy status, since customer 2 moves out of the first place in queue and into service. The queue shrinks by one, and the time-of-arrival array is moved up one place, to represent that customer 3 is now first in line. Customer 2, now entering service, will require $S_2 = 0.7$ times units, so the time of the next departure (that of customer 2) in the event list is updated to S_2 time units from now, or at time $2.4 + 0.7 = 3.1$; the time of the next arrival (that of customer 4) is unchanged, since this was scheduled earlier at the time of customer 3's arrival, and we are still waiting at this time for customer 4 to arrive. The delay statistics are updated, since at this time customer 2 is entering service and is completing her delay in queue. Here we make use of the time-of-arrival array, and compute the second delay as the current time minus the second customer's time of arrival, or $D_2 = 2.4 - 1.6 = 0.8$. (Note that the value of 1.6 was stored in the first location in the time-of-arrival array before it was changed, so this delay computation would have to be done before advancing the times of arrival in the array.) The area statistics are updated by adding in $2 \times (2.4 - 2.1)$ for $Q(t)$ [note that the previous value of $Q(t)$ was used], and $1 \times (2.4 - 2.1)$ for $B(t)$. The time of the last event is updated, we return to the main program, and the timing routine determines that the next event is a departure at time 3.1.

t = 3.1: Departure of customer 2. The changes at this departure are similar to those at the departure of customer 1 at time 2.4 just discussed. Note that we observe another delay in queue, and that after this event is processed the queue is again empty, but the server is still busy.

t = 3.3: Departure of customer 3. Again, the changes are similar to those in the above two departure events, with one important exception: Since the queue is now empty, the server becomes idle and we must set the next departure time in the event list to ∞ , since the system now looks the same as it did at time 0 and we want to force the next event to be the arrival of customer 4.

t = 3.8: Arrival of customer 4. Since this customer arrives to find the server idle, he has a delay of zero (i.e., $D_4 = 0$) and goes right into service. Thus, the changes here are very similar to those at the arrival of the first customer at time $t = 0.4$.

The remaining six event times are depicted in Figs. 1.7i through 1.7n, and readers should work through these to be sure they understand why the variables and arrays are as they appear; it may be helpful to follow along in the

plots of $Q(t)$ and $B(t)$ in Figs. 1.5 and 1.6. With the departure of customer 5 at time $t = 8.6$, customer 6 leaves the queue and enters service, at which time the number delayed reaches 6 (the specified value of n) and the simulation ends. At this point, the main program would invoke the report generator to compute the final output measures [$\hat{d}(6) = 5.7/6 = 0.95$, $\hat{q}(6) = 9.9/8.6 = 1.15$, and $\hat{u}(6) = 7.7/8.6 = 0.90$] and write them out.

A few specific comments about the above example illustrating the logic of a simulation should be made:

- Perhaps the key element in the dynamics of a simulation is the interaction between the simulation clock and the event list. The event list is maintained, and the clock jumps to the next event, as determined by scanning the event list at the end of each event's processing for the smallest (i.e., next) event time. This is how the simulation progresses through time.
- While processing an event, no "simulated" time passes. However, even though time is standing still for the model, care must be taken to process updates of the state variables and statistical counters in the appropriate order. For example, it would be incorrect to update the number in queue before updating the area-under- $Q(t)$ counter, since the height of the rectangle to be used is the *previous* value of $Q(t)$ [before the effect of the current event on $Q(t)$ has been implemented]. Similarly, it would be incorrect to update the time of the last event before updating the area accumulators. Yet another type of error would result if the queue list were changed at a departure before the delay of the first customer in queue were computed, since his time of arrival to the system would be lost.
- It is sometimes easy to overlook contingencies that seem out of the ordinary but that nevertheless must be accommodated. For example, it would be easy to forget that a departing customer could leave behind an empty queue, necessitating that the server be idled and the departure event again be eliminated from consideration. Also, termination conditions are often more involved than they might seem at first sight; in the above example, the simulation stopped in what seems to be the "usual" way, after a departure of one customer, allowing another to enter service and contribute the last delay needed, but the simulation *could* actually have ended instead with an arrival event—how?
- In some simulations it can happen that two (or more) entries in the event list are tied for smallest, and a decision rule must be incorporated to break such *time ties* (this happens with the inventory simulation considered later in Sec. 1.5). The tie-breaking rule can affect the results of the simulation, so must be chosen in accordance with how the system is to be modeled. In many simulations, however, we can ignore the possibility of ties, since the use of continuous random variables may make their occurrence an event with probability zero. In the above model, for example, if the interarrival-time or service-time distribution is continuous, then a time tie in the event list is a probability-zero event.

The above exercise is intended to illustrate the changes and data structures involved in carrying out a discrete-event simulation from the event-scheduling point of view, and contains most of the important ideas needed for more complex simulations of this type. The interarrival and service times used could have been drawn from a random-number table of some sort, constructed to reflect the desired probability distributions; this would result in what might be called a *hand simulation*, which in principle could be carried out to any length. The tedium of doing this should now be clear, so we will next turn to the use of computers (which are not easily bored) to carry out the arithmetic and bookkeeping involved in longer or more complex simulations.

1.4.3 Program Organization and Logic

In this section we set up the necessary ingredients for the programs to simulate the single-server queueing system in FORTRAN (Sec. 1.4.4), Pascal (Sec. 1.4.5), and C (Sec. 1.4.6). The organization and logic described in this section apply for all three languages, so the reader need only go through one of Secs. 1.4.4, 1.4.5, or 1.4.6, according to language preference.

There are several reasons for choosing a general-purpose language such as FORTRAN, Pascal, or C, rather than a more powerful high-level simulation language, for introducing computer simulation at this point:

- By learning to simulate in a general-purpose language, in which one must pay attention to every detail, there will be a greater understanding of how simulations actually operate, and thus less chance of conceptual errors if a switch is later made to a high-level simulation language.
- Despite the fact that there are now several very good and powerful simulation languages available (see Chap. 3), it is often necessary to write at least parts of complex simulations in a general-purpose language if the specific, detailed logic of complex systems is to be represented faithfully.
- General-purpose languages are widely available, and entire simulations are sometimes still written in this way.

It is not our purpose in this book to teach any particular simulation language in detail, although we survey several in Chap. 3. With the understanding promoted by our more general approach and by going through our simulations in this and the next chapter, the reader should find it easier to learn a specialized simulation language. Appendix 1C contains details on the particular computers and compilers used for the examples in this and the next chapter.

The single-server queueing model that we will simulate in the following three sections differs in two respects from the model used in the previous section:

- The simulation will end when $n = 1000$ delays in queue have been completed, rather than $n = 6$, in order to collect more data (and maybe to

impress the reader with the patience of computers, since we have just slugged it out by hand in the $n = 6$ case in the preceding section). It is important to note that this change in the stopping rule changes the model itself, in that the output measures are defined relative to the stopping rule; hence the “ n ” in the notation for the quantities $d(n)$, $q(n)$, and $u(n)$ being estimated.

- The interarrival and service times will now be modeled as independent random variables from exponential distributions with mean 1 minute for the interarrival times and mean 0.5 minute for the service times. The exponential distribution with mean β (any positive real number) is continuous, with probability density function

$$f(x) = \frac{1}{\beta} e^{-x/\beta} \quad \text{for } x \geq 0$$

(See Chaps. 4 and 6 for more information on density functions in general, and on the exponential distribution in particular.) We make this change here since it is much more common to generate input quantities (which drive the simulation) such as interarrival and service times from specified distributions than to assume that they are “known” as we did in the preceding section. The choice of the exponential distribution with the above particular values of β is essentially arbitrary, and is made primarily because it is easy to generate exponential random variates on a computer. (Actually, the assumption of exponential interarrival times is often quite realistic; assuming exponential service times, however, is seldom plausible.) Chapter 6 addresses in detail the important issue of how one chooses distribution forms and parameters for modeling simulation input random variables.

- The single-server queue with exponential interarrival and service times is commonly called the *M/M/1 queue*, as discussed in App. 1B.

To simulate this model we need a way to generate random variates from an exponential distribution. The subprograms used by the FORTRAN, Pascal, and C codes all operate in the same way, which we will now develop. First, a *random-number generator* (discussed in detail in Chap. 7) is invoked to generate a variate U that is distributed (continuously) uniformly between 0 and 1; this distribution will henceforth be referred to as $U(0, 1)$ and has probability density function

$$f(x) = \begin{cases} 1 & \text{if } 0 \leq x \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

It is easy to show that the probability that a $U(0, 1)$ random variable falls in any subinterval $[x, x + \Delta x]$ contained in the interval $[0, 1]$ is (uniformly) Δx (see Sec. 6.2.2). The $U(0, 1)$ distribution is fundamental to simulation modeling because, as we shall see in Chap. 8, a random variate from any distribution can be generated by first generating one or more $U(0, 1)$ random variates and then performing some kind of transformation. After obtaining U , we shall take

the natural logarithm of it, multiply the result by β , and finally change the sign to return what we will show to be an exponential random variate with mean β , that is, $-\beta \ln U$.

To see why this algorithm works, recall that the (cumulative) distribution function of a random variable X is defined, for any real x , to be $F(x) = P(X \leq x)$ (Chap. 4 contains a review of basic probability theory). If X is exponential with mean β , then

$$\begin{aligned} F(x) &= \int_0^x \frac{1}{\beta} e^{-t/\beta} dt \\ &= 1 - e^{-x/\beta} \end{aligned}$$

for any real $x \geq 0$, since the probability density function of the exponential distribution at the argument $t \geq 0$ is $(1/\beta)e^{-t/\beta}$. To show that our method is correct, we can try to verify that the value it returns will be less than or equal to x (any nonnegative real number), with probability $F(x)$ given above:

$$\begin{aligned} P(-\beta \ln U \leq x) &= P\left(\ln U \geq -\frac{x}{\beta}\right) \\ &= P(U \geq e^{-x/\beta}) \\ &= P(e^{-x/\beta} \leq U \leq 1) \\ &= 1 - e^{-x/\beta} \end{aligned}$$

The first line in the above is obtained by dividing through by $-\beta$ (recall that $\beta > 0$, so $-\beta < 0$ and the inequality reverses), the second line is obtained by exponentiating both sides (the exponential function is monotone increasing, so the inequality is preserved), the third line is just rewriting, together with knowing that U is in $[0, 1]$ anyway, and the last line follows since U is $U(0, 1)$, and the interval $[e^{-x/\beta}, 1]$ is contained within the interval $[0, 1]$. Since the last line is $F(x)$ for the exponential distribution, we have verified that our algorithm is correct. Chapter 8 discusses how to generate random variates and processes in general.

In our programs, we prefer to use a particular method for random-number generation to obtain the variate U described above, as expressed in the FORTRAN, Pascal, and C codes of Figs. 7.5 through 7.8 in App. 7A of Chap. 7. While most compilers do have some kind of built-in random-number generator, many of these are of extremely poor quality and should not be used; this issue is discussed fully in Chap. 7.

It is convenient (if not the most computationally efficient) to modularize the programs into several subprograms to clarify the logic and interactions, as discussed in general in Sec. 1.3.2. In addition to a main program, the simulation program includes routines for initialization, timing, report generation, and generating exponential random variates, as in Fig. 1.3. It also simplifies matters if we write a separate routine to update the continuous-time

statistic, being the accumulated areas under the $Q(t)$ and $B(t)$ curves. The most important action, however, takes place in the routines for the events, which we number as follows:

Event description	Event type
Arrival of a customer to the system	1
Departure of a customer from the system after completing service	2

As the logic of these event routines is independent of the particular language to be used, we shall discuss it here. Figure 1.8 contains a flowchart for

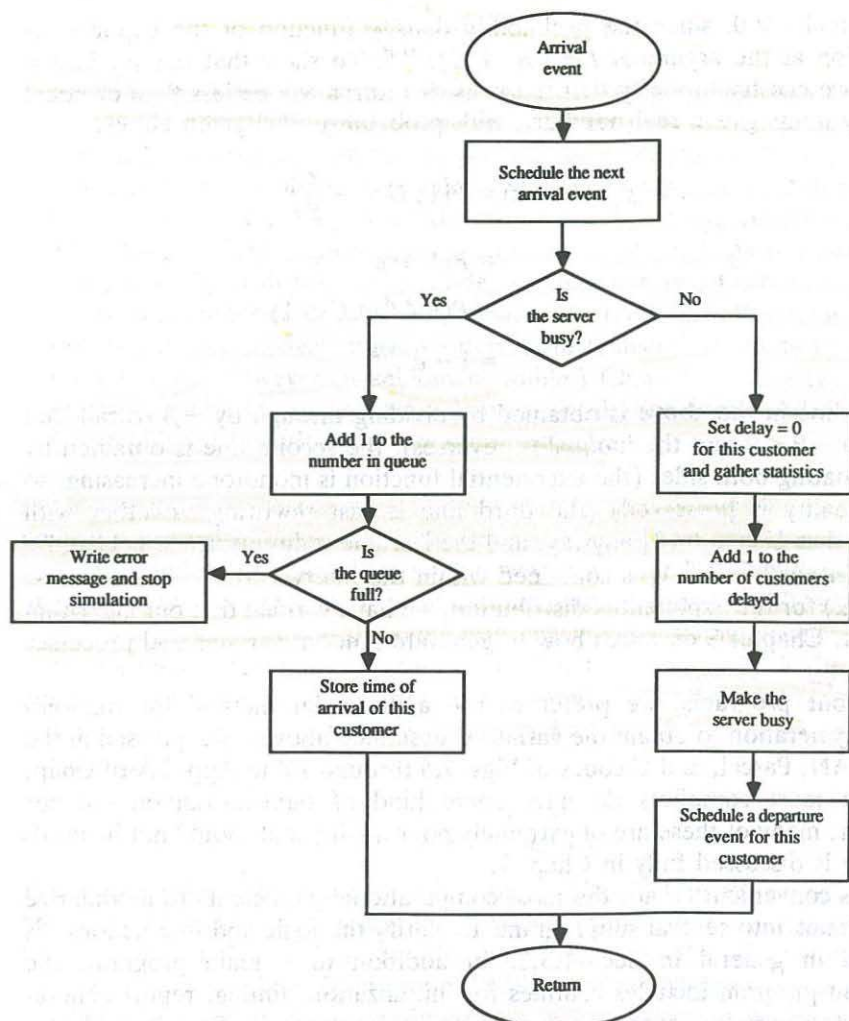


FIGURE 1.8
Flowchart for arrival routine, queuing model.

the arrival event. First, the time of the next arrival in the future is generated and placed in the event list. Then a check is made to determine whether the server is busy. If so, the number of customers in the queue is incremented by one, and we ask whether the storage space allocated to hold the queue is already full (see the code in Sec. 1.4.4, 1.4.5, or 1.4.6 for details). If the queue

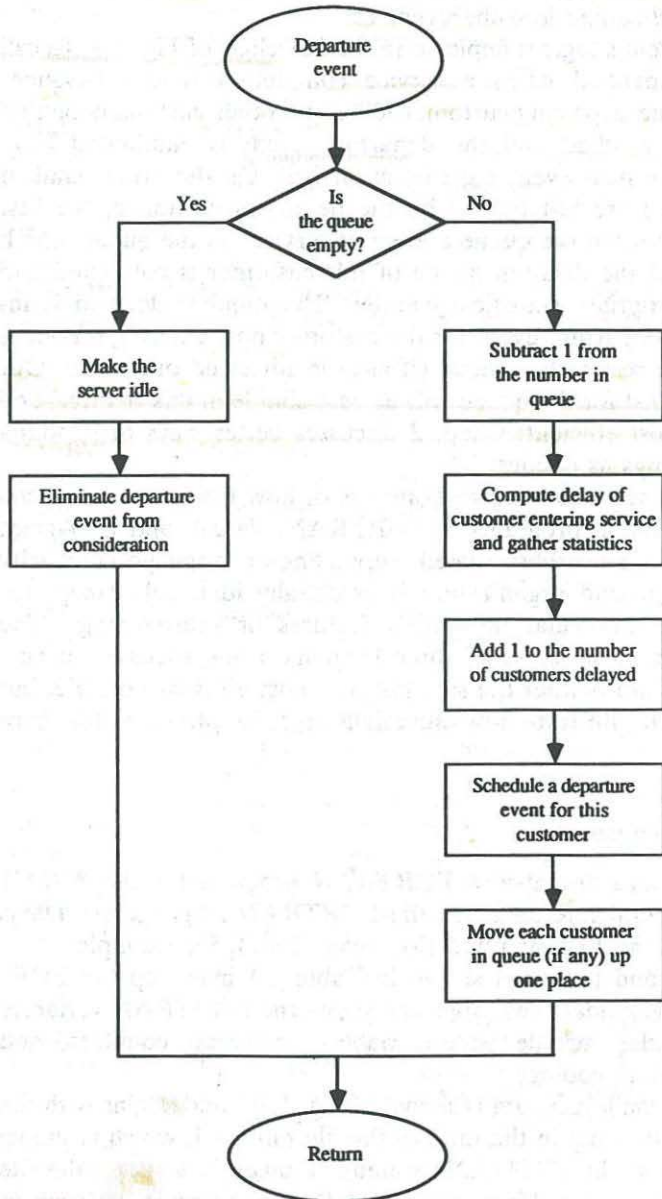


FIGURE 1.9
Flowchart for departure routine, queueing model.

is already full, an error message is produced and the simulation is stopped; if there is still room in the queue, the arriving customer's time of arrival is put at the (new) end of the queue. On the other hand, if the arriving customer finds the server idle, then this customer has a delay of zero, which is counted as a delay, and the number of customer delays completed is incremented by one. The server must be made busy, and the time of departure from service of the arriving customer is scheduled into the event list.

The departure event's logic is depicted in the flowchart of Fig. 1.9. Recall that this routine is invoked when a service completion (and subsequent departure) occurs. If the departing customer leaves no other customers behind in queue, the server is idled and the departure event is eliminated from consideration, since the next event must be an arrival. On the other hand, if one or more customers are left behind by the departing customer, the first customer in queue will leave the queue and enter service, so the queue length is reduced by one, and the delay in queue of this customer is computed and registered in the appropriate statistical counter. The number delayed is increased by one, and a departure event for the customer now entering service is scheduled. Finally, the rest of the queue (if any) is advanced one place. Our implementation of the list for the queue will be very simple in this chapter, and is certainly not the most efficient; Chap. 2 discusses better ways of handling lists to model such things as queues.

In the next three sections we give examples of how the above setup can be used to write simulation programs in FORTRAN, Pascal, and C. Again, only one of these sections need be studied, depending on language familiarity or preference; the logic and organization is essentially identical, except for changes dictated by a particular language's features or shortcomings. The results (which were identical across all three languages) are discussed in Sec. 1.4.7. These programs are neither the simplest nor most efficient possible, but were instead designed to illustrate how one might organize programs for more complex simulations.

1.4.4 FORTRAN Program

This section presents and describes a FORTRAN program for the $M/M/1$ queue simulation. General references on the FORTRAN language are Davis and Hoffmann (1988) and Koffman and Friedman (1987), for example.

The subroutines and functions shown in Table 1.1 make up the FORTRAN program for this model. The table also shows the FORTRAN variables used (modeling variables include state variables, statistical counters, and variables used to facilitate coding).

The code for the main program is shown in Fig. 1.10, and begins with the INCLUDE statement to bring in the lines in the file mm1.dcl, which is shown in Fig. 1.11. The action the INCLUDE statement takes is to copy the file named between the quotes (in this case, mm1.dcl) into the main program in place of the INCLUDE statement. The file mm1.dcl contains "declarations" of

TABLE 1.1
Subroutines, functions, and FORTRAN variables for the queueing model

Subprogram	Purpose
INIT	Initialization routine
TIMING	Timing routine
ARRIVE	Event routine to process type 1 events
DEPART	Event routine to process type 2 events
REPORT	Generates report when simulation ends
UPTAVG	Updates continuous-time area-accumulator statistics just before each event occurrence
EXPON(RMEAN)	Function to generate an exponential random variate with mean RMEAN
RAND(1)	Function to generate a uniform random variate between 0 and 1 (shown in Fig. 7.5)
Variable	Definition
Input parameters:	
MARRVT	Mean interarrival time (=1.0 here)
MSERV	Mean service time (=0.5)
TOTCUS	Total number, n , of customer delays to be observed (=1000)
Modeling variables:	
ANIQ	Area under the number-in-queue function $[Q(t)]$ so far
AUTIL	Area under the server-status function $[B(t)]$ so far
BUSY	Mnemonic for server busy (=1)
DELAY	Delay in queue of a customer
IDLE	Mnemonic for server idle (=0)
MINTNE	Used by TIMING to determine which event is next
NEVNTS	Number of event types for this model, used by TIMING routine (=2 here)
NEXT	Event type (1 or 2 here) of the next event to occur (determined by TIMING routine)
NIQ	Number of customers currently in queue
NUMCUS	Number of customers who have completed their delays so far
QLIMIT	Number of storage locations for the queue TARRVL (=100)
RMEAN	Mean of the exponential random variate to be generated (used by EXPON)
SERVER	Server status (0 for idle, 1 for busy)
TARRVL(I)	Time of arrival of the customer now I th in queue (dimensioned to have 100 places)
TIME	Simulation clock
TLEVNT	Time of the last (most recent) event
TNE(I)	Time of the next event of type I ($I = 1, 2$), part of event list
TOTDEL	Total of the delays completed so far
TSLE	Time since the last event (used by UPTAVG)
U	Random variate distributed uniformly between 0 and 1
Output variables:	
AVGDEL	Average delay in queue $[\hat{d}(n)]$
AVGNIQ	Time-average number in queue $[\hat{q}(n)]$
UTIL	Server utilization $[\hat{u}(n)]$

```

*      Main program for single-server queueing system.
*      Bring in declarations file.
      INCLUDE 'mml.dcl'
*      Open input and output files.
      OPEN (5, FILE = 'mml.in')
      OPEN (6, FILE = 'mml.out')
*      Specify the number of event types for the timing routine.
      NEVNTS = 2
*      Set mnemonics for server's being busy and idle.
      BUSY = 1
      IDLE = 0
*      Read input parameters.
      READ (5,*) MARRVT, MSERVT, TOTCUS
*      Write report heading and input parameters.
      WRITE (6,2010) MARRVT, MSERVT, TOTCUS
2010 FORMAT (' Single-server queueing system'//
&          ' Mean interarrival time',F11.3,' minutes'//
&          ' Mean service time',F16.3,' minutes'//
&          ' Number of customers',I14//)
*      Initialize the simulation.
      CALL INIT
*      Determine the next event.
10 CALL TIMING
*      Update time-average statistical accumulators.
      CALL UPTAVG
*      Call the appropriate event routine.
      GO TO (20, 30), NEXT
20   CALL ARRIVE
      GO TO 40
30   CALL DEPART
*      If the simulation is over, call the report generator and end the
*      simulation. If not, continue the simulation.
40 IF (NUMCUS .LT. TOTCUS) GO TO 10
      CALL REPORT
      CLOSE (5)
      CLOSE (6)
      STOP
      END

```

FIGURE 1.10

FORTTRAN code for the main program, queueing model.

```

INTEGER QLIMIT
PARAMETER (QLIMIT = 100)
INTEGER BUSY, IDLE, NEVNTS, NEXT, NIQ, NUMCUS, SERVER, TOTCUS
REAL ANIQ, AUTIL, MARRVT, MSERVT, TARRVL(QLIMIT), TIME, TLEVNT, TNE(2),
& TOTDEL
REAL EXPON
COMMON /MODEL/ ANIQ, AUTIL, BUSY, IDLE, MARRVT, MSERVT, NEVNTS, NEXT, NIQ,
& NUMCUS, SERVER, TARRVL, TIME, TLEVNT, TNE, TOTCUS, TOTDEL

```

FIGURE 1.11

FORTRAN code for the declarations file (mm1.dcl), queueing model.

the variables and arrays to be INTEGER or REAL, the COMMON block MODEL, and the PARAMETER value QLIMIT = 100, our guess (which may have to be adjusted by trial and error) as to the longest the queue will ever get. All of the statements in the file mm1.dcl must appear at the beginning of almost all subprograms in the simulation, and using the INCLUDE statement simply makes it easier to do this, and to make any necessary changes. The variables in the COMMON block MODEL are those we want to be global; i.e., variables in the block will be known and accessible to all subprograms that contain this COMMON statement. Variables not in COMMON will be *local* to the subprogram in which they appear. Also, we have adopted the convention of explicitly declaring the type (REAL or INTEGER) of all variables, arrays, and functions regardless of whether the first letter of the variable would default to its desired type according to the FORTRAN convention. Next, the input data file (called mm1.in) is opened and assigned to unit 5, and the file to contain the output (called mm1.out) is opened and assigned to unit 6. (We do not show the contents of the file mm1.in, since it is only a single line consisting of the numbers 1.0, 0.5, and 1000, separated by any number of blanks.) The number of event types for the simulation, NEVNTS, is initialized to 2 for this model, and the mnemonic constants BUSY and IDLE are set to use with the SERVER status variable, for code readability. The input parameters are then read in free format. After writing a report heading and echoing the input parameters (as a check that they were read correctly), the initialization routine INIT is called. The timing routine, TIMING, is then called to determine the event type, NEXT, of the next event to occur and to advance the simulation clock, TIME, to its time. Before processing this event, subroutine UPTAVG is called to update the areas under the $Q(t)$ and $B(t)$ curves, for the continuous-time statistics; UPTAVG also brings the time of the last event, TLEVNT, up to the present. By doing this at this time we automatically update these areas before processing each event. Then a "computed GO TO statement," based on NEXT, is used to pass control to the appropriate event routine. If NEXT = 1, event routine ARRIVE is called (at statement label 20) to process the arrival of a customer. If NEXT = 2, event routine DEPART (at statement label 30) is called to process the departure of a customer after completing service. After control is returned to the main program from ARRIVE or DEPART, a check is made (at statement label 40) to see whether the number of customers who

have completed their delays, NUMCUS (which is incremented by 1 after each customer completes his or her delay), is still (strictly) less than the number of customers whose delays we want to observe, TOTCUS. If so, TIMING is called to continue the simulation. If the specified number of delays has been observed, the report generator, REPORT, is called to compute and write estimates of the desired measures of performance. Finally, the input and output files are closed (a precautionary measure that is not required on many systems), and the simulation run is terminated.

Code for subroutine INIT is given in Fig. 1.12. Note that the same declarations file, mml.dcl, is brought in here by the INCLUDE statement. Each statement in INIT corresponds to an element of the computer representation in Fig. 1.7a. Note that the time of the first arrival, TNE(1), is determined by adding an exponential random variate with mean MARRVT, namely, EXPON(MARRVT), to the simulation clock, TIME = 0. (We explicitly used TIME in this statement, although it has a value of 0, to show the general form of a statement to determine the time of a future event.) Since no customers are present at TIME = 0, the time of the next departure, TNE(2), is set to 1.0E + 30 (FORTRAN notation for 10^{30}), guaranteeing that the first event will be an arrival.

Subroutine TIMING is given in Fig. 1.13. The program compares TNE(1), TNE(2), ..., TNE(NEVNTS) and sets NEXT equal to the event type whose time of occurrence is the smallest. (Note that NEVNTS is set in the main

```

SUBROUTINE INIT
  INCLUDE 'mml.dcl'

*   Initialize the simulation clock.

  TIME = 0.0

*   Initialize the state variables.

  SERVER = IDLE
  NIQ = 0
  TLEVNT = 0.0

*   Initialize the statistical counters.

  NUMCUS = 0
  TOTDEL = 0.0
  ANIQ = 0.0
  AUTIL = 0.0

*   Initialize event list. Since no customers are present, the
*   departure (service completion) event is eliminated from
*   consideration.

  TNE(1) = TIME + EXPON(MARRVT)
  TNE(2) = 1.0E+30

  RETURN
END

```

FIGURE 1.12
FORTRAN code for subroutine INIT, queueing model.


```

SUBROUTINE TIMING
INCLUDE 'mm1.dcl'
INTEGER I
REAL MINTNE

MINTNE = 1.0E+29
NEXT = 0

*   Determine the event type of the next event to occur.

DO 10 I = 1, NEVNTS
  IF (TNE(I) .LT. MINTNE) THEN
    MINTNE = TNE(I)
    NEXT = I
  END IF
10 CONTINUE

*   Check to see whether the event list is empty.

IF (NEXT .EQ. 0) THEN

*       The event list is empty, so stop the simulation.

  WRITE (6,2010) TIME
2010  FORMAT (' Event list empty at time',F10.3)
  STOP

  END IF

*   The event list is not empty, so advance the simulation clock.

  TIME = MINTNE

  RETURN
END

```

FIGURE 1.13
 FORTRAN code for subroutine TIMING, queueing model.

program.) In case of ties, the lowest-numbered event type is chosen. Then the simulation clock is advanced to the time of occurrence of the chosen event type, MINTNE. The program is complicated slightly by an error check for the event list's being empty, which we define to mean that all events are scheduled to occur at $TIME = 10^{30}$. If this is ever the case (as indicated by $NEXT = 0$), an error message is produced along with the current clock time (as a possible debugging aid), and the simulation is terminated.

The code for event routine ARRIVE is in Fig. 1.14, and follows the language-independent discussion as given in Sec. 1.4.3 and in the flowchart of Fig. 1.8. Note that TIME is the time of arrival of the customer who is just now arriving, and that the queue-overflow check is made by asking whether NIQ is now greater than QLIMIT, the length for which TARRVL was dimensioned.

Event routine DEPART, whose code is shown in Fig. 1.15, is called from the main program when a service completion (and subsequent departure) occurs; the logic for it was discussed in Sec. 1.4.3, with a flowchart in Fig. 1.9. Note that if the statement $TNE(2) = 1.0E + 30$ just before the ELSE were omitted, the program would get into an infinite loop. (Why?) Advancing the

```

SUBROUTINE ARRIVE
INCLUDE 'mml.dcl'
REAL DELAY

*   Schedule next arrival.

TNE(1) = TIME + EXPON(MARRVT)

*   Check to see whether server is busy.

IF (SERVER .EQ. BUSY) THEN

*       Server is busy, so increment number of customers in queue.

      NIQ = NIQ + 1

*       Check to see whether an overflow condition exists.

      IF (NIQ .GT. QLIMIT) THEN

*           The queue has overflowed, so stop the simulation.

2010      WRITE (6,2010) TIME
          FORMAT (' Overflow of the array TARRVL at time',F10.3)
          STOP

      END IF

*       There is still room in the queue, so store the time of arrival
*       of the arriving customer at the (new) end of TARRVL.

      TARRVL(NIQ) = TIME

      ELSE

*       Server is idle, so arriving customer has a delay of zero. (The
*       following two statements are for program clarity and do not
*       affect the results of the simulation.)

      DELAY = 0.0
      TOTDEL = TOTDEL + DELAY

*       Increment the number of customers delayed, and make server
*       busy.

      NUMCUS = NUMCUS + 1
      SERVER = BUSY

*       Schedule a departure (service completion).

      TNE(2) = TIME + EXPON(MSERVT)

      END IF

      RETURN
      END

```

FIGURE 1.14

FORTRAN code for subroutine ARRIVE, queueing model.

```

SUBROUTINE DEPART
  INCLUDE 'mm1.dcl'
  INTEGER I
  REAL DELAY

*   Check to see whether the queue is empty.

  IF (NIQ .EQ. 0) THEN

*   The queue is empty so make the server idle and eliminate the
*   departure (service completion) event from consideration.

    SERVER = IDLE
    TNE(2) = 1.0E+30

  ELSE

*   The queue is nonempty, so decrement the number of customers in
*   queue.

    NIQ = NIQ - 1

*   Compute the delay of the customer who is beginning service and
*   update the total delay accumulator.

    DELAY = TIME - TARRVL(1)
    TOTDEL = TOTDEL + DELAY

*   Increment the number of customers delayed, and schedule
*   departure.

    NUMCUS = NUMCUS + 1
    TNE(2) = TIME + EXPON(MSERVT)

*   Move each customer in queue (if any) up one place.

    DO 10 I = 1, NIQ
10    TARRVL(I) = TARRVL(I + 1)

  END IF

  RETURN
END

```

FIGURE 1.15

FORTTRAN code for subroutine DEPART, queueing model.

rest of the queue (if any) one place by DO loop 10 ensures that the arrival time of the next customer entering service (after being delayed in queue) will always be stored in TARRVL(1). Note that if the queue were now empty (i.e., the customer who just left the queue and entered service had been the only one in queue), then NIQ would be equal to 0, and this DO loop would not be executed at all since the beginning value of the DO loop index, I, starts out at a value (1) that would already exceed its final value (NIQ = 0); this is a feature of FORTRAN 77 (which we use here, as detailed in App. 1C) that may not be shared by older versions of FORTRAN. (Managing the queue in this simple way, by moving the arrival times up physically, is certainly inefficient; we return to this issue in Chap. 2.) A final comment about DEPART concerns the

subtraction of TARRVL(1) from the clock value, TIME, to obtain the delay in queue. If the simulation is to run for a long period of (simulated) time, both TIME and TARRVL(1) would become very large numbers in comparison with the difference between them; thus, since they are both stored as floating-point (REAL) numbers with finite accuracy, there is potentially a serious loss of precision when doing this subtraction. For this reason, it may be necessary to make both TIME and the TARRVL array DOUBLE PRECISION if we want to run this simulation out for a long period of time.

The code for subroutine REPORT, called when the termination check in the main program determines that the simulation is over, is given in Fig. 1.16. The average delay, AVGDDEL, is computed by dividing the total of the delays by the number of customers whose delays were observed, and the time-average number in queue, AVGNIQ, is obtained by dividing the area under $Q(t)$, now updated to the end of the simulation (since UPTAVG is called from the main program before processing either an arrival or departure, one of which will end the simulation), by the clock value at termination. The server utilization, UTIL, is computed by dividing the area under $B(t)$ by the final clock time, and all three measures are written out. We also write out the final clock value itself, to see how long it took to observe the 1000 delays.

Subroutine UPTAVG is shown in Fig. 1.17. This subroutine is called just before processing each event (of any type) and updates the areas under the two functions needed for the continuous-time statistics; this routine is separate for coding convenience only, and is *not* an event routine. The time since the last event, TSLE, is first computed, and the time of the last event, TLEVNT, is brought up to the current time in order to be ready for the next entry into UPTAVG. Then the area under the number-in-queue function is augmented by the area of the rectangle under $Q(t)$ during the interval since the previous event, which is of width TSLE and height NIQ; remember, UPTAVG is called *before* processing an event, and state variables such as NIQ still have their previous values. The area under $B(t)$ is then augmented by the area of a

```

SUBROUTINE REPORT
  INCLUDE 'mm1.dcl'
  REAL AVGDDEL,AVGNIQ,UTIL

*   Compute and write estimates of desired measures of performance.

  AVGDDEL = TOTDEL / NUMCUS
  AVGNIQ = ANIQ / TIME
  UTIL = AUTIL / TIME
  WRITE (6,2010) AVGDDEL, AVGNIQ, UTIL, TIME
2010 FORMAT (/ 'Average delay in queue',F11.3, ' minutes'//
&           'Average number in queue',F10.3//
&           'Server utilization',F15.3//
&           'Time simulation ended',F12.3, ' minutes')

  RETURN
END

```

FIGURE 1.16
FORTRAN code for subroutine REPORT, queueing model.


```

SUBROUTINE UPTAVG
INCLUDE 'mm1.dcl'
REAL TSLE

*   Compute time since last event, and update last-event-time marker.

TSLE  = TIME - TLEVNT
TLEVNT = TIME

*   Update area under number-in-queue function.

ANIQ  = ANIQ + NIQ * TSLE

*   Update area under server-busy indicator function.

AUTIL = AUTIL + SERVER * TSLE

RETURN
END

```

FIGURE 1.17
FORTRAN code for subroutine UPTAVG, queueing model.

rectangle of width TSLE and height SERVER; this is why it is convenient to define SERVER to be either 0 or 1. Note that this routine, like DEPART, contains a subtraction of two floating-point numbers (TIME-TLEVNT), both of which could become quite large relative to their difference if we were to run the simulation for a long time; in this case it may be necessary to declare both TIME and TLEVNT to be DOUBLE PRECISION variables.

The function EXPON, which generates an exponential random variate with mean $\beta = \text{RMEAN}$ (passed into EXPON), is shown in Fig. 1.18, and follows the algorithm discussed in Sec. 1.4.3. The random-number generator RAND, used here with an INTEGER argument of 1, is discussed fully in Chap. 7, and is shown specifically in Fig. 7.5. The FORTRAN built-in function LOG returns the natural logarithm of its argument, and agrees in type with its argument.

The program described here must be combined with the random-number-generator code from Fig. 7.5. This could be done by separate compilations, followed by linking the object codes together in an installation-dependent way.

```

REAL FUNCTION EXPON(RMEAN)
REAL RMEAN,U
REAL RAND

*   Generate a U(0,1) random variate.

U = RAND(1)

*   Return an exponential random variate with mean RMEAN.

EXPON = -RMEAN * LOG(U)

RETURN
* END

```

FIGURE 1.18
FORTRAN code for function EXPON.

1.4.5 Pascal Program

In this section we discuss a Pascal program for the $M/M/1$ queue simulation. We follow the language conventions described by Jensen and Wirth (1985); there are also several general introductions to Pascal, such as Grogono (1984). We have taken advantage of Pascal's facility to give variables and procedures fairly long names, which should thus be self-explanatory.

The global (outer-shell) declarations are given in Fig. 1.19. The constant `QLimit` is set to 100, our guess (which may have to be adjusted by trial and error) as to the longest the queue will ever get. The constants `Busy` and `Idle` are defined to be used with the `ServerStatus` variable, for code readability. The procedures and functions for the program are declared `FORWARD`, since there are occasions to invoke them from more than one place; this also allows them to appear sequentially in the file, rather than in a strictly nested fashion. Note that the array `Zrng`, as well as the procedures and functions `Randdf`, `Rand`, `Randst`, and `Randgt`, must be declared as well in order to use the random-number generator given in Fig. 7.6.

```
PROGRAM SingleServerQ(Input, Output);

{ Global declarations for single-server queueing system. }

CONST
  QLimit = 100; { Limit on queue length. }
  Busy   = 1;   { Mnemonics for server's being busy }
  Idle   = 0;   { and idle. }

VAR
  NextEventType, NumCustsDelayed, NumDelaysRequired, NumEvents, NumInQ,
  ServerStatus : Integer;
  AreaNumInQ, AreaServerStatus, MeanInterarrival, MeanService, Time,
  TimeLastEvent, TotalOfDelays : Real;
  TimeArrival : ARRAY [1..QLimit] OF Real;
  TimeNextEvent : ARRAY [1..2] OF Real;

  { The following declaration is for the random-number generator.
    Note that the name Zrng must not be used for any other purpose. }
  Zrng : ARRAY [1..100] OF Integer;

PROCEDURE Initialize; FORWARD;
PROCEDURE Timing; FORWARD;
PROCEDURE Arrive; FORWARD;
PROCEDURE Depart; FORWARD;
PROCEDURE Report; FORWARD;
PROCEDURE UpdateTimeAvgStats; FORWARD;
FUNCTION Expon(Mean : Real) : Real; FORWARD;

{ The following four declarations are for the random-number generator.
}

PROCEDURE Randdf; FORWARD;
FUNCTION Rand(Stream : Integer) : Real; FORWARD;
PROCEDURE Randst(Zset : Integer; Stream : Integer); FORWARD;
FUNCTION Randgt(Stream : Integer) : Integer; FORWARD;
```

FIGURE 1.19

Pascal code for the global declarations, queueing model.

Code for procedure Initialize is given in Fig. 1.20; this procedure is invoked before the simulation actually starts moving through time. Each statement here corresponds to an element of the computer representation in Fig. 1.7a. Note that the time of the first arrival, TimeNextEvent[1], is determined by adding an exponential random variate with mean MeanInterarrival, namely, Expon(MeanInterarrival), to the simulation clock, Time = 0. (We explicitly used Time in this statement, although it has a value of 0, to show the general form of a statement to determine the time of a future event.) Since no customers are present at Time = 0, the time of the next departure, TimeNextEvent[2], is set to 1.0E + 30 (Pascal notation for 10^{30}), guaranteeing that the first event will be an arrival.

Procedure Timing is given in Fig. 1.21, and is invoked whenever the simulation is ready to move on to whatever event should occur next. The program compares TimeNextEvent[1], TimeNextEvent[2], ..., TimeNextEvent[NumEvents] (NumEvents is the number of event types, being 2 for this model, and is set in the main program) and sets NextEventType equal to the event type whose time of occurrence is the smallest. In case of ties, the lowest-numbered event type is chosen. Then the simulation clock is advanced to the time of occurrence of the chosen event type, MinTimeNextEvent. The program is complicated slightly by an error check for the event list's being empty, which we define to mean that all events are scheduled to occur at

```
PROCEDURE Initialize; { Initialization procedure. }

BEGIN { Initialize }

    { Initialize the simulation clock. }

    Time := 0.0;

    { Initialize the state variables. }

    ServerStatus := Idle;
    NumInQ       := 0;
    TimeLastEvent := 0.0;

    { Initialize the statistical counters. }

    NumCustsDelayed := 0;
    TotalOfDelays   := 0.0;
    AreaNumInQ      := 0.0;
    AreaServerStatus := 0.0;

    { Initialize event list. Since no customers are present, the
      departure (service completion) event is eliminated from
      consideration. }

    TimeNextEvent[1] := Time + Expon(MeanInterarrival);
    TimeNextEvent[2] := 1.0E+30;

END; { Initialize }
```

FIGURE 1.20

Pascal code for procedure Initialize, queuing model.

```

PROCEDURE Timing; { Timing procedure. }

VAR
  I          : Integer;
  MinTimeNextEvent : Real;

BEGIN { Timing }

  MinTimeNextEvent := 1.0E+29;
  NextEventType    := 0;

  { Determine the event type of the next event to occur. }

  FOR I := 1 TO NumEvents DO BEGIN
    IF TimeNextEvent[I] < MinTimeNextEvent THEN BEGIN
      MinTimeNextEvent := TimeNextEvent[I];
      NextEventType    := I
    END
  END;

  { Check to see whether the event list is empty. }

  IF NextEventType = 0 THEN BEGIN
    { The event list is empty, so stop the simulation. }

    Writeln('Event list empty at time', Time);
    Halt
  END;

  { The event list is not empty, so advance the simulation clock. }

  Time := MinTimeNextEvent

END; { Timing }

```

FIGURE 1.21

Pascal code for procedure Timing, queueing model.

$\text{Time} = 10^{30}$. If this is ever the case (as indicated by $\text{NextEventType} = 0$), an error message is produced along with the current clock time (as a possible debugging aid), and the simulation is terminated.

The code for event procedure Arrive is in Fig. 1.22, and follows the language-independent discussion as given in Sec. 1.4.3 and in the flowchart of Fig. 1.8. Note that Time is the time of arrival of the customer who is just now arriving, and that the queue-overflow check is made by asking whether NumInQ is now greater than QLimit, the length for which TimeArrival was dimensioned.

Event procedure Depart, whose code is shown in Fig. 1.23, is invoked from the main program when a service completion (and subsequent departure) occurs; the logic for it was discussed in Sec. 1.4.3, with the flowchart in Fig. 1.9. Note that if the statement $\text{TimeNextEvent}[2] := 1.0\text{E} + 30$ just before the first END were omitted, the program would get into an infinite loop. (Why?) Advancing the rest of the queue (if any) one place by the FOR loop near the end of the procedure ensures that the arrival time of the next customer entering service (after being delayed in queue) will always be stored in

```

PROCEDURE Arrive; { Arrival event procedure. }

VAR
  Delay : Real;

BEGIN { Arrive }

  { Schedule next arrival. }
  TimeNextEvent[1] := Time + Expon(MeanInterarrival);

  { Check to see whether server is busy. }
  IF ServerStatus = Busy THEN BEGIN
    { Server is busy, so increment number of customers in queue. }
    NumInQ := NumInQ + 1;

    { Check to see whether an overflow condition exists. }
    IF NumInQ > QLimit THEN BEGIN
      { The queue has overflowed, so stop the simulation. }
      Writeln('Overflow of the array TimeArrival at time', Time);
      Halt
    END;

    { There is still room in the queue, so store the time of
      arrival of the arriving customer at the (new) end of
      TimeArrival. }
    TimeArrival[NumInQ] := Time
  END
  ELSE BEGIN
    { Server is idle, so arriving customer has a delay of zero.
      (The following two statements are for program clarity and do
      not affect the results of the simulation.) }
    Delay := 0.0;
    TotalOfDelays := TotalOfDelays + Delay;

    { Increment the number of customers delayed, and make server
      busy. }
    NumCustsDelayed := NumCustsDelayed + 1;
    ServerStatus := Busy;

    { Schedule a departure (service completion). }
    TimeNextEvent[2] := Time + Expon(MeanService)
  END
END; { Arrive }

```

FIGURE 1.22
Pascal code for procedure Arrive, queueing model.


```

PROCEDURE Depart; { Departure event procedure. }

VAR
  I      : Integer;
  Delay  : Real;

BEGIN { Depart }

  { Check to see whether the queue is empty. }

  IF NumInQ = 0 THEN BEGIN

    { The queue is empty so make the server idle and eliminate the
      departure (service completion) event from consideration. }

    ServerStatus      := Idle;
    TimeNextEvent[2] := 1.0E+30

  END
  ELSE BEGIN

    { The queue is nonempty, so decrement the number of customers
      in queue. }

    NumInQ := NumInQ - 1;

    { Compute the delay of the customer who is beginning service
      and update the total delay accumulator. }

    Delay      := Time - TimeArrival[1];
    TotalOfDelays := TotalOfDelays + Delay;

    { Increment the number of customers delayed, and schedule
      departure. }

    NumCustsDelayed := NumCustsDelayed + 1;
    TimeNextEvent[2] := Time + Expon(MeanService);

    { Move each customer in queue (if any) up one place. }

    FOR I := 1 TO NumInQ DO
      TimeArrival[I] := TimeArrival[I + 1]
    END

  END; { Depart }

```

FIGURE 1.23

Pascal code for procedure Depart, queueing model.

TimeArrival[1]. Note that if the queue were now empty (i.e., the customer who just left the queue and entered service had been the only one in queue), then NumInQ would be equal to 0, and this loop would not be executed at all since the beginning value of the loop index, I, starts out at a value (1) which would already exceed its final value (NumInQ = 0). (Managing the queue in this simple way is certainly inefficient; we return to this issue in Chap. 2). A final comment about Depart concerns the subtraction of TimeArrival[1] from the clock value, Time, to obtain the delay in queue. If the simulation is to run for a long period of (simulated) time, both Time and TimeArrival[1] would become very large numbers in comparison with the difference between them;

thus, since they are both stored as floating-point (Real) numbers with finite accuracy, there is potentially a serious loss of precision when doing this subtraction. For this reason, it may be necessary to make both Time and the TimeArrival array double precision (if available) if we want to run this simulation out for a long period of time.

The code for procedure Report, invoked when the termination check in the main program determines that the simulation is over, is given in Fig. 1.24. The average delay is computed by dividing the total of the delays by the number of customers whose delays were observed, and the time-average number in queue is obtained by dividing the area under $Q(t)$, now updated to the end of the simulation (since the procedure to update the areas is called from the main program before processing either an arrival or departure, one of which will end the simulation), by the clock value at termination. The server utilization is computed by dividing the area under $B(t)$ by the final clock time, and all three measures are written out. We also write out the final clock value itself, to see how long it took to observe the 1000 delays.

Procedure UpdateTimeAvgStats is shown in Fig. 1.25. This procedure is invoked just before processing each event (of any type) and updates the areas under the two functions needed for the continuous-time statistics; this routine is separate for coding convenience only, and is *not* an event routine. The time since the last event is first computed, and the time of the last event is brought up to the current time in order to be ready for the next entry into this procedure. Then the area under the number-in-queue function is augmented by the area of the rectangle under $Q(t)$ during the interval since the previous event, which is of width TimeSinceLastEvent and of height NumInQ; remember, this procedure is invoked *before* processing an event, and state

```
PROCEDURE Report; { Report generator procedure. }

VAR
    AvgDelayInQ, AvgNumInQ, ServerUtilization : Real;

BEGIN { Report }

    { Compute and write estimates of desired measures of performance. }

    AvgDelayInQ      := TotalOfDelays / NumCustsDelayed;
    AvgNumInQ        := AreaNumInQ / Time;
    ServerUtilization := AreaServerStatus / Time;
    Writeln;
    Writeln('Average delay in queue', AvgDelayInQ:11:3, ' minutes');
    Writeln;
    Writeln('Average number in queue', AvgNumInQ:10:3);
    Writeln;
    Writeln('Server utilization', ServerUtilization:15:3);
    Writeln;
    Writeln('Time simulation ended', Time:12:3)

END; { Report }
```

FIGURE 1.24

Pascal code for procedure Report, queueing model.

```

PROCEDURE UpdateTimeAvgStats; { Update area accumulators for
                                time-average statistics. }
VAR
    TimeSinceLastEvent : Real;
BEGIN { UpdateTimeAvgStats }
    { Compute time since last event, and update last-event-time
      marker. }
    TimeSinceLastEvent := Time - TimeLastEvent;
    TimeLastEvent      := Time;
    { Update area under number-in-queue function. }
    AreaNumInQ        := AreaNumInQ + NumInQ * TimeSinceLastEvent;
    { Update area under server-busy indicator function. }
    AreaServerStatus := AreaServerStatus +
                        ServerStatus * TimeSinceLastEvent
END; { UpdateTimeAvgStats }

```

FIGURE 1.25

Pascal code for procedure UpdateTimeAvgStats, queueing model.

variables such as NumInQ still have their previous values. The area under $B(t)$ is then augmented by the area of a rectangle of width TimeSinceLastEvent and height ServerStatus; this is why it is convenient to define ServerStatus to be either 0 or 1. Note that this procedure, like Depart, contains a subtraction of two floating-point numbers ($\text{Time} - \text{TimeLastEvent}$), both of which could become quite large relative to their difference if we were to run the simulation for a long time; in this case it may be necessary to declare both Time and TimeLastEvent to be double-precision variables, if available.

The function Expon, which generates an exponential random variate with mean $\beta = \text{Mean}$ (passed into Expon), is shown in Fig. 1.26, and follows the algorithm discussed in Sec. 1.4.3. The random-number generator Rand, used

```

FUNCTION Expon; { Exponential variate generation function. }
                { Pass in Real parameter Mean giving mean, as
                  declared in FORWARD declarations earlier. }
VAR
    U : Real;
BEGIN { Expon }
    { Generate a U(0,1) random variate. }
    U := Rand(1);
    { Return an exponential random variate with mean Mean. }
    Expon := -Mean * Ln(U)
END; { Expon }

```

FIGURE 1.26

Pascal code for function Expon.

```

BEGIN { SingleServerQ main program. }

  { Initialize the random-number generator. }
  Randdf;

  { Specify the number of events for the timing procedure. }
  NumEvents := 2;

  { Read input parameters. }
  Readln(MeanInterarrival, MeanService, NumDelaysRequired);

  { Write report heading and input parameters. }
  Writeln('Single-server queueing system');
  Writeln;
  Writeln('Mean interarrival time', MeanInterarrival:11:3, ' minutes');
  Writeln;
  Writeln('Mean service time', MeanService:16:3, ' minutes');
  Writeln;
  Writeln('Number of customers', NumDelaysRequired:14);
  Writeln;
  Writeln;

  { Initialize the simulation. }
  Initialize;

  { Run the simulation while more delays are still needed. }
  WHILE NumCustsDelayed < NumDelaysRequired DO BEGIN
    { Determine the next event. }
    Timing;

    { Update time-average statistical accumulators. }
    UpdateTimeAvgStats;

    { Invoke the appropriate event procedure. }
    CASE NextEventType OF
      1 : Arrive;
      2 : Depart
    END

  END;

  { Invoke the report generator and end the simulation. }
  Report

END. { SingleServerQ }

```

FIGURE 1.27

Pascal code for the main program, queueing model.

here with an Integer argument of 1, is discussed fully in Chap. 7, and is shown specifically in Fig. 7.6. The Pascal built-in function `Ln` returns the natural logarithm of its argument.

The code for the main program is shown in Fig. 1.27, and ties the foregoing pieces together. The random-number generator in Fig. 7.6 must be initialized by invoking `Randdf`. The number of event types for the simulation is initialized to 2 for this model. The input parameters are then read in. (In order to keep the code as general as possible, we assume that if input is to be from a file, it will be assigned at the operating-system level, perhaps with some kind of redirection of "standard" input; the same is true for the output.) After writing a report heading and echoing the input parameters (as a check that they were read correctly), the initialization procedure is invoked. The `WHILE` loop then executes the simulation as long as more customer delays are still needed to fulfill the 1000-delay stopping rule. Inside the `WHILE` loop, the Timing procedure is first invoked to determine the type of the next event to occur and to advance the simulation clock to its time. Before processing this event, the procedure to update the areas under the $Q(t)$ and $B(t)$ curves is invoked; by doing this at this time we automatically update these areas before processing each event. Then a `CASE` statement, based on `NextEventType` (=1 for an arrival and 2 for a departure), passes control to the appropriate event procedure. After the `WHILE` loop is done, the Report procedure is invoked, and the simulation ends.

As a final note, the random-number-generator procedures and functions (`Randdf`, `Rand`, `Randst`, and `Randgt`, as listed in Fig. 7.6) must be placed inside the above program between the `Expon` procedure and the main program. This can be done either physically with an editor, or by inserting a compiler-dependent include directive at this point to bring in the file containing the code from Fig. 7.6.

1.4.6 C Program

This section presents a C program for the $M/M/1$ queue simulation. We have chosen to write in the ANSI-standard version of the language, as described by Kernighan and Ritchie (1988), and in particular use function prototyping. We have also taken advantage of C's facility to give variables and functions fairly long names, which should thus be self-explanatory.

The external definitions are given in Fig. 1.28. The header file `rand.h` (listed in Fig. 7.8) is included to declare the functions for the random-number generator. The symbolic constant `Q_LIMIT` is set to 100, our guess (which may have to be adjusted by trial and error) as to the longest the queue will ever get. The symbolic constants `BUSY` and `IDLE` are defined to be used with the `server_status` variable, for code readability. File pointers `*infile` and `*outfile` are defined to allow us to open the input and output files from within the code, rather than at the operating-system level. Note also that the event list, as we have discussed it so far, will be implemented in an array called `time_next_`


```

/* External definitions for single-server queueing system. */

#include <stdio.h>
#include <math.h>
#include "rand.h" /* Header file for random-number generator. */

#define Q_LIMIT 100 /* Limit on queue length. */
#define BUSY .1 /* Mnemonics for server's being busy */
#define IDLE 0 /* and idle. */

int next_event_type, num_custs_delayed, num_delays_required,
    num_events, num_in_q, server_status;
float area_num_in_q, area_server_status, mean_interarrival,
    mean_service, time, time_arrival[Q_LIMIT + 1],
    time_last_event, time_next_event[3], total_of_delays;
FILE *infile, *outfile;

void initialize(void);
void timing(void);
void arrive(void);
void depart(void);
void report(void);
void update_time_avg_stats(void);
float expon(float mean);

```

FIGURE 1.28

C code for the external definitions, queueing model.

event, whose 0th entry will be ignored in order to make the index agree with the event type.

The code for the main function is shown in Fig. 1.29. The input and output files are opened, and the number of event types for the simulation is initialized to 2 for this model. The input parameters then are read in from the file `mm1.in`, which contains a single line with the numbers 1.0, 0.5, and 1000, separated by blanks. After writing a report heading and echoing the input parameters (as a check that they were read correctly), the initialization function is invoked. The “while” loop then executes the simulation as long as more customer delays are needed to fulfill the 1000-delay stopping rule. Inside the “while” loop, the timing function is first invoked to determine the type of the next event to occur and to advance the simulation clock to its time. Before processing this event, the function to update the areas under the $Q(t)$ and $B(t)$ curves is invoked; by doing this at this time we automatically update these areas before processing each event. Then a switch statement, based on `next_event_type` (=1 for an arrival and 2 for a departure), passes control to the appropriate event function. After the “while” loop is done, the report function is invoked, the input and output files are closed, and the simulation ends.

Code for the initialization function is given in Fig. 1.30. Each statement here corresponds to an element of the computer representation in Fig. 1.7a. Note that the time of the first arrival, `time_next_event[1]`, is determined by adding an exponential random variate with mean `mean_interarrival`, namely, `expon(mean_interarrival)`, to the simulation clock, `time = 0`. (We explicitly used “time” in this statement, although it has a value of 0, to show the general

```

main() /* Main function. */
{
    /* Open input and output files. */
    infile = fopen("mml.in", "r");
    outfile = fopen("mml.out", "w");

    /* Specify the number of events for the timing function. */
    num_events = 2;

    /* Read input parameters. */
    fscanf(infile, "%f %f %d", &mean_interarrival, &mean_service,
        &num_delays_required);

    /* Write report heading and input parameters. */
    fprintf(outfile, "Single-server queueing system\n\n");
    fprintf(outfile, "Mean interarrival time%11.3f minutes\n\n",
        mean_interarrival);
    fprintf(outfile, "Mean service time%16.3f minutes\n\n",
        mean_service);
    fprintf(outfile, "Number of customers%14d\n\n",
        num_delays_required);

    /* Initialize the simulation. */
    initialize();

    /* Run the simulation while more delays are still needed. */
    while (num_custs_delayed < num_delays_required) {
        /* Determine the next event. */
        timing();

        /* Update time-average statistical accumulators. */
        update_time_avg_stats();

        /* Invoke the appropriate event function. */
        switch (next_event_type) {
            case 1:
                arrive();
                break;
            case 2:
                depart();
                break;
        }
    }

    /* Invoke the report generator and end the simulation. */
    report();

    fclose(infile);
    fclose(outfile);

    return 0;
}

```

FIGURE 1.29

C code for the main function, queueing model.

```

void initialize(void) /* Initialization function. */
{
    /* Initialize the simulation clock. */

    time = 0.0;

    /* Initialize the state variables. */

    server_status = IDLE;
    num_in_q      = 0;
    time_last_event = 0.0;

    /* Initialize the statistical counters. */

    num_custs_delayed = 0;
    total_of_delays    = 0.0;
    area_num_in_q      = 0.0;
    area_server_status = 0.0;

    /* Initialize event list. Since no customers are present, the
       departure (service completion) event is eliminated from
       consideration. */

    time_next_event[1] = time + expon(mean_interarrival);
    time_next_event[2] = 1.0e+30;
}

```

FIGURE 1.30

C code for function initialize, queueing model.

form of a statement to determine the time of a future event.) Since no customers are present at time = 0, the time of the next departure, time_next_event[2], is set to 1.0e + 30 (C notation for 10^{30}), guaranteeing that the first event will be an arrival.

The timing function is given in Fig. 1.31 to compare time_next_event[1], time_next_event[2], ..., time_next_event[num_events] (recall that num_events was set in the main function) and set next_event_type equal to the event type whose time of occurrence is the smallest. In case of ties, the lowest-numbered event type is chosen. Then the simulation clock is advanced to the time of occurrence of the chosen event type, min_time_next_event. The program is complicated slightly by an error check for the event list's being empty, which we define to mean that all events are scheduled to occur at time = 10^{30} . If this is ever the case (as indicated by next_event_type = 0), an error message is produced along with the current clock time (as a possible debugging aid), and the simulation is terminated.

The code for event function arrive is in Fig. 1.32, and follows the language-independent discussion as given in Sec. 1.4.3 and in the flowchart of Fig. 1.8. Note that "time" is the time of arrival of the customer who is just now arriving, and that the queue-overflow check is made by asking whether num_in_q is now greater than Q_LIMIT, the length for which the array time_arrival was dimensioned.

Event function depart, whose code is shown in Fig. 1.33, is invoked from the main program when a service completion (and subsequent departure)

```

void timing(void) /* Timing function. */
{
    int i;
    float min_time_next_event = 1.0e+29;

    next_event_type = 0;

    /* Determine the event type of the next event to occur. */

    for (i = 1; i <= num_events; ++i) {
        if (time_next_event[i] < min_time_next_event) {
            min_time_next_event = time_next_event[i];
            next_event_type = i;
        }
    }

    /* Check to see whether the event list is empty. */

    if (next_event_type == 0) {

        /* The event list is empty, so stop the simulation. */

        fprintf(outfile, "\nEvent list empty at time %f", time);
        exit(1);
    }

    /* The event list is not empty, so advance the simulation clock. */

    time = min_time_next_event;
}

```

FIGURE 1.31

C code for function timing, queueing model.

occurs; the logic for it was discussed in Sec. 1.4.3, with the flowchart in Fig. 1.9. Note that if the statement “time_next_event[2] = 1.0e + 30;” just before the “else” were omitted, the program would get into an infinite loop. (Why?) Advancing the rest of the queue (if any) one place by the “for” loop near the end of the function ensures that the arrival time of the next customer entering service (after being delayed in queue) will always be stored in time_arrival[1]. Note that if the queue were now empty (i.e., the customer who just left the queue and entered service had been the only one in queue), then num_in_q would be equal to 0, and this loop would not be executed at all since the beginning value of the loop index, i, starts out at a value (1) that would already exceed its final value (num_in_q = 0). (Managing the queue in this simple way is certainly inefficient, and could be improved by using pointers; we return to this issue in Chap. 2). A final comment about depart concerns the subtraction of time_arrival[1] from the clock value, time, to obtain the delay in queue. If the simulation is to run for a long period of (simulated) time, both time and time_arrival[1] would become very large numbers in comparison with the difference between them; thus, since they are both stored as floating-point (float) numbers with finite accuracy, there is potentially a serious loss of precision when doing this subtraction. For this reason, it may be necessary to **make** both time and the time_arrival array of type double if we are to run this **simulation** out for a long period of time.

```

void arrive(void) /* Arrival event function. */
{
    float delay;

    /* Schedule next arrival. */
    time_next_event[1] = time + expon(mean_interarrival);

    /* Check to see whether server is busy. */
    if (server_status == BUSY) {
        /* Server is busy, so increment number of customers in queue.
           */
        ++num_in_q;

        /* Check to see whether an overflow condition exists. */
        if (num_in_q > Q_LIMIT) {
            /* The queue has overflowed, so stop the simulation. */
            fprintf(outfile, "\nOverflow of the array time_arrival at");
            fprintf(outfile, " time %f", time);
            exit(2);
        }

        /* There is still room in the queue, so store the time of
           arrival of the arriving customer at the (new) end of
           time_arrival. */
        time_arrival[num_in_q] = time;
    }
    else {
        /* Server is idle, so arriving customer has a delay of zero.
           (The following two statements are for program clarity and do
           not affect the results of the simulation.) */
        delay = 0.0;
        total_of_delays += delay;

        /* Increment the number of customers delayed, and make server
           busy. */
        ++num_custs_delayed;
        server_status = BUSY;

        /* Schedule a departure (service completion). */
        time_next_event[2] = time + expon(mean_service);
    }
}

```

FIGURE 1.32
C code for function arrive, queuing model.


```

void depart(void) /* Departure event function. */
{
    int i;
    float delay;

    /* Check to see whether the queue is empty. */
    if (num_in_q == 0) {
        /* The queue is empty so make the server idle and eliminate the
           departure (service completion) event from consideration. */
        server_status = IDLE;
        time_next_event[2] = 1.0e+30;
    }
    else {
        /* The queue is nonempty, so decrement the number of customers
           in queue. */
        --num_in_q;

        /* Compute the delay of the customer who is beginning service
           and update the total delay accumulator. */
        delay = time - time_arrival[1];
        total_of_delays += delay;

        /* Increment the number of customers delayed, and schedule
           departure. */
        ++num_custs_delayed;
        time_next_event[2] = time + expon(mean_service);

        /* Move each customer in queue (if any) up one place. */
        for (i = 1; i <= num_in_q; ++i)
            time_arrival[i] = time_arrival[i + 1];
    }
}

```

FIGURE 1.33

C code for function depart, queueing model.

The code for the report function, invoked when the “while” loop in the main program is over, is given in Fig. 1.34. The average delay is computed by dividing the total of the delays by the number of customers whose delays were observed, and the time-average number in queue is obtained by dividing the area under $Q(t)$, now updated to the end of the simulation (since the function to update the areas is called from the main program before processing either an arrival or departure, one of which will end the simulation), by the clock value at termination. The server utilization is computed by dividing the area under $B(t)$ by the final clock time, and all three measures are written out directly. We also write out the final clock value itself, to see how long it took to observe the 1000 delays.

Function `update_time_avg_stats` is shown in Fig. 1.35. This function is invoked just before processing each event (of any type) and updates the areas

```

void report(void) /* Report generator function. */
{
    /* Compute and write estimates of desired measures of performance.
    */

    fprintf(outfile, "\n\nAverage delay in queue%11.3f minutes\n\n",
        total_of_delays / num_custs_delayed);
    fprintf(outfile, "Average number in queue%10.3f\n\n",
        area_num_in_q / time);
    fprintf(outfile, "Server utilization%15.3f\n\n",
        area_server_status / time);
    fprintf(outfile, "Time simulation ended%12.3f", time);
}

```

FIGURE 1.34

C code for function report, queueing model.

under the two functions needed for the continuous-time statistics; this routine is separate for coding convenience only, and is *not* an event routine. The time since the last event is first computed, and then the time of the last event is brought up to the current time in order to be ready for the next entry into this function. Then the area under the number-in-queue function is augmented by the area of the rectangle under $Q(t)$ during the interval since the previous event, which is of width `time_since_last_event` and of height `num_in_q`; remember, this function is invoked *before* processing an event, and state variables such as `num_in_q` still have their previous values. The area under $B(t)$ is then augmented by the area of a rectangle of width `time_since_last_event` and height `server_status`; this is why it is convenient to define `server_status` to be either 0 or 1. Note that this function, like `depart`, contains a subtraction of two floating-point numbers (`time - time_last_event`), both of which could become quite large relative to their difference if we were to run the simulation for a long time; in this case it might be necessary to declare both `time` and `time_last_event` to be of type `double`.

```

void update_time_avg_stats(void) /* Update area accumulators for
                                time-average statistics. */
{
    float time_since_last_event;

    /* Compute time since last event, and update last-event-time
    marker. */

    time_since_last_event = time - time_last_event;
    time_last_event       = time;

    /* Update area under number-in-queue function. */

    area_num_in_q += num_in_q * time_since_last_event;

    /* Update area under server-busy indicator function. */

    area_server_status += server_status * time_since_last_event;
}

```

FIGURE 1.35

C code for function `update_time_avg_stats`, queueing model.

```

float expon(float mean) /* Exponential variate generation function.
                          */
{
    float u;

    /* Generate a U(0,1) random variate. */
    u = rand(1);

    /* Return an exponential random variate with mean "mean". */
    return -mean * log(u);
}

```

FIGURE 1.36

C code for function expon.

The function `expon`, which generates an exponential random variate with mean $\beta = \text{mean}$ (passed into `expon`), is shown in Fig. 1.36, and follows the algorithm discussed in Sec. 1.4.3. The random-number generator `rand`, used here with an `int` argument of 1, is discussed fully in Chap. 7, and is shown specifically in Fig. 7.7. The C predefined function `log` returns the natural logarithm of its argument.

The program described here must be combined with the random-number-generator code from Fig. 7.7. This could be done by separate compilations, followed by linking the object codes together in an installation-dependent way.

1.4.7 Simulation Output and Discussion

The output (in a file named `mm1.out` if the FORTRAN or C program above was used) is shown in Fig. 1.37; since the same method for random-number generation was used for the programs in all three languages, they produced identical results. In this run, the average delay in queue was 0.430 minute, there was an average of 0.418 customer in the queue, and the server was busy

Single-server queueing system

Mean interarrival time	1.000 minutes
Mean service time	0.500 minutes
Number of customers	1000
Average delay in queue	0.430 minutes
Average number in queue	0.418
Server utilization	0.460
Time simulation ended	1027.915 minutes

FIGURE 1.37

Output report, queueing model.

46 percent of the time. It took 1027.915 simulated minutes to run the simulation to the completion of 1000 delays, which seems reasonable since the expected time between customer arrivals was 1 minute. (It is not a coincidence that the average delay, average number in queue, and utilization are all so close together for this model; see App. 1B.)

Note that these particular numbers in the output were determined, at root, by the numbers the random-number generator happened to come up with this time. If a different random-number generator were used, or if this one were used in another way (with another "seed" or "stream," as discussed in Chap. 7), then different numbers would have been produced in the output. Thus, these numbers are not to be regarded as "The Answers," but rather as estimates (and perhaps poor ones) of the expected quantities we want to know about, $d(n)$, $q(n)$, and $u(n)$; the statistical analysis of simulation output data is discussed in Chaps. 9 through 12. Also, the results are functions of the input parameters, in this case the mean interarrival and service times, and the $n = 1000$ stopping rule; they are also affected by the way we initialized the simulation (empty and idle).

In some simulation studies, we might want to estimate *steady-state* characteristics of the model (see Chap. 9), i.e., characteristics of a model after the simulation has been running a very long (in theory, an infinite) amount of time. For the simple $M/M/1$ queue we have been considering, it is possible to compute *analytically* the steady-state average delay in queue, the steady-state time-average number in queue, and the steady-state server utilization, all of these measures of performance being 0.5 [see, for example, Ross (1989, p. 352)]. Thus, if we wanted to determine these steady-state measures, our estimates based on the stopping rule $n = 1000$ delays were not too far off, at least in absolute terms. However, we were somewhat lucky, since $n = 1000$ was chosen arbitrarily! In practice, the choice of a stopping rule that will give good estimates of steady-state measures is quite difficult. To illustrate this point, suppose for the $M/M/1$ queue that the arrival rate of customers were increased from 1 per minute to 1.98 per minute (the mean interarrival time is now 0.505 minute), that the mean service time is unchanged, and that we wish to estimate the steady-state measures from a run of length $n = 1000$ delays, as before. We performed this simulation run and got values for the average delay, average number in queue, and server utilization of 17.404 minutes, 34.831, and 0.997, respectively. Since the true steady-state values of these measures are 49.5 minutes, 98.01, and 0.99 (respectively), it is clear that the stopping rule cannot be chosen arbitrarily. We discuss how to specify the run length for a steady-state simulation in Chap. 9.

The reader may have wondered why we did not estimate the expected average waiting time in the system of a customer, $w(n)$, rather than the expected average delay in queue, $d(n)$, where the waiting time of a customer is defined as the time interval from the instant the customer arrives to the instant the customer completes service and departs. There were two reasons. First, for many queueing systems we believe that the customer's delay in queue while

waiting for other customers to be served is the most troublesome part of the customer's wait in the system. Moreover, if the queue represents part of a manufacturing system where the "customers" are actually parts waiting for service at a machine (the "server"), then the delay in queue represents a loss, whereas the time spent in service is "necessary." Our second reason for focusing on the delay in queue is one of statistical efficiency. The usual estimator of $w(n)$ would be

$$\hat{w}(n) = \frac{\sum_{i=1}^n W_i}{n} = \frac{\sum_{i=1}^n D_i}{n} + \frac{\sum_{i=1}^n S_i}{n} = \hat{d}(n) + \bar{S}(n) \quad (1.7)$$

where $W_i = D_i + S_i$ is the waiting time in system of the i th customer and $\bar{S}(n)$ is the average of the n customers' service times. Since the service-time distribution would have to be known to perform a simulation in the first place, the expected or mean service time, $E(S)$, would also be known and an alternative estimator of $w(n)$ is

$$\tilde{w}(n) = \hat{d}(n) + E(S)$$

[Note that $\bar{S}(n)$ is an unbiased estimator of $E(S)$ in Eq. (1.7).] In almost all queueing simulations, $\tilde{w}(n)$ will be a more efficient (less variable) estimator of $w(n)$ than $\hat{w}(n)$ and is thus preferable (both estimators are unbiased). Therefore, if one wants an estimate of $w(n)$, estimate $\hat{d}(n)$ and add the known expected service time, $E(S)$. In general, the moral is to replace estimators by their expected values whenever possible (see the discussion of indirect estimators in Sec. 11.5).

1.4.8 Alternative Stopping Rules

In the above queueing example, the simulation was terminated when the number of customers delayed became equal to 1000; the final value of the simulation clock was thus a random variable. However, for many real-world models, the simulation is to stop after some fixed amount of time, say 8 hours. Since the interarrival and service times for our example are continuous random variables, the probability of the simulation's terminating after exactly 480 minutes is 0 (neglecting the finite accuracy of a computer). Therefore, to stop the simulation at a specified time, we introduce a dummy "end-simulation" event (call it an event of type 3), which is scheduled to occur at time 480. When the time of occurrence of this event (being held in the third spot of the event list) is less than all other entries in the event list, the report generator is called and the simulation is terminated. The number of customers delayed is now a random variable.

These ideas can be implemented in the computer programs by making changes to the main program, the initialization routine, and the report generator, as described below. The reader need go through the changes for only one of the three languages, but should review carefully the corresponding code.

FORTRAN Program. Changes must be made in the main program, the declarations file (renamed mm1alt.dcl), INIT, and REPORT, as shown in Figs. 1.38 through 1.41. The only changes in TIMING, ARRIVE, DEPART, and UPTAVG are in the file name in the INCLUDE statements, and there are no changes at all in EXPON. In Figs. 1.38 and 1.39, note that we now have 3 events, that the desired simulation run length, TEND, is now an input parameter and a member of the COMMON block MODEL (TOTCUS has been removed), and that the statements after the "computed GO TO" statement have been changed. In the main program (as before), we call UPTAVG before entering an event routine, so that in particular the areas will be updated to the end of the simulation here when the type 3 event (end simulation) is next. The only change to INIT (other than the file name to INCLUDE) is the addition of the statement $TNE(3) = TEND$, which schedules the end of the simulation. The only change to REPORT in Fig. 1.41 is to write the number of customers delayed instead of the time the simulation ends, since in this case we know that the ending time will be 480 minutes but will not know how many customer delays will have been completed during that time.

Pascal Program. Changes must be made in the global (outer-shell) declarations, procedures Initialize and Report, and in the main program, as shown in Figs. 1.42 through 1.45; the rest of the program is unaffected. In Figs. 1.42 and 1.45, note that there are now 3 events, that the desired simulation run length, TimeEnd, is now an input parameter (NumDelaysRequired has been removed), and that the CASE statement in the main program has been changed. The only change to the initialization procedure in Fig. 1.43 is the addition of the statement $TimeNextEvent[3] := TimeEnd$, which schedules the end of the simulation. The only change to the Report procedure in Fig. 1.44 is to write the number of customers delayed instead of the time the simulation ends, since in this case we know that the ending time will be 480 minutes but will not know how many customer delays will have been completed during that time. To stop the simulation in the main program of Fig. 1.45, the original WHILE loop has been replaced by a REPEAT UNTIL loop, where the loop is repeated until the type of event just executed is 3 (end simulation), in which case the loop ends and the simulation stops. In the main program (as before), we invoke UpdateTimeAvgStats before entering an event procedure, so that in particular the areas will be updated to the end of the simulation here when the type 3 event (end simulation) is next.

C Program. Changes must be made in the external definitions, the main function, and in the initialize and report functions, as shown in Figs. 1.46 through 1.49; the rest of the program is unaltered. In Figs. 1.46 and 1.47, note that we now have 3 events, that the desired simulation run length, time_end, is now an input parameter (num_delays_required has been removed), and that the "switch" statement has been changed. To stop the simulation, the original "while" loop has been replaced by a "do while" loop in Fig. 1.47, where the loop keeps repeating itself as long as the type of event just executed is not 3

```

*      Main program for single-server queueing system, fixed run length.
*
*      Bring in declarations file.
      INCLUDE 'mmlalt.dcl'
*
*      Open input and output files.
      OPEN (5, FILE = 'mmlalt.in')
      OPEN (6, FILE = 'mmlalt.out')
*
*      Specify the number of event types for the timing routine.
      NEVNTS = 3
*
*      Set mnemonics for server's being busy and idle.
      BUSY = 1
      IDLE = 0
*
*      Read input parameters.
      READ (5,*) MARRVT, MSERVT, TEND
*
*      Write report heading and input parameters.
      WRITE (6,2010) MARRVT, MSERVT, TEND
2010 FORMAT (' Single-server queueing system with fixed run length'//
&          ' Mean interarrival time',F11.3,' minutes'//
&          ' Mean service time',F16.3,' minutes'//
&          ' Length of the simulation',F9.3,' minutes'//)
*
*      Initialize the simulation.
      CALL INIT
*
*      Determine the next event.
      10 CALL TIMING
*
*      Update time-average statistical accumulators.
      CALL UPTAVG
*
*      Call the appropriate event routine.
      GO TO (20, 30, 40), NEXT
20    CALL ARRIVE
      GO TO 10
30    CALL DEPART
      GO TO 10
*
*      Simulation is over; call report generator and end
*      simulation.
      40 CALL REPORT
      CLOSE (5)
      CLOSE (6)
      STOP
      END

```

FIGURE 1.38

FORTRAN code for the main program, queueing model with fixed run length.

```

INTEGER QLIMIT
PARAMETER (QLIMIT = 100)
INTEGER BUSY, IDLE, NEVNTS, NEXT, NIQ, NUMCUS, SERVER
REAL ANIQ, AUTIL, MARRVT, MSERVT, TARRVL(QLIMIT), TEND, TIME, TLEVNT,
& TNE(3), TOTDEL
REAL EXPON
COMMON /MODEL/ ANIQ, AUTIL, BUSY, IDLE, MARRVT, MSERVT, NEVNTS, NEXT, NIQ,
& NUMCUS, SERVER, TARRVL, TEND, TIME, TLEVNT, TNE, TOTDEL

```

FIGURE 1.39

FORTRAN code for the declarations file (mmlalt.dcl), queueing model with fixed run length.

(end simulation); after a type 3 event is chosen for execution, the loop ends and the simulation stops. In the main program (as before), we invoke `update_time_avg_stats` before entering an event function, so that in particular the areas will be updated to the end of the simulation here when the type 3 event (end simulation) is next. The only change to the initialization function in Fig. 1.48 is the addition of the statement `time_next_event[3] = time_end`, which schedules the end of the simulation. The only change to the report function in Fig. 1.49 is to write the number of customers delayed instead of the time the simulation ends, since in this case we know that the ending time will be 480 minutes but will not know how many customer delays will have been completed during that time.

```

SUBROUTINE INIT
INCLUDE 'mmlalt.dcl'

* Initialize the simulation clock.

TIME = 0.0

* Initialize the state variables.

SERVER = IDLE
NIQ = 0
TLEVNT = 0.0

* Initialize the statistical counters.

NUMCUS = 0
TOTDEL = 0.0
ANIQ = 0.0
AUTIL = 0.0

* Initialize event list. Since no customers are present, the
* departure (service completion) event is eliminated from
* consideration. The end-simulation event (type 3) is scheduled for
* time TEND.

TNE(1) = TIME + EXPON(MARRVT)
TNE(2) = 1.0E+30
TNE(3) = TEND

RETURN
END

```

FIGURE 1.40

FORTRAN code for subroutine INIT, queueing model with fixed run length.

```

SUBROUTINE REPORT
INCLUDE 'mmlalt.dcl'
REAL AVGDEL,AVGNIQ,UTIL

*   Compute and write estimates of desired measures of performance.

AVGDEL = TOTDEL / NUMCUS
AVGNIQ = ANIQ / TIME
UTIL = AUTIL / TIME
WRITE (6,2010) AVGDEL, AVGNIQ, UTIL, NUMCUS
2010 FORMAT (/ ' Average delay in queue',F11.3,' minutes'//
&           ' Average number in queue',F10.3//
&           ' Server utilization',F15.3//
&           ' Number of delays completed',I7)

RETURN
END

```

FIGURE 1.41

FORTTRAN code for subroutine REPORT, queueing model with fixed run length.

```

PROGRAM SingleServerQAlt(Input, Output);

{ Global declarations for single-server queueing system, fixed run
length. }

CONST
  QLimit = 100; { Limit on queue length. }
  Busy = 1; { Mnemonics for server's being busy }
  Idle = 0; { and idle. }

VAR
  NextEventType, NumCustsDelayed, NumEvents, NumInQ, ServerStatus :
    Integer;
  AreaNumInQ, AreaServerStatus, MeanInterarrival, MeanService, Time,
  TimeEnd, TimeLastEvent, TotalOfDelays : Real;
  TimeArrival : ARRAY [1..QLimit] OF Real;
  TimeNextEvent : ARRAY [1..3] OF Real;

{ The following declaration is for the random-number generator.
Note that the name Zrng must not be used for any other purpose. }

  Zrng : ARRAY [1..100] OF Integer;

PROCEDURE Initialize; FORWARD;
PROCEDURE Timing; FORWARD;
PROCEDURE Arrive; FORWARD;
PROCEDURE Depart; FORWARD;
PROCEDURE Report; FORWARD;
PROCEDURE UpdateTimeAvgStats; FORWARD;
FUNCTION Expon(Mean : Real) : Real; FORWARD;

{ The following four declarations are for the random-number generator.
}

PROCEDURE Randdf; FORWARD;
FUNCTION Rand(Stream : Integer) : Real; FORWARD;
PROCEDURE Randst(Zset : Integer; Stream : Integer); FORWARD;
FUNCTION Randgt(Stream : Integer) : Integer; FORWARD;

```

FIGURE 1.42

Pascal code for the global declarations, queueing model with fixed run length.

```

PROCEDURE Initialize; ( Initialization procedure. )
  BEGIN ( Initialize )
    { Initialize the simulation clock. }
    Time := 0.0;
    { Initialize the state variables. }
    ServerStatus := Idle;
    NumInQ       := 0;
    TimeLastEvent := 0.0;
    { Initialize the statistical counters. }
    NumCustsDelayed := 0;
    TotalOfDelays   := 0.0;
    AreaNumInQ      := 0.0;
    AreaServerStatus := 0.0;
    { Initialize event list. Since no customers are present, the
      departure (service completion) event is eliminated from
      consideration. The end-simulation event (type 3) is scheduled
      for time TimeEnd. }
    TimeNextEvent[1] := Time + Expon(MeanInterarrival);
    TimeNextEvent[2] := 1.0E+30;
    TimeNextEvent[3] := TimeEnd
  END; ( Initialize )

```

FIGURE 1.43

Pascal code for procedure Initialize, queueing model with fixed run length.

```

PROCEDURE Report; ( Report generator procedure. )
  VAR
    AvgDelayInQ, AvgNumInQ, ServerUtilization : Real;
  BEGIN ( Report )
    { Compute and write estimates of desired measures of performance.
      }
    AvgDelayInQ      := TotalOfDelays / NumCustsDelayed;
    AvgNumInQ        := AreaNumInQ / Time;
    ServerUtilization := AreaServerStatus / Time;
    Writeln;
    Writeln('Average delay in queue', AvgDelayInQ:11:3, ' minutes');
    Writeln;
    Writeln('Average number in queue', AvgNumInQ:10:3);
    Writeln;
    Writeln('Server utilization', ServerUtilization:15:3);
    Writeln;
    Writeln('Number of delays completed', NumCustsDelayed:7)
  END; ( Report )

```

FIGURE 1.44

Pascal code for procedure Report, queueing model with fixed run length.

```

BEGIN ( SingleServerQAlt main program. )

  { Initialize the random-number generator. }

  Randdf;

  { Specify the number of events for the timing procedure. }

  NumEvents := 3;

  { Read input parameters. }

  Readln(MeanInterarrival, MeanService, TimeEnd);

  { Write report heading and input parameters. }

  Writeln('Single-server queueing system with fixed run length');
  Writeln;
  Writeln('Mean interarrival time', MeanInterarrival:11:3, ' minutes');
  Writeln;
  Writeln('Mean service time', MeanService:16:3, ' minutes');
  Writeln;
  Writeln('Length of the simulation', TimeEnd:9:3, ' minutes');
  Writeln;
  Writeln;

  { Initialize the simulation. }

  Initialize;

  { Run the simulation until it terminates after an end-simulation
    event (type 3) occurs. }

  REPEAT

    { Determine the next event. }

    Timing;

    { Update time-average statistical accumulators. }

    UpdateTimeAvgStats;

    { Invoke the appropriate event procedure. }

    CASE NextEventType OF
      1 : Arrive;
      2 : Depart;
      3 : Report
    END

    { If the event just executed was the end-simulation event (type 3),
      end the simulation. Otherwise, continue simulating. }

  UNTIL NextEventType = 3

END. { SingleServerQAlt }

```

FIGURE 1.45

Pascal code for the main program, queueing model with fixed run length.


```

/* External definitions for single-server queueing system, fixed run
length. */

#include <stdio.h>
#include <math.h>
#include "rand.h" /* Header file for random-number generator. */

#define Q_LIMIT 100 /* Limit on queue length. */
#define BUSY 1 /* Mnemonics for server's being busy */
#define IDLE 0 /* and idle. */

int next_event_type, num_custs_delayed, num_events, num_in_q,
server_status;
float area_num_in_q, area_server_status, mean_interarrival,
mean_service, time, time_arrival[Q_LIMIT + 1], time_end,
time_last_event, time_next_event[4], total_of_delays;
FILE *infile, *outfile;

void initialize(void);
void timing(void);
void arrive(void);
void depart(void);
void report(void);
void update_time_avg_stats(void);
float expon(float mean);

```

FIGURE 1.46

C code for the external definitions, queueing model with fixed run length.

```

main() /* Main function. */
{
    /* Open input and output files. */

    infile = fopen("mmlalt.in", "r");
    outfile = fopen("mmlalt.out", "w");

    /* Specify the number of events for the timing function. */

    num_events = 3;

    /* Read input parameters. */

    fscanf(infile, "%f %f %f", &mean_interarrival, &mean_service,
        &time_end);

    /* Write report heading and input parameters. */

    fprintf(outfile, "Single-server queueing system with fixed run");
    fprintf(outfile, " length\n\n");
    fprintf(outfile, "Mean interarrival time%11.3f minutes\n\n",
        mean_interarrival);
    fprintf(outfile, "Mean service time%16.3f minutes\n\n",
        mean_service);
    fprintf(outfile, "Length of the simulation%9.3f minutes\n\n",
        time_end);
}

```

FIGURE 1.47

C code for the main function, queueing model with fixed run length.

```

/* Initialize the simulation. */
initialize();

/* Run the simulation until it terminates after an end-simulation
   event (type 3) occurs. */
do (
    /* Determine the next event. */
    timing();

    /* Update time-average statistical accumulators. */
    update_time_avg_stats();

    /* Invoke the appropriate event function. */
    switch (next_event_type) (
        case 1:
            arrive();
            break;
        case 2:
            depart();
            break;
        case 3:
            report();
            break;
    )

    /* If the event just executed was not the end-simulation event
       (type 3), continue simulating. Otherwise, end the simulation.
       */

    ) while (next_event_type != 3);

    fclose(infile);
    fclose(outfile);

    return 0;
)

```

FIGURE 1.47
(Continued.)

The output file (named `mm1alt.out` if either the FORTRAN or C program was run) is shown in Fig. 1.50. The number of customer delays completed was 475 in this run, which seems reasonable in a 480-minute run where customers are arriving at an average rate of 1 per minute. The same three measures of performance are again numerically close to each other, but are all somewhat less than their earlier values in the 1000-delay simulation. A possible reason for this is that the current run is roughly only half as long as the earlier one, and since the initial conditions for the simulation are empty and idle (an uncongested state), the model in this shorter run has less chance to become congested. Again, however, this is just a single run and is thus subject to perhaps considerable uncertainty; there is no easy way to assess the degree of uncertainty from only a single run.

```

void initialize(void) /* Initialization function. */
{
    /* Initialize the simulation clock. */

    time = 0.0;

    /* Initialize the state variables. */

    server_status = IDLE;
    num_in_q = 0;
    time_last_event = 0.0;

    /* Initialize the statistical counters. */

    num_custs_delayed = 0;
    total_of_delays = 0.0;
    area_num_in_q = 0.0;
    area_server_status = 0.0;

    /* Initialize event list. Since no customers are present, the
       departure (service completion) event is eliminated from
       consideration. The end-simulation event (type 3) is scheduled
       for time time_end. */

    time_next_event[1] = time + expon(mean_interarrival);
    time_next_event[2] = 1.0e+30;
    time_next_event[3] = time_end;
}

```

FIGURE 1.48

C code for function initialize, queueing model with fixed run length.

```

void report(void) /* Report generator function. */
{
    /* Compute and write estimates of desired measures of performance.
       */

    fprintf(outfile, "\n\nAverage delay in queue%11.3f minutes\n\n",
            total_of_delays / num_custs_delayed);
    fprintf(outfile, "Average number in queue%10.3f\n\n",
            area_num_in_q / time);
    fprintf(outfile, "Server utilization%15.3f\n\n",
            area_server_status / time);
    fprintf(outfile, "Number of delays completed%7d",
            num_custs_delayed);
}

```

FIGURE 1.49

C code for function report, queueing model with fixed run length.

Single-server queueing system with fixed run length

Mean interarrival time	1.000 minutes
Mean service time	0.500 minutes
Length of the simulation	480.000 minutes

Average delay in queue	0.399 minutes
Average number in queue	0.394
Server utilization	0.464
Number of delays completed	475

FIGURE 1.50

Output report, queueing model with fixed run length.

If the queueing system being considered had actually been a one-operator barbershop open from 9 A.M. to 5 P.M., stopping the simulation after exactly 8 hours might leave a customer with hair partially cut. In such a case, we might want to close the door of the barbershop after 8 hours but continue to run the simulation until all customers present when the door closes (if any) have been served. The reader is asked in Prob. 1.10 to supply the program changes necessary to implement this stopping rule (see also Sec. 2.6).

1.4.9 Determining the Events and Variables

We defined an event in Sec. 1.3 as an instantaneous occurrence that may change the system state, and in the simple single-server queue of Sec. 1.4.1 it was not too hard to identify the events. However, the question sometimes arises, especially for complex systems, of how one determines the number and definition of events in general for a model. It may also be difficult to specify the state variables needed to keep the simulation running in the correct event sequence and to obtain the desired output measures. There is no completely general way to answer these questions, and different people may come up with different ways of representing a model in terms of events and variables, all of which may be correct. But there are some principles and techniques to help simplify the model's structure and to avoid logical errors.

Schruben (1983) presented an *event-graph* method, which was subsequently refined and extended by Sargent (1988) and Som and Sargent (1989). In this approach proposed events, each represented by a *node*, are connected by *directed arcs* (arrows) depicting how events may be scheduled from other events and from themselves. For example, in the queueing simulation of Sec. 1.4.3, the arrival event schedules another future occurrence of itself and (possibly) a departure event, and the departure event may schedule another future occurrence of itself; in addition, the arrival event must be

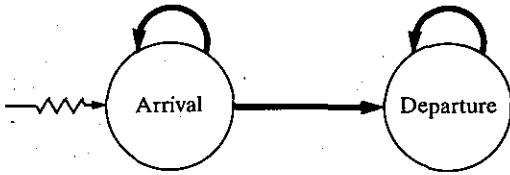


FIGURE 1.51
Event graph, queueing model.

initially scheduled in order to get the simulation going. Event graphs connect the proposed set of events (nodes) by arcs indicating the type of event scheduling that can occur. In Fig. 1.51 we show the event graph for our single-server queueing system, where the heavy smooth arrows indicate that an event at the end of the arrow *may* be scheduled from the event at the beginning of the arrow in a (possibly) *nonzero* amount of time, and the thin jagged arrow indicates that the event at its end is scheduled initially. Thus, the arrival event reschedules itself and may schedule a departure (in the case of an arrival who finds the server idle), and the departure event may reschedule itself (if a departure leaves behind someone else in queue).

For this model, it could be asked why we did not explicitly account for the act of a customer's entering service (either from the queue or upon arrival) as a separate event. This certainly happens, and it could cause the state to change (i.e., the queue length to fall by one). In fact, this could have been put in as a separate event without making the simulation incorrect, and would give rise to the event diagram in Fig. 1.52. The two thin smooth arrows each represent an event at the beginning of an arrow potentially scheduling an event at the end of the arrow without any intervening time, i.e., immediately; in this case the straight thin smooth arrow refers to a customer who arrives to an empty system and whose "enter-service" event is thus scheduled to occur immediately, and the curved thin smooth arrow represents a customer departing with a queue left behind, and so the first customer in the queue would be scheduled to enter service immediately. The number of events has now increased by one, and so we have a somewhat more complicated model. One of the uses of event graphs is to simplify a simulation's event structure by eliminating unnecessary events. There are several "rules" that allow for simplification, and one of them is that if an event node has incoming arcs that are all thin and smooth (i.e., the only way this event is scheduled is by other

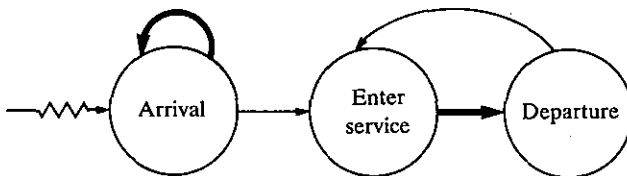


FIGURE 1.52
Event graph, queueing model with separate "enter-service" event.

events and without any intervening time), then this event can be eliminated from the model and its action built into the events that schedule it in zero time. Here, the “enter-service” event could be eliminated, and its action put partly into the arrival event (when a customer arrives to an idle server and begins service immediately) and partly into the departure event (when a customer finishes service and there is a queue from which the next customer is taken to enter service); this takes us back to the simpler event graph in Fig. 1.51. Basically, “events” that can happen only in conjunction with other events do not need to be in the model. Reducing the number of events not only simplifies model conceptualization, but may also speed its execution. Care must be taken, however, when “collapsing” events in this way to handle priorities and time ties appropriately.

Another rule has to do with initialization. The event graph is decomposed into *strongly connected* components, within each of which it is possible to “travel” from every node to every other node by following the arcs in their indicated directions. The graph in Fig. 1.51 decomposes into two strongly connected components (with a single node in each), and that in Fig. 1.52 has two strongly connected components (one of which is the arrival node by itself, and the other of which consists of the enter-service and departure nodes). The initialization rule states that in any strongly connected component of nodes that has no incoming arcs from other event nodes outside the component, there must be at least one node that is initially scheduled; if this rule were violated, it would never be possible to execute any of the events in the component. In Figs. 1.51 and 1.52, the arrival node is such a strongly connected component since it has no incoming arcs from other nodes, and so it must be initialized. Figure 1.53 shows the event graph for the queueing model of Sec. 1.4.8 with the fixed run length, for which we introduced the dummy “end simulation” event. Note that this event is itself a strongly connected component without any arcs coming in, and so it must be initialized, i.e., the end of the simulation is scheduled as part of the initialization. Failure to do so would result in erroneous termination of the simulation.

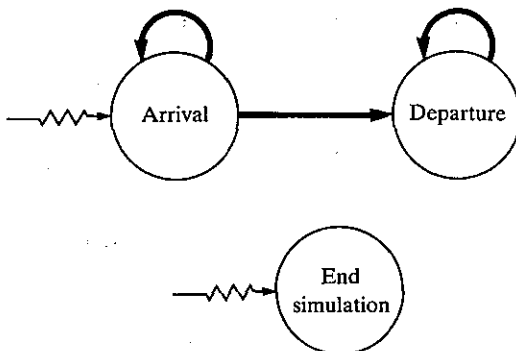


FIGURE 1.53

Event graph, queueing model with fixed run length.

There are several other features, including event-canceling relations, ways to combine similar events into one, refining the event-scheduling arcs to include conditional scheduling, and incorporating the state variables needed; see the original paper by Schruben (1983). Sargent (1988) and Som and Sargent (1989) extend and refine the technique, giving comprehensive illustrations involving a flexible manufacturing system and computer network models.

In modeling a system, the event-graph technique can be used to simplify the structure and to detect certain kinds of errors, and is especially useful in complex models involving a large number of interrelated events. Other considerations should also be kept in mind, such as continually asking why a particular state variable is needed; see Prob. 1.4.

1.5 SIMULATION OF AN INVENTORY SYSTEM

We shall now see how simulation can be used to compare alternative ordering policies for an inventory system. Many of the elements of our model are representative of those found in actual inventory systems.

1.5.1 Problem Statement

A company that sells a single product would like to decide how many items it should have in inventory for each of the next n months. The times between demands are IID exponential random variables with a mean of 0.1 month. The sizes of the demands, D , are IID random variables (independent of when the demands occur), with

$$D = \begin{cases} 1 & \text{w.p. } \frac{1}{6} \\ 2 & \text{w.p. } \frac{1}{3} \\ 3 & \text{w.p. } \frac{1}{3} \\ 4 & \text{w.p. } \frac{1}{6} \end{cases}$$

where w.p. is read "with probability."

At the beginning of each month, the company reviews the inventory level and decides how many items to order from its supplier. If the company orders Z items, it incurs a cost of $K + iZ$, where $K = \$32$ is the *setup cost* and $i = \$3$ is the *incremental cost* per item ordered. (If $Z = 0$, no cost is incurred.) When an order is placed, the time required for it to arrive (called the *delivery lag* or *lead time*) is a random variable that is distributed uniformly between 0.5 and 1 month.

The company uses a stationary (s, S) policy to decide how much to order, i.e.,

$$Z = \begin{cases} S - I & \text{if } I < s \\ 0 & \text{if } I \geq s \end{cases}$$

where I is the inventory level at the beginning of the month.

When a demand occurs, it is satisfied immediately if the inventory level is at least as large as the demand. If the demand exceeds the inventory level, the excess of demand over supply is backlogged and satisfied by future deliveries. (In this case, the new inventory level is equal to the old inventory level minus the demand size, resulting in a negative inventory level.) When an order arrives, it is first used to eliminate as much of the backlog (if any) as possible; the remainder of the order (if any) is added to the inventory.

So far we have discussed only one type of cost incurred by the inventory system, the ordering cost. However, most real inventory systems also have two additional types of costs, *holding* and *shortage* costs, which we discuss after introducing some additional notation. Let $I(t)$ be the inventory level at time t [note that $I(t)$ could be positive, negative, or zero], let $I^+(t) = \max\{I(t), 0\}$ be the number of items physically on hand in the inventory at time t [note that $I^+(t) \geq 0$], and let $I^-(t) = \max\{-I(t), 0\}$ be the backlog at time t [$I^-(t) \geq 0$ as well]. A possible realization of $I(t)$, $I^+(t)$, and $I^-(t)$ is shown in Fig. 1.54. The time points at which $I(t)$ decreases are the ones at which demands occur.

For our model, we shall assume that the company incurs a holding cost of $h = \$1$ per item per month held in (positive) inventory. The holding cost includes such costs as warehouse rental, insurance, taxes, and maintenance, as well as the opportunity cost of having capital tied up in inventory rather than invested elsewhere. We have ignored in our formulation the fact that some holding costs are still incurred when $I^+(t) = 0$. However, since our goal is to *compare* ordering policies, ignoring this factor, which after all is independent of the policy used, will not affect our assessment of which policy is best. Now, since $I^+(t)$ is the number of items held in inventory at time t , the time-average (per month) number of items held in inventory for the n -month period is

$$\bar{I}^+ = \frac{\int_0^n I^+(t) dt}{n}$$

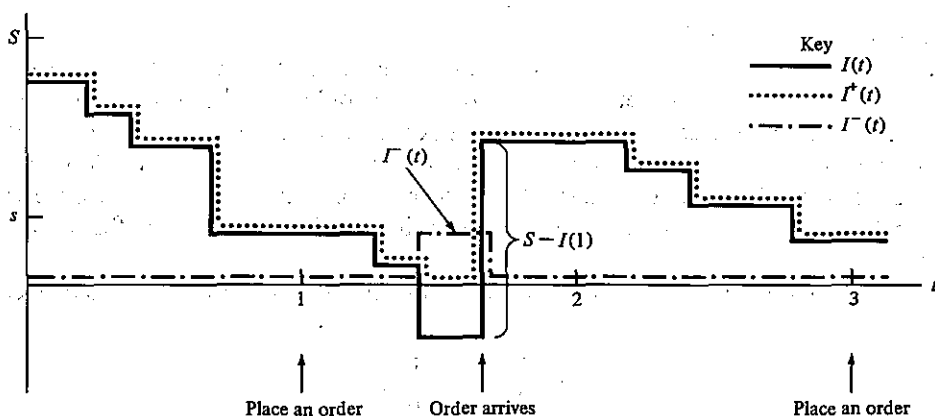


FIGURE 1.54

A realization of $I(t)$, $I^+(t)$, and $I^-(t)$ over time.

which is akin to the definition of the time-average number of customers in queue given in Sec. 1.4.1. Thus, the average holding cost per month is $h\bar{I}^+$.

Similarly, suppose that the company incurs a backlog cost of $\pi = \$5$ per item per month in backlog; this accounts for the cost of extra record keeping when a backlog exists, as well as loss of customers' goodwill. The time-average number of items in backlog is

$$\bar{I}^- = \frac{\int_0^n I^-(t) dt}{n}$$

so the average backlog cost per month is $\pi\bar{I}^-$.

Assume that the initial inventory level is $I(0) = 60$ and that no order is outstanding. We simulate the inventory system for $n = 120$ months and use the average total cost per month (which is the sum of the average ordering cost per month, the average holding cost per month, and the average shortage cost per month) to compare the following nine inventory policies:

<i>s</i>	20	20	20	20	40	40	40	60	60
<i>S</i>	40	60	80	100	60	80	100	80	100

We do not address here the issue of how these particular policies were chosen for consideration; statistical techniques for making such a determination are discussed in Chap. 12.

It should be noted that the state variables for a simulation model of this inventory system are the inventory level $I(t)$, the amount of an outstanding order from the company to the supplier, and the time of the last event [which is needed to compute the areas under the $I^+(t)$ and $I^-(t)$ functions].

1.5.2 Program Organization and Logic

Our model of the inventory system uses the following types of events:

Event description	Event type
Arrival of an order to the company from the supplier	1
Demand for the product from a customer	2
End of the simulation after n months	3
Inventory evaluation (and possible ordering) at the beginning of a month	4

We have chosen to make the end of the simulation event type 3 rather than type 4, since at time 120 both "end-simulation" and "inventory-evaluation" events will eventually be scheduled and we would like to execute the former event first at this time. (Since the simulation is over at time 120, there is no sense in evaluating the inventory and possibly ordering, incurring an ordering cost for an order that will never arrive.) The execution of event type 3 before event type 4 is guaranteed because the timing routines (in all three languages) give preference to the lowest-numbered event if two or more events are

scheduled to occur at the same time. In general, a simulation model should be designed to process events in an appropriate order when time ties occur. An event graph (see Sec. 1.4.9) appears in Fig. 1.55.

There are three types of random variates needed to simulate this system. The interdemand times are distributed exponentially, so the same algorithm (and code) as developed in Sec. 1.4 can be used here. The demand-size random variate D must be discrete, as described above, and can be generated as follows. First divide the unit interval into the contiguous subintervals $C_1 = [0, \frac{1}{6})$, $C_2 = [\frac{1}{6}, \frac{1}{2})$, $C_3 = [\frac{1}{2}, \frac{5}{6})$, and $C_4 = [\frac{5}{6}, 1]$, and obtain a $U(0, 1)$ random variate U from the random-number generator. If U falls in C_1 , return $D = 1$; if U falls in C_2 , return $D = 2$; and so on. Since the width of C_1 is $\frac{1}{6} - 0 = \frac{1}{6}$, and since U is uniformly distributed over $[0, 1]$, the probability that U falls in C_1 (and thus that we return $D = 1$) is $\frac{1}{6}$; this agrees with the desired probability that $D = 1$. Similarly, we return $D = 2$ if U falls in C_2 , having probability equal to the width of C_2 , $\frac{1}{2} - \frac{1}{6} = \frac{1}{3}$, as desired, and so on for the other intervals. The subprograms to generate the demand sizes all use this principle, and take as input the cutoff points defining the above subintervals, which are the *cumulative* probabilities of the distribution of D .

The delivery lags are uniformly distributed, but not over the unit interval $[0, 1]$. In general, we can generate a random variate distributed uniformly over any interval $[a, b]$ by generating a $U(0, 1)$ random number U , and then returning $a + U(b - a)$. That this method is correct seems intuitively clear, but will be formally justified in Sec. 8.3.1.

Of the four events, only three actually involve state changes (the end-simulation event being the exception). Since their logic is language-independent, we will describe it here.

The order-arrival event is flowcharted in Fig. 1.56, and must make the changes necessary when an order (which was previously placed) arrives from the supplier. The inventory level is increased by the amount of the order, and

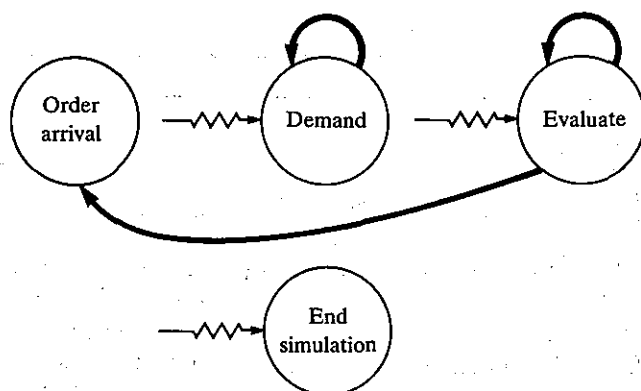


FIGURE 1.55
Event graph, inventory model.

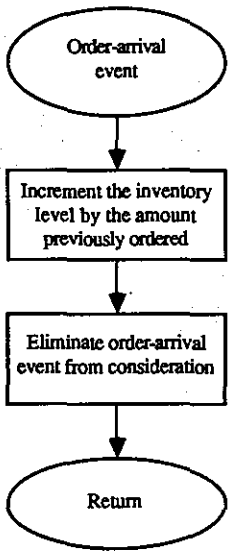


FIGURE 1.56
Flowchart for order-arrival routine, inventory model.

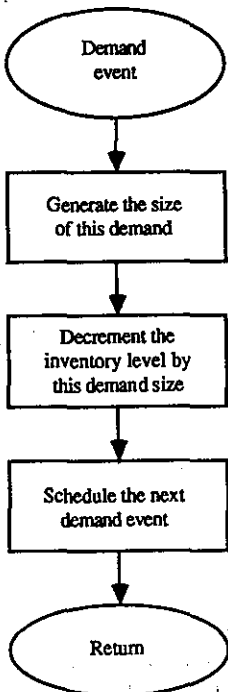


FIGURE 1.57
Flowchart for demand routine, inventory model.

the order-arrival event must be eliminated from consideration. (See Prob. 1.12 for consideration of the issue of whether there could be more than one order outstanding at a time for this model with these parameters.)

A flowchart for the demand event is given in Fig. 1.57, and processes the changes necessary to represent a demand's occurrence. First, the demand size is generated, and the inventory is decremented by this amount. Finally, the time of the next demand is scheduled into the event list. Note that this is the place where the inventory level might become negative.

The inventory-evaluation event, which takes place at the beginning of each month, is flowcharted in Fig. 1.58. If the inventory level $I(t)$ at the time of the evaluation is at least s , then no order is placed, and nothing is done except

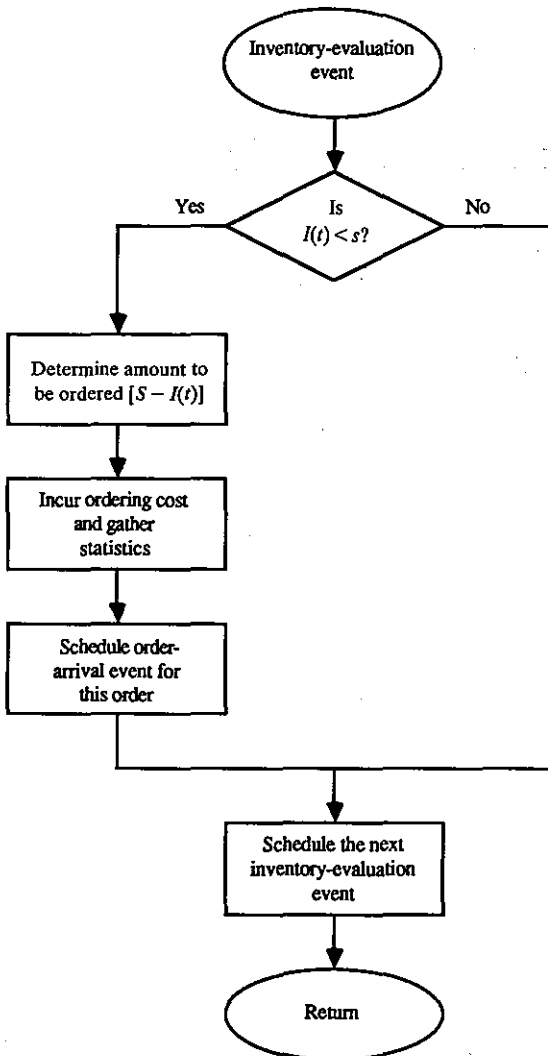


FIGURE 1.58

Flowchart for inventory-evaluation routine, inventory model.

to schedule the next evaluation into the event list. On the other hand, if $I(t) < s$, we want to place an order for $S - I(t)$ items. This is done by storing the amount of the order $[S - I(t)]$ until the order arrives, and scheduling its arrival time. In this case as well, we want to schedule the next inventory-evaluation event.

As in the single-server queueing model, it is convenient to write a separate nonevent routine to update the continuous-time statistical accumulators. For this model, however, doing so is slightly more complicated, so a flowchart for this activity appears in Fig. 1.59. The principal issue is whether we need to update the area under $I^-(t)$ or $I^+(t)$ (or neither). If the inventory level since the last event has been negative, then we have been in backlog, so the area under $I^-(t)$ only should be updated. On the other hand, if the inventory level has been positive, we need only update the area under $I^+(t)$. If the inventory level has been zero (a possibility), then neither update is needed. The code in each language for this routine also brings the variable for the time of the last event up to the present time. This routine will be invoked from the main program just after returning from the timing routine, regardless of the event type or whether the inventory level is actually changing at this point. This provides a simple (if not the most computationally efficient) way of updating integrals for continuous-time statistics.

Sections 1.5.3, 1.5.4, and 1.5.5, respectively, contain programs to simu-

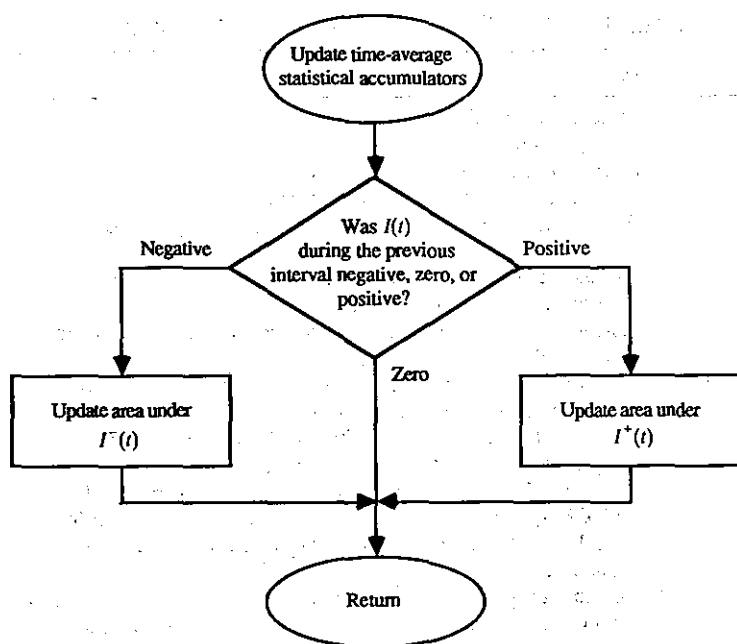


FIGURE 1.59

Flowchart for routine to update the continuous-time statistical accumulators, inventory model.

late this model in FORTRAN, Pascal, and C. As in the single-server queueing model, only one of these sections should be read, according to language preference. Neither the timing nor exponential-variate-generation subprograms will be shown, as they are the same as for the single-server queueing model in Sec. 1.4 (except for the FORTRAN version of TIMING, where the declarations file "mm1.dcl" must be changed to "inv.dcl" in the INCLUDE statement). The reader should also note the considerable similarity between the main programs of the queueing and inventory models in a given language.

1.5.3 FORTRAN Program

In addition to a main program, the model uses the subprograms and FORTRAN variables in Table 1.2.

The code for the main program is given in Fig. 1.60. After bringing in the declarations file (shown in Fig. 1.61) and declaring the local variables I and $NPOLCY$ to be INTEGER, the input and output files are opened, and the number of events, $NEVNTS$, is set to 4. The input parameters (except s and S) are then read in and written out, and a report heading is produced; for each (s, S) pair the simulation will then produce in subroutine REPORT a single line of output corresponding to this heading. Then a DO loop (with foot at label 60) is begun, each iteration of which performs an entire simulation for a

TABLE 1.2
Subroutines, functions, and FORTRAN variables for the inventory model

Subprogram	Purpose
INIT	Initialization routine
TIMING	Timing routine
ORDARV	Event routine to process type 1 events
DEMAND	Event routine to process type 2 events
REPORT	Event routine to process type 3 events (report generator)
EVALU8	Event routine to process type 4 events
UPTAVG	Subroutine to update areas under $I^+(t)$ and $I^-(t)$ functions just before each event occurrence
EXPON(RMEAN)	Function to generate an exponential random variate with mean RMEAN
IRANDI(NVALUE,PROBD)	Function to generate a random integer between 1 and NVALUE (a positive integer) in accordance with the distribution function PROBD(I) ($I = 1, 2, \dots, NVALUE$). If X is the random integer, the probability that X takes on a value less than or equal to I is given by PROBD(I). The values of NVALUE and PROBD(I) are set in the main program. (This particular format for IRANDI was chosen to make its use here consistent with Chap. 2.)
UNIFRM(A,B)	Function to generate a continuous random variate distributed uniformly between A and B (A and B are both real numbers, with $A < B$)
RAND(1)	Function to generate a uniform random variate between 0 and 1 (shown in Fig. 7.5)

TABLE 1.2
(Continued)

Variable	Definition
Input parameters:	
BIGS	S , second number in the specification of (s, S) inventory policy
H	h , value of unit holding cost (=1 here)
INCRMC	i , incremental cost per item ordered (=3)
INITIL	Initial inventory level (=60)
MAXLAG	Maximum delivery lag (=1.0)
MDEMDT	Mean interdemand time (=0.1)
MINLAG	Minimum delivery lag (=0.5)
NMNTHS	Length of the simulation in months (=120)
NPOLCY	Number of inventory policies being considered (=9)
NVALUE	Maximum possible demand size (=4)
PI	π , value of unit backlog cost (=5)
PROBD(I)	Probability that a demand is $\leq I$
SETUPC	K , setup cost of placing an order (=32)
SMALLS	s , first number in specification of (s, S) inventory policy
Modeling variables:	
AMINUS	Area under the $I^-(t)$ function so far
AMOUNT	Amount, Z , ordered by company from supplier
APLUS	Area under the $I^+(t)$ function so far
DSIZE	A particular demand size
INVLEV	$I(t)$, the inventory level
NEVNTS	Number of event types for model (i.e., 4)
NEXT	Event type (1, 2, 3, or 4) of the next event to occur
TIME	Simulation clock
TLEVNT	Time of the last (most recent) event
TNE(I)	Time of the next event of type I ($I = 1, 2, 3, 4$)
TORDC	Total ordering cost
TSLE	Time since last event
Output variables:	
ACOST	Average total cost per month
AHLDC	Average holding cost per month
AORDC	Average ordering cost per month
ASHRC	Average shortage cost per month

given (s, S) pair; the first thing done in the loop is to read the next (s, S) pair. The model is initialized by a call to INIT, and TIMING is used to determine the next event type, NEXT, and to update the simulation clock, TIME. After returning from TIMING with the next event type, a call is made to UPTAVG to update the continuous-time statistics before executing the event routine itself. A computed GO TO is then used as before to transfer control to the appropriate event routine; if the event is not the end-simulation event, control is passed back to statement 10 (the call to TIMING) and the simulation continues. If the simulation is over (NEXT = 3), REPORT is called and the simulation is ended for the current (s, S) pair.

Subroutine INIT is listed in Fig. 1.62. Observe that the first inventory evaluation is scheduled at TIME = 0 since, in general, the initial inventory

84 SIMULATION MODELING AND ANALYSIS

```

*   Main program for inventory system.

*   Bring in declarations file and declare local variables.
    INCLUDE 'inv.dcl'
    INTEGER I,NPOLCY

*   Open input and output files.

    OPEN (5, FILE = 'inv.in')
    OPEN (6, FILE = 'inv.out')

*   Specify the number of event types for the timing routine.

    NEVNTS = 4

*   Read input parameters.

    READ (5,*) INITIL, NMNTHS, NPOLCY, NVALUE, MDEMDT, SETUPC, INCRMC,
&      H, PI, MINLAG, MAXLAG
    READ (5,*) (PROBD(I), I = 1, NVALUE)

*   Write report heading and input parameters.

    WRITE (6,2010) INITIL, NVALUE, (PROBD(I), I = 1, NVALUE)
2010 FORMAT (' Single-product inventory system'//
&      ' Initial inventory level',I24,' items'//
&      ' Number of demand sizes',I25//
&      ' Distribution function of demand sizes',2X,8F8.3)
    WRITE (6,2020) MDEMDT, MINLAG, MAXLAG, NMNTHS, SETUPC, INCRMC, H,
&      PI, NPOLCY
2020 FORMAT (/ ' Mean interdemand time',F26.2,' months'//
&      ' Delivery lag range',F29.2,' to',F10.2,' months'//
&      ' Length of the simulation',I23,' months'//
&      ' K =',F6.1,' i =',F6.1,' h =',F6.1,' pi =',F6.1//
&      ' Number of policies',I29//
&      10X,4(8X,'Average')//
&      ' Policy      total cost      ordering cost',
&      ' holding cost  shortage cost')

*   Run the simulation varying the inventory policy.

    DO 60 I = 1, NPOLCY

*       Read the inventory policy, and initialize the simulation.

        READ (5,*) SMALLS, BIGS
        CALL INIT

*       Determine the next event.

10      CALL TIMING

*       Update time-average statistical accumulators.

        CALL UPTAVG

*       Call the appropriate event routine.

        GO TO (20, 30, 50, 40), NEXT
20      CALL ORDARV
        GO TO 10
30      CALL DEMAND
        GO TO 10
40      CALL EVALUS
        GO TO 10
50      CALL REPORT

60 CONTINUE

    CLOSE (5)
    CLOSE (6)

    STOP
    END

```

FIGURE 1.60

FORTRAN code for the main program, inventory model.

```

INTEGER AMOUNT,BIGS,INITIL,INVLEV,NEVNTS,NEXT,NMNTHS,NVALUE,
&      SMALLS
REAL AMINUS,APLUS,H,INCRMC,MAXLAG,MDEMDT,MINLAG,PI,PROBD(25),
&      SETUPC,TIME,TLEVNT,TNE(4),TORDC
INTEGER IRANDI
REAL EXPON,UNIFRM
COMMON /MODEL/ AMINUS,AMOUNT,APLUS,BIGS,H,INCRMC,INITIL,INVLEV,
&      MAXLAG,MDEMDT,MINLAG,NEVNTS,NEXT,NMNTHS,NVALUE,PI,
&      PROBD,SETUPC,SMALLS,TIME,TLEVNT,TNE,TORDC

```

FIGURE 1.61

FORTRAN code for the declarations file (inv.dcl), inventory model.

```

SUBROUTINE INIT
INCLUDE 'inv.dcl'

*      Initialize the simulation clock.

TIME    = 0.0

*      Initialize the state variables.

INVLEV = INITIL
TLEVNT = 0.0

*      Initialize the statistical counters.

TORDC   = 0.0
APLUS   = 0.0
AMINUS  = 0.0

*      Initialize the event list. Since no order is outstanding, the
*      order-arrival event is eliminated from consideration.

TNE(1) = 1.0E+30
TNE(2) = TIME + EXPON(MDEMDT)
TNE(3) = NMNTHS
TNE(4) = 0.0

RETURN
END

```

FIGURE 1.62

FORTRAN code for subroutine INIT, inventory model.

```

SUBROUTINE ORDARV
INCLUDE 'inv.dcl'

*      Increment the inventory level by the amount ordered.

INVLEV = INVLEV + AMOUNT

*      Since no order is now outstanding, eliminate the order-arrival
*      event from consideration.

TNE(1) = 1.0E+30

RETURN
END

```

FIGURE 1.63

FORTRAN code for subroutine ORDARV, inventory model.

```

SUBROUTINE DEMAND
INCLUDE 'inv.dcl'
INTEGER DSIZE

*   Generate the demand size.

DSIZE = IRANDI(NVALUE,PROBD)

*   Decrement the inventory level by the demand size.

INVLEV = INVLEV - DSIZE

*   Schedule the time of the next demand.

TNE(2) = TIME + EXPON(MDEMDT)

RETURN
END

```

FIGURE 1.64

FORTRAN code for subroutine DEMAND, inventory model.

level could be less than s . Note also that event type 1 (order arrival) is eliminated from consideration, since our modeling assumption was that there are no outstanding orders initially.

The event routines ORDARV, DEMAND, and EVALU8 are shown in Figs. 1.63 through 1.65, and correspond to the general discussion given in Sec. 1.5.2, and to the flowcharts in Figs. 1.56 through 1.58. Note that in DEMAND, the demand-size variate is generated from the function IRANDI. Also, in EVALU8, note that the variable TORDC is increased by the ordering cost for any order that might be placed here.

```

SUBROUTINE EVALU8
INCLUDE 'inv.dcl'

*   Check whether the inventory level is less than SMALLS.

IF (INVLEV .LT. SMALLS) THEN

*   The inventory level is less than SMALLS, so place an order for
*   the appropriate amount.

    AMOUNT = BIGS - INVLEV
    TORDC = TORDC + SETUPC + INCRMC * AMOUNT

*   Schedule the arrival of the order.

    TNE(1) = TIME + UNIFRM(MINLAG,MAXLAG)

END IF

*   Regardless of the place-order decision, schedule the next
*   inventory evaluation.

TNE(4) = TIME + 1.0

RETURN
END

```

FIGURE 1.65

FORTRAN code for subroutine EVALU8, inventory model.

```

SUBROUTINE REPORT
INCLUDE 'inv.dcl'
REAL ACOST,AHLDC,AORDC,ASHRC

*   Compute and write estimates of desired measures of performance.

AORDC = TORDC / NMNTHS
AHLDC = H * APLUS / NMNTHS
ASHRC = PI * AMINUS / NMNTHS
ACOST = AORDC + AHLDC + ASHRC
WRITE (6,2010) SMALLS, BIGS, ACOST, AORDC, AHLDC, ASHRC
2010 FORMAT (' (' ,I3,',',',I3,')',4F15.2)

RETURN
END

```

FIGURE 1.66

FORTRAN code for subroutine REPORT, inventory model.

The report generator is listed in Fig. 1.66, and computes the three components of the total cost separately, adding them together to get the average total cost per month, ACOST. The current values of s and S are written out for identification purposes, along with the average total cost and its three components (ordering, holding, and shortage costs).

Subroutine UPTAVG, which was discussed in general in Sec. 1.5.2 and flowcharted in Fig. 1.59, is shown in Fig. 1.67. Implementation is facilitated in FORTRAN by the use of an *arithmetic if* statement (a fairly old and seldom-used feature), which ends with three statement labels (10, 20, and 30 in this case) to which control is transferred if the argument (simply INVLEV here) is negative, zero, or positive, in that order. As in the single-server queueing model of Sec. 1.4, it might be necessary to make both the TIME and TLEVNT

```

SUBROUTINE UPTAVG
INCLUDE 'inv.dcl'
REAL TSLE

*   Compute time since last event, and update last-event-time marker.

TSLE = TIME - TLEVNT
TLEVNT = TIME

*   Determine the status of the inventory level during the previous
*   interval. If the inventory level during the previous interval was
*   negative, update AMINUS. If it was zero, no update is needed. If
*   it was positive, update APLUS.

IF (INVLEV) 10, 20, 30
10  AMINUS = AMINUS - INVLEV * TSLE
20  RETURN
30  APLUS = APLUS + INVLEV * TSLE

RETURN
END

```

FIGURE 1.67

FORTRAN code for subroutine UPTAVG, inventory model.


```

      INTEGER FUNCTION IRANDI(NVALUE,PROBD)
      INTEGER I,NVALUE
      REAL PROBD(1),U
      REAL RAND

*      Generate a U(0,1) random variate.

      U = RAND(1)

*      Return a random integer between 1 and NVALUE in accordance with
*      the (cumulative) distribution function PROBD.

      DO 10 I = 1, NVALUE - 1
        IF (U .LT. PROBD(I)) THEN
          IRANDI = I
          RETURN
        END IF
10    CONTINUE
      IRANDI = NVALUE

      RETURN
      END

```

FIGURE 1.68

FORTRAN code for function IRANDI.

variables **DOUBLE PRECISION** to avoid severe roundoff error in their subtraction at the top of the routine if the simulation is to be run for a long period of simulated time.

The code for function **IRANDI** is given in Fig. 1.68, and is general in that it will generate an integer between 1 and **NVALUE** according to distribution function **PROBD(I)**, provided that **NVALUE** and **PROBD(I)** ($I = 1, 2, \dots, \text{NVALUE}$) are specified. [In our case, $\text{NVALUE} = 4$ and $\text{PROBD}(1) = \frac{1}{6}$, $\text{PROBD}(2) = \frac{1}{2}$, $\text{PROBD}(3) = \frac{5}{6}$, and $\text{PROBD}(4) = 1$, all specified to three-decimal accuracy on input.] The logic agrees with the discussion in Sec. 1.5.2; note that the input array **PROBD** must contain the *cumulative* distribution function rather than the probabilities that the variate takes on its possible values.

The function **UNIFRM** is given in Fig. 1.69, and is as described in Sec. 1.5.2.

```

      REAL FUNCTION UNIFRM(A,B)
      REAL A,B,U
      REAL RAND

*      Generate a U(0,1) random variate.

      U = RAND(1)

*      Return a U(A,B) random variate.

      UNIFRM = A + U * (B - A)

      RETURN
      END

```

FIGURE 1.69

FORTRAN code for function UNIFRM.

1.5.4 Pascal Program

The global declarations are shown in Fig. 1.70. The array ProbDistribDemand will be used to hold the cumulative probabilities for the demand sizes, and is passed into the random-integer-generation function RandomInteger. As for the queueing model, we must define the array Zrng and the four functions and procedures at the bottom of Fig. 1.70 for the random-number generator of Fig. 7.6.

Procedure Initialize appears in Fig. 1.71. Observe that the first inventory evaluation is scheduled at Time = 0 since, in general, the initial inventory level could be less than s . Note also that event type 1 (order arrival) is eliminated from consideration, since our modeling assumption was that there are no outstanding orders initially.

The event routines OrderArrival, Demand, and Evaluate are shown in Figs. 1.72 through 1.74, and correspond to the general discussion given in Sec.

```

PROGRAM Inventory(Input, Output);

( Global declarations for inventory system. )

TYPE
  DistribArray = ARRAY [1..25] OF Real;

VAR
  Amount, Bigs, DemandIndex, InitialInvLevel, InvLevel, NextEventType,
    NumEvents, NumMonths, NumPolicies, NumValuesDemand, Policy, Smalls
    : Integer;
  AreaHolding, AreaShortage, HoldingCost, IncrementalCost, Maxlag,
    MeanInterdemand, Minlag, SetupCost, ShortageCost, Time,
    TimeLastEvent, TotalOrderingCost : Real;
  ProbDistribDemand : DistribArray;
  TimeNextEvent : ARRAY [1..4] OF Real;

( The following declaration is for the random-number generator.
  Note that the name Zrng must not be used for any other purpose. )

  Zrng : ARRAY [1..100] OF Integer;

PROCEDURE Initialize; FORWARD;
PROCEDURE Timing; FORWARD;
PROCEDURE OrderArrival; FORWARD;
PROCEDURE Demand; FORWARD;
PROCEDURE Evaluate; FORWARD;
PROCEDURE Report; FORWARD;
PROCEDURE UpdateTimeAvgStats; FORWARD;
FUNCTION Expon(Mean : Real) : Real; FORWARD;
FUNCTION RandomInteger
  (ProbDistrib : DistribArray) : Integer; FORWARD;
FUNCTION Uniform(A, B : Real) : Real; FORWARD;

( The following four declarations are for the random-number generator.
  )

PROCEDURE Randdf; FORWARD;
FUNCTION Rand(Stream : Integer) : Real; FORWARD;
PROCEDURE Randst(Zset : Integer; Stream : Integer); FORWARD;
FUNCTION Randgt(Stream : Integer) : Integer; FORWARD;

```

FIGURE 1.70

Pascal code for the global declarations, inventory model.

```

PROCEDURE Initialize; { Initialization procedure. }

BEGIN { Initialize }

  { Initialize the simulation clock. }

  Time := 0.0;

  { Initialize the state variables. }

  InvLevel      := InitialInvLevel;
  TimeLastEvent := 0.0;

  { Initialize the statistical counters. }

  TotalOrderingCost := 0.0;
  AreaHolding       := 0.0;
  AreaShortage      := 0.0;

  { Initialize the event list. Since no order is outstanding, the
    order-arrival event is eliminated from consideration. }

  TimeNextEvent[1] := 1.0E+30;
  TimeNextEvent[2] := Time + Expon(MeanInterdemand);
  TimeNextEvent[3] := NumMonths;
  TimeNextEvent[4] := 0.0

END; { Initialize }

```

FIGURE 1.71

Pascal code for procedure Initialize, inventory model.

1.5.2, and to the flowcharts in Figs. 1.56 through 1.58. In Evaluate, note that the variable TotalOrderingCost is increased by the ordering cost for any order that might be placed here.

The report generator is listed in Fig. 1.75, and computes the three components of the total cost separately, adding them together to get the average total cost per month, AvgTotCost. The current values of s and S are written out for identification purposes, along with the average total cost and its three components (ordering, holding, and shortage costs).

```

PROCEDURE OrderArrival; { Order arrival event procedure. }

BEGIN { OrderArrival }

  { Increment the inventory level by the amount ordered. }

  InvLevel := InvLevel + Amount;

  { Since no order is now outstanding, eliminate the order-arrival
    event from consideration. }

  TimeNextEvent[1] := 1.0E+30

END; { OrderArrival }

```

FIGURE 1.72

Pascal code for procedure OrderArrival, inventory model.

```

PROCEDURE Demand; { Demand event procedure. }

VAR
  SizeDemand : Integer;

BEGIN { Demand }

  { Generate the demand size. }
  SizeDemand := RandomInteger(ProbDistribDemand);

  { Decrement the inventory level by the demand size. }
  InvLevel := InvLevel - SizeDemand;

  { Schedule the time of the next demand. }
  TimeNextEvent[2] := Time + Expon(MeanInterdemand)

END; { Demand }

```

FIGURE 1.73

Pascal code for procedure Demand, inventory model.

```

PROCEDURE Evaluate; { Inventory-evaluation event procedure. }

BEGIN { Evaluate }

  { Check whether the inventory level is less than Smalls. }
  IF InvLevel < Smalls THEN BEGIN

    { The inventory level is less than Smalls, so place an order
      for the appropriate amount. }

    Amount := Bigs - InvLevel;
    TotalOrderingCost := TotalOrderingCost + SetupCost +
      IncrementalCost * Amount;

    { Schedule the arrival of the order. }
    TimeNextEvent[1] := Time + Uniform(Minlag, Maxlag)

  END;

  { Regardless of the place-order decision, schedule the next
    inventory evaluation. }
  TimeNextEvent[4] := Time + 1.0

END; { Evaluate }

```

FIGURE 1.74

Pascal code for procedure Evaluate, inventory model.

```
PROCEDURE Report; { Report generator procedure. }
```

```
VAR
    AvgHoldingCost, AvgOrderingCost, AvgShortageCost, AvgTotCost :
        Real;

BEGIN { Report }

    { Compute and write estimates of desired measures of performance.
      }

    AvgOrderingCost := TotalOrderingCost / NumMonths;
    AvgHoldingCost  := HoldingCost * AreaHolding / NumMonths;
    AvgShortageCost := ShortageCost * AreaShortage / NumMonths;
    AvgTotCost      := AvgOrderingCost + AvgHoldingCost +
        AvgShortageCost;

    Writeln;
    Writeln('(', Smalls:3, ',', Bigs:3, ')', AvgTotCost:15:2,
        AvgOrderingCost:15:2, AvgHoldingCost:15:2,
        AvgShortageCost:15:2)

    END; { Report }
```

FIGURE 1.75

Pascal code for procedure Report, inventory model.

Procedure UpdateTimeAvgStats, which was discussed in general in Sec. 1.5.2 and flowcharted in Fig. 1.59, is shown in Fig. 1.76. Note that if the inventory level *InvLevel* is zero, neither the IF nor the ELSE IF condition is satisfied, resulting in no update at all, as desired. As in the single-server queueing model of Sec. 1.4, it might be necessary to make both the *Time* and *TimeLastEvent* variables double precision (if available) to avoid severe round-

```
PROCEDURE UpdateTimeAvgStats; { Update area accumulators for
                                time-average statistics. }
```

```
VAR
    TimeSinceLastEvent : Real;

BEGIN { UpdateTimeAvgStats }

    { Compute time since last event, and update last-event-time
      marker. }

    TimeSinceLastEvent := Time - TimeLastEvent;
    TimeLastEvent      := Time;

    { Determine the status of the inventory level during the previous
      interval. If the inventory level during the previous interval
      was negative, update AreaShortage. If it was positive, update
      AreaHolding. If it was zero, no update is needed. }

    IF InvLevel < 0 THEN
        AreaShortage := AreaShortage - InvLevel * TimeSinceLastEvent
    ELSE IF InvLevel > 0 THEN
        AreaHolding  := AreaHolding + InvLevel * TimeSinceLastEvent

    END; { UpdateTimeAvgStats }
```

FIGURE 1.76

Pascal code for procedure UpdateTimeAvgStats, inventory model.

```

FUNCTION RandomInteger; { Random integer generation function. }
                        { Pass in Real array ProbDistrib giving
                        cumulative probability distribution
                        function, as declared in FORWARD
                        declarations earlier. }

VAR
  I : Integer;
  U : Real;

BEGIN { RandomInteger }

  { Generate a U(0,1) random variate. }

  U := Rand(1);

  { Return a random integer in accordance with the (cumulative)
  distribution function ProbDistrib. }

  I := 0;
  REPEAT
    I := I + 1
  UNTIL U < ProbDistrib[I];

  RandomInteger := I

END; { RandomInteger }

```

FIGURE 1.77

Pascal code for function RandomInteger.

off error in their subtraction at the top of the routine if the simulation is to be run for a long period of simulated time.

The code for function RandomInteger is given in Fig. 1.77, and is general in that it will generate an integer according to distribution function ProbDistrib[I], provided that the values of ProbDistrib[I] are specified. (In our case, ProbDistrib[1] = $\frac{1}{6}$, ProbDistrib[2] = $\frac{1}{2}$, ProbDistrib[3] = $\frac{5}{6}$, and ProbDistrib[4] = 1, all specified to three-decimal accuracy on input.) The logic agrees with the discussion in Sec. 1.5.2; note that the input array ProbDistrib

```

FUNCTION Uniform; { Uniform variate generation function. }
                  { Pass in Real parameters A and B giving left and
                  right endpoints, as declared in FORWARD
                  declarations earlier. }

VAR
  U : Real;

BEGIN { Uniform }

  { Generate a U(0,1) random variate. }

  U := Rand(1);

  { Return a U(A,B) random variate. }

  Uniform := A + U * (B - A)

END; { Uniform }

```

FIGURE 1.78

Pascal code for function Uniform.

must contain the *cumulative* distribution function rather than the probabilities that the variate takes on its possible values.

The function Uniform is given in Fig. 1.78, and is as described in Sec. 1.5.2.

The code for the main program is given in Fig. 1.79. After initializing the random-number generator by invoking Randdf, the number of events is set to 4. The input parameters (except s and S) are then read in and written out, and a report heading is produced; for each (s, S) pair the simulation will then

```
BEGIN { Inventory main program. }

  { Initialize the random-number generator. }

  Randdf;

  { Specify the number of events for the timing procedure. }

  NumEvents := 4;

  { Read input parameters. }

  Readln(InitialInvLevel, NumMonths, NumPolicies, NumValuesDemand);
  Readln(MeanInterdemand, SetupCost, IncrementalCost, HoldingCost,
    ShortageCost, Minlag, Maxlag);
  FOR DemandIndex := 1 TO NumValuesDemand DO
    Read(ProbDistribDemand[DemandIndex]);
  Readln;

  { Write report heading and input parameters. }

  Writeln('Single-product inventory system');
  Writeln;
  Writeln('Initial inventory level', InitialInvLevel:24, ' items');
  Writeln;
  Writeln('Number of demand sizes', NumValuesDemand:25);
  Writeln;
  Write ('Distribution function of demand sizes ');
  FOR DemandIndex := 1 TO NumValuesDemand DO
    Write(ProbDistribDemand[DemandIndex]:8:3);
  Writeln;
  Writeln;
  Writeln('Mean interdemand time', MeanInterdemand:26:2, ' months');
  Writeln;
  Writeln('Delivery lag range', Minlag:29:2, ' to', Maxlag:10:2,
    ' months');
  Writeln;
  Writeln('Length of the simulation', NumMonths:23, ' months');
  Writeln;
  Writeln('K =', SetupCost:6:1, ' i =', IncrementalCost:6:1,
    ' h =', HoldingCost:6:1, ' pi =', ShortageCost:6:1);
  Writeln;
  Writeln('Number of policies', NumPolicies:29);
  Writeln;
  Write ('          Average          Average');
  Writeln('          Average          Average');
  Write (' Policy total cost ordering cost');
  Writeln(' holding cost shortage cost');
```

FIGURE 1.79

Pascal code for the main program, inventory model.


```

{ Run the simulation varying the inventory policy. }

FOR Policy := 1 TO NumPolicies DO BEGIN
    { Read the inventory policy, and initialize the simulation. }
    Readln(Small, Big);
    Initialize;

    { Run the simulation until it terminates after an end-simulation
      event (type 3) occurs. }

    REPEAT
        { Determine the next event. }

        Timing;

        { Update time-average statistical accumulators. }

        UpdateTimeAvgStats;

        { Invoke the appropriate event procedure. }

        CASE NextEventType OF
            1: OrderArrival;
            2: Demand;
            4: Evaluate;
            3: Report
        END

        { If the event just executed was the end-simulation event (type
          3), end the simulation for this (s,S) pair and go on to the next
          pair (if any). Otherwise, continue simulating for this (s,S)
          pair. }

    UNTIL NextEventType = 3

END

END. { Inventory }

```

FIGURE 1.79

(Continued.)

produce in procedure Report a single line of output corresponding to this heading. Then a FOR loop is begun, each iteration of which performs an entire simulation for a given (s, S) pair; the first thing done in the loop is to read the next (s, S) pair. The model is initialized, and a REPEAT UNTIL loop is used to keep simulating until a type 3 (end-simulation) event occurs, as in Sec. 1.4.8. Inside this loop, Timing is used to determine the next event type and to update the simulation clock. After returning from Timing with the next event type, the continuous-time statistics are updated before executing the event routine itself. A CASE statement is then used as before to transfer control to the appropriate event routine. Unlike the fixed-time stopping rule of Sec. 1.4.8, when the REPEAT UNTIL loop ends here we do not stop the program, but go to the next step of the enclosing FOR loop to read in the next (s, S) pair and do a separate simulation; the entire program stops only when the FOR loop is over and there are no more (s, S) pairs to consider.

1.5.5 C Program

The external definitions are shown in Fig. 1.80. The array `prob_distrib_demand` will be used to hold the cumulative probabilities for the demand sizes, and is passed into the random-integer-generation function `random_integer`. As for the queuing model, we must include the header file `rand.h` (in Fig. 7.8) for the random-number generator of Fig. 7.7.

The code for the main function is given in Fig. 1.81. After opening the input and output files, the number of events is set to 4. The input parameters (except s and S) are then read in and written out, and a report heading is produced; for each (s, S) pair the simulation will then produce in the report function a single line of output corresponding to this heading. Then a "for" loop is begun, each iteration of which performs an entire simulation for a given (s, S) pair; the first thing done in the loop is to read the next (s, S) pair. The model is initialized, and a "do while" loop is used to keep simulating as long as the type 3 (end-simulation) event does not occur, as in Sec. 1.4.8. Inside this loop, the timing function is used to determine the next event type and to update the simulation clock. After returning from timing with the next event type, the continuous-time statistics are updated before executing the event routine itself. A "switch" statement is then used as before to transfer control to the appropriate event routine. Unlike the fixed-time stopping rule of Sec. 1.4.8, when the "do while" loop ends here we do not stop the program, but go to the next step of the enclosing "for" loop to read in the next (s, S) pair and do a separate simulation; the entire program stops only when the "for" loop is over and there are no more (s, S) pairs to consider.

```
/* External definitions for inventory system. */

#include <stdio.h>
#include <math.h>
#include "rand.h" /* Header file for random-number generator. */

int amount, bigs, initial_inv_level, inv_level, next_event_type,
    num_events, num_months, num_values_demand, smalls;
float area_holding, area_shortage, holding_cost, incremental_cost,
    maxlag, mean_interdemand, minlag, prob_distrib_demand[26],
    setup_cost, shortage_cost, time, time_last_event,
    time_next_event[5], total_ordering_cost;
FILE *infile, *outfile;

void initialize(void);
void timing(void);
void order_arrival(void);
void demand(void);
void evaluate(void);
void report(void);
void update_time_avg_stats(void);
float expon(float mean);
int random_integer(float prob_distrib []);
float uniform(float a, float b);
```

FIGURE 1.80

C code for the external definitions, inventory model.

```

main() /* Main function. */
{
    int i, num_policies;

    /* Open input and output files. */

    infile = fopen("inv.in", "r");
    outfile = fopen("inv.out", "w");

    /* Specify the number of events for the timing function. */

    num_events = 4;

    /* Read input parameters. */

    fscanf(infile, "%d %d %d %d %f %f %f %f %f %f",
           &initial_inv_level, &num_months, &num_policies,
           &num_values_demand, &mean_interdemand, &setup_cost,
           &incremental_cost, &holding_cost, &shortage_cost, &minlag,
           &maxlag);
    for (i = 1; i <= num_values_demand; ++i)
        fscanf(infile, "%f", &prob_distrib_demand[i]);

    /* Write report heading and input parameters. */

    fprintf(outfile, "Single-product inventory system\n\n");
    fprintf(outfile, "Initial inventory level%24d items\n\n",
           initial_inv_level);
    fprintf(outfile, "Number of demand sizes%25d\n\n",
           num_values_demand);
    fprintf(outfile, "Distribution function of demand sizes ");
    for (i = 1; i <= num_values_demand; ++i)
        fprintf(outfile, "%8.3f", prob_distrib_demand[i]);
    fprintf(outfile, "\n\nMean interdemand time%26.2f\n\n",
           mean_interdemand);
    fprintf(outfile, "Delivery lag range%29.2f to%10.2f months\n\n",
           minlag, maxlag);
    fprintf(outfile, "Length of the simulation%23d months\n\n",
           num_months);
    fprintf(outfile, "K=%6.1f i=%6.1f h=%6.1f pi=%6.1f\n\n",
           setup_cost, incremental_cost, holding_cost, shortage_cost);
    fprintf(outfile, "Number of policies%29d\n\n", num_policies);
    fprintf(outfile, "           Average           Average");
    fprintf(outfile, "           Average\n\n");
    fprintf(outfile, "Policy           total cost   ordering cost");
    fprintf(outfile, "   holding cost   shortage cost");

    /* Run the simulation varying the inventory policy. */

    for (i = 1; i <= num_policies; ++i) {

        /* Read the inventory policy, and initialize the simulation. */

        fscanf(infile, "%d %d", &smalls, &biggs);
        initialize();

        /* Run the simulation until it terminates after an
           end-simulation event (type 3) occurs. */

        do {

            /* Determine the next event. */

            timing();

```

FIGURE 1.81
C code for the main function, inventory model.

```

/* Update time-average statistical accumulators. */
update_time_avg_stats();

/* Invoke the appropriate event function. */
switch (next_event_type) {
    case 1:
        order_arrival();
        break;
    case 2:
        demand();
        break;
    case 4:
        evaluate();
        break;
    case 3:
        report();
        break;
}

/* If the event just executed was not the end-simulation event
   (type 3), continue simulating. Otherwise, end the
   simulation for the current (s,S) pair and go on to the next
   pair (if any). */
} while (next_event_type != 3);

}

/* End the simulations. */
fclose(infile);
fclose(outfile);

return 0;
}

```

FIGURE 1.81
(Continued.)

The initialization function appears in Fig. 1.82. Observe that the first inventory evaluation is scheduled at time 0 since, in general, the initial inventory level could be less than s . Note also that event type 1 (order arrival) is eliminated from consideration, since our modeling assumption was that there are no outstanding orders initially.

The event functions `order_arrival`, `demand`, and `evaluate` are shown in Figs. 1.83 through 1.85, and correspond to the general discussion given in Sec. 1.5.2, and to the flowcharts in Figs. 1.56 through 1.58. In `evaluate`, note that the variable `total_ordering_cost` is increased by the ordering cost for any order that might be placed here.

The report generator is listed in Fig. 1.86, and computes the three components of the total cost separately, adding them together to get the average total cost per month. The current values of s and S are written out for identification purposes, along with the average total cost and its three components (ordering, holding, and shortage costs).

Function `update_time_avg_stats`, which was discussed in general in Sec. 1.5.2 and flowcharted in Fig. 1.59, is shown in Fig. 1.87. Note that if the

```

void initialize(void) /* Initialization function. */
{
    /* Initialize the simulation clock. */
    time = 0.0;

    /* Initialize the state variables. */
    inv_level      = initial_inv_level;
    time_last_event = 0.0;

    /* Initialize the statistical counters. */
    total_ordering_cost = 0.0;
    area_holding        = 0.0;
    area_shortage       = 0.0;

    /* Initialize the event list. Since no order is outstanding, the
       order-arrival event is eliminated from consideration. */
    time_next_event[1] = 1.0e+30;
    time_next_event[2] = time + expon(mean_interdemand);
    time_next_event[3] = num_months;
    time_next_event[4] = 0.0;
}

```

FIGURE 1.82

C code for function initialize, inventory model.

```

void order_arrival(void) /* Order arrival event function. */
{
    /* Increment the inventory level by the amount ordered. */
    inv_level += amount;

    /* Since no order is now outstanding, eliminate the order-arrival
       event from consideration. */
    time_next_event[1] = 1.0e+30;
}

```

FIGURE 1.83

C code for function order_arrival, inventory model.

```

void demand(void) /* Demand event function. */
{
    int size_demand;

    /* Generate the demand size. */
    size_demand = random_integer(prob_distrib_demand);

    /* Decrement the inventory level by the demand size. */
    inv_level -= size_demand;

    /* Schedule the time of the next demand. */
    time_next_event[2] = time + expon(mean_interdemand);
}

```

FIGURE 1.84

C code for function demand, inventory model.

```

void evaluate(void) /* Inventory-evaluation event function. */
{
    /* Check whether the inventory level is less than smalls. */
    if (inv_level < smalls) {
        /* The inventory level is less than smalls, so place an order
           for the appropriate amount. */
        amount = bigs - inv_level;
        total_ordering_cost += setup_cost + incremental_cost * amount;

        /* Schedule the arrival of the order. */
        time_next_event[1] = time + uniform(minlag, maxlag);
    }

    /* Regardless of the place-order decision, schedule the next
       inventory evaluation. */
    time_next_event[4] = time + 1.0;
}

```

FIGURE 1.85

C code for function evaluate, inventory model.

```

void report(void) /* Report generator function. */
{
    /* Compute and write estimates of desired measures of performance.
       */
    float avg_holding_cost, avg_ordering_cost, avg_shortage_cost;

    avg_ordering_cost = total_ordering_cost / num_months;
    avg_holding_cost = holding_cost * area_holding / num_months;
    avg_shortage_cost = shortage_cost * area_shortage / num_months;
    fprintf(outfile, "\n\n(%3d,%3d)%15.2f%15.2f%15.2f%15.2f",
        smalls, bigs,
        avg_ordering_cost + avg_holding_cost + avg_shortage_cost,
        avg_ordering_cost, avg_holding_cost, avg_shortage_cost);
}

```

FIGURE 1.86

C code for function report, inventory model.

inventory level `inv_level` is zero, neither the “if” nor the “else if” condition is satisfied, resulting in no update at all, as desired. As in the single-server queueing model of Sec. 1.4, it might be necessary to make both the time and `time_last_event` variables be of type double to avoid severe roundoff error in their subtraction at the top of the routine if the simulation is to be run for a long period of simulated time.

The code for function `random_integer` is given in Fig. 1.88, and is general in that it will generate an integer according to distribution function `prob_distrib[I]`, provided that the values of `prob_distrib[I]` are specified. (In our case, $\text{prob_distrib}[1] = \frac{1}{6}$, $\text{prob_distrib}[2] = \frac{1}{2}$, $\text{prob_distrib}[3] = \frac{5}{6}$, and $\text{prob_distrib}[4] = 1$, all specified to three-decimal accuracy on input.) The logic

```

void update_time_avg_stats(void) /* Update area accumulators for
                                time-average statistics. */
{
    float time_since_last_event;

    /* Compute time since last event, and update last-event-time
       marker. */

    time_since_last_event = time - time_last_event;
    time_last_event       = time;

    /* Determine the status of the inventory level during the previous
       interval. If the inventory level during the previous interval
       was negative, update area_shortage. If it was positive, update
       area_holding. If it was zero, no update is needed. */

    if (inv_level < 0)
        area_shortage -= inv_level * time_since_last_event;
    else if (inv_level > 0)
        area_holding += inv_level * time_since_last_event;
}

```

FIGURE 1.87

C code for function update_time_avg_stats, inventory model.

```

int random_integer(float probab_distrib[])
/* Random integer generation function. */
{
    int i;
    float u;

    /* Generate a U(0,1) random variate. */

    u = rand(1);

    /* Return a random integer in accordance with the (cumulative)
       distribution function probab_distrib. */

    for (i = 1; u >= probab_distrib[i]; ++i)
        ;
    return i;
}

```

FIGURE 1.88

C code for function random_integer.

```

float uniform(float a, float b) /* Uniform variate generation function.
                                */
{
    float u;

    /* Generate a U(0,1) random variate. */

    u = rand(1);

    /* Return a U(a,b) random variate. */

    return a + u * (b - a);
}

```

FIGURE 1.89

C code for function uniform.

agrees with the discussion in Sec. 1.5.2; note that the input array prob_distrib must contain the *cumulative* distribution function rather than the probabilities that the variate takes on its possible values.

The function uniform is given in Fig. 1.89, and is as described in Sec. 1.5.2.

1.5.6 Simulation Output and Discussion

The simulation report (in file inv.out if either the FORTRAN or C version was used) is given in Fig. 1.90. For this model, there were some differences in the results across different languages, compilers, and computers, even though the same random-number-generator algorithm was being used; see App. 1C for details and an explanation of this discrepancy.

The three separate components of the average total cost per month were reported to see how they respond individually to changes in s and S , as a possible check on the model and the code. For example, fixing $s = 20$ and

Single-product inventory system

Initial inventory level	60 items			
Number of demand sizes	4			
Distribution function of demand sizes	0.167	0.500	0.833	1.000
Mean interdemand time	0.10 months			
Delivery lag range	0.50 to		1.00 months	
Length of the simulation	120 months			
K = 32.0	i = 3.0	h = 1.0	pi = 5.0	
Number of policies	9			
Policy	Average total cost	Average ordering cost	Average holding cost	Average shortage cost
(20, 40)	126.61	99.26	9.25	18.10
(20, 60)	122.74	90.52	17.39	14.83
(20, 80)	123.86	87.36	26.24	10.26
(20,100)	125.32	81.37	36.00	7.95
(40, 60)	126.37	98.43	25.99	1.95
(40, 80)	125.46	88.40	35.92	1.14
(40,100)	132.34	84.62	46.42	1.30
(60, 80)	150.02	105.69	44.02	0.31
(60,100)	143.20	89.05	53.91	0.24

FIGURE 1.90
Output report, inventory model.

increasing S from 40 to 100 increases the holding cost steadily from \$9.25 per month to \$36.00 per month, while reducing shortage cost at the same time; the effect of this increase in S on the ordering cost is to reduce it, evidently since ordering up to larger values of S implies that these larger orders will be placed less frequently, thereby avoiding the fixed ordering cost more often. Similarly, fixing S at, say, 100, and increasing s from 20 to 60 leads to a decrease in shortage cost (\$7.95, \$1.30, \$0.24) but an increase in holding cost (\$36.00, \$46.42, \$53.91), since increases in s translate into less willingness to let the inventory level fall to low values. While we could probably have predicted the *direction* of movement of these components of cost without doing the simulation, it would not have been possible to have said much about their *magnitude* without the aid of the simulation output.

Since the overall criterion of *total* cost per month is the sum of three components that move in sometimes different directions in reaction to changes in s and S , we cannot predict even the direction of movement of this criterion without the simulation. Thus, we simply look at the values of this criterion, and it would *appear* that the (20, 60) policy is the best, having an average total cost of \$122.74 per month. However, in the present context where the length of the simulation is fixed (the company wants a planning horizon of 10 years), what we *really* want to estimate for each policy is the *expected* average total cost per month for the first 120 months. The numbers in Fig. 1.90 are *estimates* of these expected values, each estimate based on a sample of size 1 (simulation run or replication). Since these estimates may have large variances, the ordering of them may differ considerably from the ordering of the expected values, which is the desired information. In fact, if we reran the nine simulations using different $U(0, 1)$ random variates, the estimates obtained might differ greatly from those in Fig. 1.90. Furthermore, the ordering of the new estimates might also be different.

We conclude from the above discussion that when the simulation run length is fixed by the problem context, it will generally not be sufficient to make a single simulation run of each policy or system of interest. In Chap. 9 we address the issue of just how many runs are required to get a good estimate of a desired expected value. Chapters 10 and 12 consider related problems when we are concerned with several different expected values arising from alternative system designs.

1.6 DISTRIBUTED SIMULATION

The simulations in Secs. 1.4 and 1.5 (as well as those to be considered in Chap. 2) all operate in basically the same way. A simulation clock and event list interact to determine which event will be processed next, the clock is advanced to the time of this event, and the computer executes the event logic, which may include updating state variables, manipulating lists for queues and events, generating random numbers and random variates, and collecting statistics. This

logic is executed in order of the events' simulated time of occurrence; i.e., the simulation is *sequential*. Furthermore, all work is done on a single computer.

In recent years computer technology has enabled individual computers or processors to be linked together into *parallel* or *distributed* computing environments. For example, several relatively inexpensive minicomputers (or even microcomputers) can be networked together, or a larger computer can house a number of individual processors that can work on their own as well as communicate with each other. In such an environment, it may be possible to "distribute" different parts of a computing task across individual processors operating at the same time, or in "parallel," and thus reduce the overall time to complete the task. The ability to accomplish this naturally depends on the nature of the computing task, as well as on the hardware and software available. Distributed and parallel processing is currently being investigated in many areas, such as optimization and database design; in this section we briefly discuss a few of the efforts to apply this idea to dynamic simulation. More detailed surveys, together with many references to the original sources, can be found in Chandrasekaran and Sheppard (1987) and Misra (1986).

There are many conceivable ways of splitting up a dynamic simulation to distribute its work over different processors. Perhaps the most direct approach is to allocate the distinct "support functions" (such as random-number generation, random-variate generation, event-list handling, manipulating lists and queues, and statistics collection) to different processors. The logical execution of the simulation is still sequential, as in the programs of Secs. 1.4 and 1.5, but now the "master" simulation program can delegate execution of the support functions to other processors and get on with its work. For example, when the queue list in the model of Sec. 1.4 needs updating in some way, the master simulation program sends a message to the processor handling this function concerning what is to be done, which will do it at the same time that the master simulation program goes on. Specific implementation of this idea was reported by Sheppard et al. (1985). Comfort (1984) considered in particular processing the event list in a "master-slave" arrangement of processors, since event-list processing can represent a major portion of the time to run a simulation (see Sec. 2.8).

A very different way to distribute a simulation across separate processors is to decompose the model itself into several submodels, which are then assigned to different processors for execution. For example, a manufacturing facility is often modeled as an interconnected network of queueing stations, each representing a different type of activity; see Sec. 2.7 for an example. The individual submodels (or groups of them) are assigned to different processors, each of which then goes to work simulating its piece of the model. The processors must communicate with each other whenever necessary to maintain the proper logical relationships between the submodels; in the manufacturing example, this could occur when a workpiece leaves one queueing station and goes to another station that is being simulated on a different processor. Care must be taken to maintain the correct time-ordering of actions, i.e., to *synchronize* the operation of the submodels on different processors so as to

represent the model's overall actions correctly. One major advantage of this type of distributed simulation is that there is neither a (global) simulation clock nor a (complete) event list; since event-list processing in traditional simulation modeling may take up a lot of the time to run the program (see Sec. 2.8), getting rid of the event list is an attractive idea. What takes the place of the clock and event list is a system for *message passing* between the processors, where each message carries with it a "time stamp." A drawback, however, is that it is possible for *deadlock* to occur in the simulation (two processors must each wait for a message from the other before they can proceed), even if such is not possible in the real system being simulated. This causes the simulation to grind to a halt, so there must be some method of detecting and breaking deadlocks (or perhaps of avoiding them). This method of distributed simulation was developed primarily by Chandy and Misra (1979, 1981, 1983), and was surveyed in Misra (1986).

Another concept, related to the preceding discussion of distributing submodels across parallel processors, is known as *virtual time*, implemented by the *time-warp mechanism*; see Jefferson (1985). As above, each processor simulates its own piece of the model forward in time, but does *not* wait to receive messages from other processors that may be moving along at different rates; this waiting is necessary in the above message-passing approach. If a submodel being simulated on a particular processor does receive a message that should have been received in its past (and thus potentially affecting its actions from that point in time on), a *rollback* occurs for the receiving submodel, whereby its time reverts to the (earlier) time of the incoming message. For example, if submodel B has been simulated up to time 87 and a message from submodel A comes in that was supposed to have been received at time 61, the clock for submodel B is rolled back to 61, and the simulation of submodel B between times 61 and 87 is canceled since it might have been done incorrectly without knowing the contents of the time 61 message. Part of this canceled work may have been sending messages to other submodels, each of which is nullified by sending a corresponding *antimessage*; the antimessages may themselves generate secondary rollbacks at their destination submodels, and so on. It seems unfortunate that the work done between times 61 and 87 is lost, and that we must incur the extra overhead associated with executing the rollback; however, all processors are busily simulating at all times (except during a rollback), rather than sitting idly waiting for messages before proceeding "correctly" through time in a forward way. In a stochastic simulation, whether a rollback will be necessary is uncertain, so the time-warp mechanism has been called a "game of chance," with the downside being the overhead in extra memory and processing for possible rollbacks but the upside being the possibility that rollbacks will be rare and that all processors will keep moving forward at all times. Furthermore, with the time-warp mechanism, deadlocks are avoided.

Development and evaluation of distributed processing in simulation is currently an active area of research. It does seem clear, however, that how well (or even whether) a particular method will work may depend on the model's

structure and parameters, as well as on the computing environment available. For example, if a model can be decomposed into submodels that are only weakly related (e.g., a network of queues in which customers only rarely move from one queue to another), then either of the model-based schemes discussed above for distributing the simulation may be expected to show more promise. For specific investigations into the efficacy of distributed simulation, see Lavenberg, Muntz, and Samadi (1983), Comfort (1984), and Heidelberger (1988); the last of these papers considers the impact of distributed simulation on statistical (rather than run-time) efficiency, with mixed results.

Martin Marietta Corporation used distributed simulation with an extremely complex simulation related to the Strategic Defense Initiative program. Seven "super" minicomputers were linked together by message passing in order to make the overall simulation model execute in real time, which was necessary for this man-in-the-loop application.

1.7 STEPS IN A SIMULATION STUDY

Now that we have looked in some detail at the inner workings of discrete-event simulations, we should step back and recognize that detailed modeling and coding are just part of an overall simulation effort to understand or design a complex system, and that attention must be paid to a variety of other concerns, ranging from statistical experimental design to budget and personnel management. Figure 1.91 shows the steps that will compose a typical, sound simulation study and the relationships among them [see also Banks and Carson (1984, p. 12), Law and McComas (1990) Shannon (1975, p. 23), and Gordon (1978, p. 52)]. The number beside the symbol representing each step refers to the more detailed discussion of that step below. Not all studies will necessarily contain all these steps and in the order stated; some studies may contain steps that do not fit neatly into the diagram. Moreover, a simulation study is not a simple sequential process. As one proceeds with a study and a better understanding of the system of interest is obtained, it is often desirable to go back to a previous step. For example, new insights about the system obtained during the study may necessitate reformulating the problem to be solved.

1. *Formulate problem and plan the study.* Every study must begin with a clear statement of the study's overall objectives and specific issues to be addressed; without such a statement there is little hope for success. The alternative system designs to be studied should be delineated (if possible), and criteria for evaluating the efficacy of these alternatives should be given. The overall study should be planned in terms of the number of people, the cost, and the time required for each aspect of the study.
2. *Collect data and define a model.* Information and data should be collected on the system of interest (if it exists) and used to specify operating procedures and probability distributions for the random variables used in the model (see Chap. 6). For example, in modeling a bank, one might

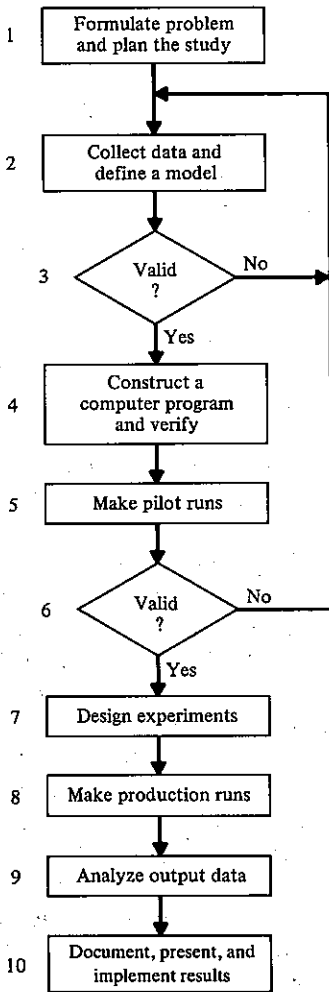


FIGURE 1.91
Steps in a simulation study.

collect interarrival times and service times and use these data to specify interarrival-time and service-time distributions for use in the model. If possible, data on the performance of the system, e.g., delays in queue of customers in a bank, should be collected for validation purposes in step 6. The construction of a mathematical and logical model of a real system for a given objective is still as much an art as it is a science. Although there are few firm rules on how one should go about the modeling process, one point on which most authors agree is that it is always a good idea to start with a model that is only moderately detailed, which can later be made more sophisticated if necessary. A model should contain only enough detail to capture the essence of the system for the purposes for which the model is intended; it is not necessary to have a one-to-one correspondence between

elements of the model and elements of the system. A model with excessive detail may be too expensive to program and to execute. A good discussion of the art of modeling can be found in Shannon (1975).

3. *Valid?* Although we believe that validation (see Chap. 5) is something that should be done throughout the entire simulation study (rather than after the model has been built and only if there is time and money still remaining), there are several points in the study where validation is particularly appropriate. One such point is during step 3. In building the model, it is imperative for the modelers to involve people in the study who are intimately familiar with the operations of the actual system. It is also advisable for the modelers to interact with the decision maker (or the model's intended user) on a regular basis. This will increase the actual validity of the model, and the credibility (or perceived validity) of the model to the decision maker will also be increased (see Sec. 5.5.1 for further discussion). In addition, the adequacy of the probability distributions specified for generating input random variates should be tested using goodness-of-fit tests (see Sec. 6.6).
4. *Construct a computer program and verify.* The simulation modeler must decide whether to program the model in a general-purpose language such as FORTRAN, Pascal, or C (Chaps. 1 and 2) or in a specially designed simulation language such as GPSS, SIMAN, SIMSCRIPT II.5, or SLAM II (Chap. 3). A general-purpose language will probably already be known and available on the modeler's computer. It may also lead to shorter execution times. On the other hand, by providing many of the features needed in programming a model, a simulation language may reduce the required programming time significantly. Chapter 8 discusses techniques for generating random variates on a computer with a specified probability distribution. This capability may be needed in programming a model, depending on the language used. Chapter 7 discusses the related topic of generating $U(0, 1)$ random variates (often called *random numbers*), which are the basis for generating all other types of random variates in Chap. 8. Techniques for verifying or debugging a computer program are discussed in Sec. 5.3.
5. *Make pilot runs.* Pilot runs of the verified model are made for validation purposes in step 6.
6. *Valid?* Pilot runs can be used to test the sensitivity of the model's output to small changes in an input parameter. If the output changes greatly, a better estimate of the input parameter must be obtained (see Sec. 5.5.2 for further discussion of this and other uses of sensitivity analyses). If a system similar to the one of interest currently exists, output data from pilot runs for a model of the *existing* system can be compared with those from the actual existing system (collected in step 2). If the agreement is "good," the "validated" model is modified so that it represents the system of interest; we would hope that this modification is not too extensive. (See Secs. 5.5 and 5.6 for further discussion of this idea.)

7. *Design experiments.* It must be decided what system designs to simulate if, as is sometimes the case in practice, there are more alternatives than one can reasonably simulate. Often the complete decision cannot be made at this time. Instead, using output data from the production runs (from step 8) of certain selected system designs and also techniques discussed in Chap. 12, the analyst can decide which additional systems to simulate. For each system design to be simulated, decisions have to be made on such issues as initial conditions for the simulation run(s), the length of the warmup period (if any), the length of the simulation run(s), and the number of independent simulation runs (replications) to make for each alternative. These issues are discussed in Chap. 9. When designing and making the production runs, it is sometimes possible to use certain *variance-reduction techniques* to give results with greater statistical precision (the variances of the estimators are decreased) at little or no additional cost. These techniques are discussed in Chap. 11. (A review of basic probability and statistics is given in Chap. 4.)
8. *Make production runs.* Production runs are made to provide performance data on the system designs of interest.
9. *Analyze output data.* Statistical techniques are used to analyze the output data from the production runs. Typical goals are to construct a confidence interval for a measure of performance for one particular system design (see Chap. 9) or to decide which simulated system is best relative to some specified measure of performance (see Chap. 10).
10. *Document, present, and implement results.* Because simulation models are often used for more than one application, it is important to document the assumptions that went into the model as well as the computer program itself. Finally, a simulation study whose results are never implemented is most likely a failure. Furthermore, results from highly credible models are much more likely to be used.

1.8 OTHER TYPES OF SIMULATION

Although the emphasis in this book is on discrete-event simulation, several other types of simulation are of considerable importance. Our goal here is to explain these other types of simulation briefly and to contrast them with discrete-event simulation. In particular, we shall discuss continuous, combined discrete-continuous, and Monte Carlo simulations.

1.8.1 Continuous Simulation

Continuous simulation concerns the modeling over time of a system by a representation in which the state variables change continuously with respect to time. Typically, continuous simulation models involve differential equations that give relationships for the rates of change of the state variables with time. If the differential equations are particularly simple, they can be solved analytical-

ly to give the values of the state variables for all values of time as a function of the values of the state variables at time 0. For most continuous models analytic solutions are not possible, however, and numerical-analysis techniques, e.g., Runge-Kutta integration, are used to integrate the differential equations numerically, given specific values for the state variables at time 0.

Several languages, such as ACSL and CSSL-IV [see Pratt (1987)], have been specifically designed for building continuous simulation models. In addition, the discrete-event simulation languages SIMAN [see Pegden (1989)], SIMSCRIPT II.5 [see Fayek (1988)], and SLAM II [see Pritsker (1986)] also have continuous modeling capabilities. These three languages have the added advantage of allowing both discrete and continuous components simultaneously in one model (see Sec. 1.8.2). Readers interested in applications of continuous simulation may wish to consult the journal *Simulation*.

Example 1.3. We now consider a continuous model of competition between two populations. Biological models of this type, which are called *predator-prey* (or *parasite-host*) models, have been considered by many authors, including Braun (1975, p. 583) and Gordon (1978, p. 103). An environment consists of two populations, predators and prey, which interact with each other. The prey are passive, but the predators depend on the prey as their source of food. [For example, the predators might be sharks and the prey might be food fish; see Braun (1975).] Let $x(t)$ and $y(t)$ denote, respectively, the numbers of individuals in the prey and predator populations at time t . Suppose that there is an ample supply of food for the prey and, in the absence of predators, that their rate of growth is $rx(t)$ for some positive r . (We can think of r as the natural birth rate minus the natural death rate.) Because of the interaction between predators and prey, it is reasonable to assume that the death rate of the prey due to interaction is proportional to the product of the two population sizes, $x(t)y(t)$. Therefore, the overall rate of change of the prey population, dx/dt , is given by

$$\frac{dx}{dt} = rx(t) - ax(t)y(t) \quad (1.8)$$

where a is a positive constant of proportionality. Since the predators depend on the prey for their very existence, the rate of change of the predators in the absence of prey is $-sy(t)$ for some positive s . Furthermore, the interaction between the two populations causes the predator population to increase at a rate that is also proportional to $x(t)y(t)$. Thus, the overall rate of change of the predator population, dy/dt , is

$$\frac{dy}{dt} = -sy(t) + bx(t)y(t) \quad (1.9)$$

where b is a positive constant. Given initial conditions $x(0) > 0$ and $y(0) > 0$, the solution of the model given by Eqs. (1.8) and (1.9) has the interesting property that $x(t) > 0$ and $y(t) > 0$ for all $t \geq 0$ [see Braun (1975)]. Thus, the prey population can never be completely extinguished by the predators. The solution $\{x(t), y(t)\}$ is also a periodic function of time. That is, there is a $T > 0$ such that $x(t + nT) = x(t)$ and $y(t + nT) = y(t)$ for all positive integers n . This result is not unexpected. As the predator population increases, the prey population decreases.

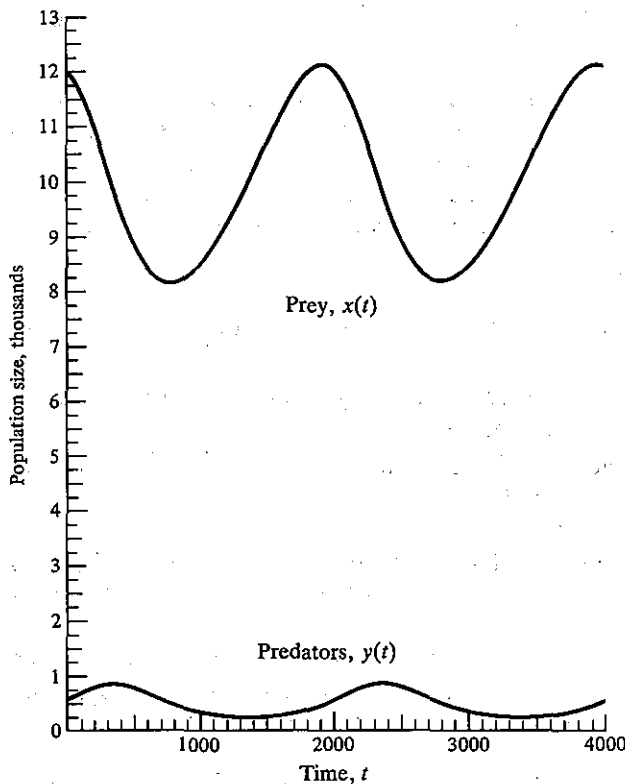


FIGURE 1.92
Numerical solution of a predator-prey model.

This causes a decrease in the rate of increase of the predators, which eventually results in a decrease in the number of predators. This in turn causes the number of prey to increase, etc.

Consider the particular values $r = 0.001$, $a = 2 \times 10^{-6}$, $s = 0.01$, $b = 10^{-6}$ and the initial population sizes $x(0) = 12,000$ and $y(0) = 600$. Figure 1.92 is a numerical solution of Eqs. (1.8) and (1.9) resulting from using a computer package designed to solve systems of differential equations numerically (not explicitly a continuous simulation language).

Note that the above example was completely deterministic; i.e., it contained no random components. It is possible, however, for a continuous simulation model to embody uncertainty; in Example 1.3 there could have been random terms added to Eqs. (1.8) and (1.9) that might depend on time in some way, or the constant factors could be modeled as quantities that change their value randomly at certain points in time.

1.8.2 Combined Discrete-Continuous Simulation

Since some systems are neither completely discrete nor completely continuous, the need may arise to construct a model with aspects of both discrete-event and continuous simulation, resulting in a *combined discrete-continuous* simulation. Pritsker (1986, pp. 61–62) describes the three fundamental types of interactions that can occur between discretely changing and continuously changing state variables:

- A discrete event may cause a discrete change in the value of a continuous state variable.
- A discrete event may cause the relationship governing a continuous state variable to change at a particular time.
- A continuous state variable achieving a threshold value may cause a discrete event to occur or to be scheduled.

Combined discrete-continuous simulation models can be built in SIMAN [Pegden (1989)], SIMSCRIPT II.5 [Fayek (1988)], and SLAM II [Pritsker (1986)].

The following example of a combined discrete-continuous simulation is a brief description of a model described in detail by Pritsker (1986, pp. 354–364), who also provides other examples of this type of simulation.

Example 1.4. Tankers carrying crude oil arrive at a single unloading dock, supplying a storage tank that in turn feeds a refinery through a pipeline. An unloading tanker delivers oil to the storage tank at a specified constant rate. (Tankers that arrive when the dock is busy form a queue.) The storage tank supplies oil to the refinery at a different specified rate. The dock is open from 6 A.M. to midnight, and, because of safety considerations, unloading of tankers ceases when the dock is closed.

The discrete events for this (simplified) model are the arrival of a tanker for unloading, closing the dock at midnight, and opening the dock at 6 A.M. The levels of oil in the unloading tanker and in the storage tank are given by continuous state variables whose rates of change are described by differential equations [see Pritsker (1986, pp. 354–364) for details]. Unloading the tanker is considered complete when the level of oil in the tanker is less than 5 percent of its capacity, but unloading must be temporarily stopped if the level of oil in the storage tank reaches its capacity. Unloading can be resumed when the level of oil in the tank decreases to 80 percent of its capacity. If the level of oil in the tank ever falls below 5000 barrels, the refinery must be shut down temporarily. In order to avoid frequent startups and shutdowns of the refinery, the tank does not resume supplying oil to the refinery until the tank once again contains 50,000 barrels. Each of the five events concerning the levels of oil, e.g., the level of oil in the tanker falling below 5 percent of the tanker's capacity, is what Pritsker calls a *state event*. Unlike discrete events, state events are not scheduled but occur when a continuous state variable crosses a threshold.

1.8.3 Monte Carlo Simulation

We define *Monte Carlo* simulation to be a scheme employing random numbers, that is, $U(0, 1)$ random variates, which is used for solving certain stochastic or deterministic problems where the passage of time plays no substantive role. Thus, Monte Carlo simulations are generally static rather than dynamic. The reader should note that although some authors define Monte Carlo simulation to be any simulation involving the use of random numbers, our definition is more restrictive. The name "Monte Carlo" simulation or method originated during World War II, when this approach was applied to problems related to the development of the atomic bomb. For a more detailed discussion of Monte Carlo simulation, see Hammersley and Handscomb (1964), Halton (1970), Rubinstein (1981), and Morgan (1984).

Example 1.5. Suppose that we want to evaluate the integral

$$I = \int_a^b g(x) dx$$

where $g(x)$ is a real-valued function that is not analytically integrable. (In practice, Monte Carlo simulation would probably not be used to evaluate a single integral, since there are more efficient numerical-analysis techniques for this purpose. It is more likely to be used on a multiple-integral problem with an ill-behaved integrand.) To see how this *deterministic* problem can be approached by Monte Carlo simulation, let Y be the random variable $(b - a)g(X)$, where X is a continuous random variable distributed uniformly on $[a, b]$ [denoted by $U(a, b)$]. Then the expected value of Y is

$$\begin{aligned} E(Y) &= E[(b - a)g(X)] \\ &= (b - a)E[g(X)] \\ &= (b - a) \int_a^b g(x)f_X(x) dx \\ &= (b - a) \frac{\int_a^b g(x) dx}{(b - a)} \\ &= I \end{aligned}$$

where $f_X(x) = 1/(b - a)$ is the probability density function of a $U(a, b)$ random variable (see Sec. 6.2.2). [For justification of the third equality, see, for example, Ross (1989, p. 43).] Thus, the problem of evaluating the integral has been reduced to one of estimating the expected value $E(Y)$. In particular, we shall estimate $E(Y) = I$ by the sample mean

$$\bar{Y}(n) = \frac{\sum_{i=1}^n Y_i}{n} = (b - a) \frac{\sum_{i=1}^n g(X_i)}{n}$$

where $X_1, X_2, \dots, X_n$ are IID $U(a, b)$ random variables. (It is instructive to think of $\bar{Y}(n)$ as an estimate of the area of the rectangle that has a base of length

TABLE 1.3

$\bar{Y}(n)$ for various values of n resulting from applying Monte Carlo simulation to the estimation of the integral $\int_0^\pi \sin x \, dx = 2$

n	10	20	40	80	160
$\bar{Y}(n)$	2.213	1.951	1.948	1.989	1.993

$(b-a)$ and a height $I/(b-a)$, which is the continuous average of $g(x)$ over $[a, b]$. Furthermore, it can be shown that $E[\bar{Y}(n)] = I$, that is, $\bar{Y}(n)$ is an unbiased estimator of I , and $\text{Var}[\bar{Y}(n)] = \text{Var}(Y)/n$ (see Sec. 4.4). Assuming that $\text{Var}(Y)$ is finite, it follows that $\bar{Y}(n)$ will be arbitrarily close to I for sufficiently large n (with probability 1) (see Sec. 4.6).

To illustrate the above scheme numerically, suppose that we would like to evaluate the integral

$$I = \int_0^\pi \sin x \, dx$$

which can be shown by elementary calculus to have a value of 2. Table 1.3 shows the results of applying Monte Carlo simulation to the estimation of this integral for various values of n .

Monte Carlo simulation is now widely used to solve certain problems in statistics that are not analytically tractable. For example, it has been applied to estimate the critical values or the power of a new hypothesis test. Determining the critical values for the Kolmogorov-Smirnov test for normality, discussed in Sec. 6.6, is such an application. The advanced reader might also enjoy perusing the technical journals *Communications in Statistics* (Part B, Simulation and Computation), *Journal of Statistical Computation and Simulation*, and *Technometrics*, all of which contain many examples of this type of Monte Carlo simulation.

Finally, it should be mentioned that the procedures discussed in Sec. 9.4 can be used to determine the sample size, n , required to obtain a specified precision in a Monte Carlo simulation study.

1.9 ADVANTAGES, DISADVANTAGES, AND PITFALLS OF SIMULATION

We conclude this introductory chapter by listing some good and bad characteristics of simulation (as opposed to other methods of studying systems), and by noting some common mistakes made in simulation studies that can impair or even ruin a simulation project. This subject was also discussed to some extent in Sec. 1.2, but now that we have worked through some simulation examples, it may be possible to be more specific.

As mentioned in Sec. 1.2, simulation is a widely used and increasingly popular method for studying complex systems. Some possible advantages of simulation that may account for its widespread appeal are the following.

- Most complex, real-world systems with stochastic elements cannot be accurately described by a mathematical model that can be evaluated *analytically*. Thus, a simulation is often the only type of investigation possible.
- Simulation allows one to estimate the performance of an existing system under some projected set of operating conditions.
- Alternative proposed system designs (or alternative operating policies for a single system) can be compared via simulation to see which best meets a specified requirement.
- In a simulation we can maintain much better control over experimental conditions than would generally be possible when experimenting with the system itself (see Chap. 11).
- Simulation allows us to study a system with a long time frame—e.g., an economic system—in compressed time, or alternatively to study the detailed workings of a system in expanded time.

Simulation is not without its drawbacks. Some disadvantages are as follows.

- Each run of a *stochastic* simulation model produces only *estimates* of a model's true characteristics for a particular set of input parameters. Thus, several independent runs of the model will probably be required for each set of input parameters to be studied (see Chap. 9). For this reason, simulation models are generally not as good at optimization as they are at comparing a fixed number of specified alternative system designs. On the other hand, an analytic model, *if appropriate*, can often easily produce the *exact* true characteristics of that model for a variety of sets of input parameters. Thus, if a "valid" analytic model is available or can easily be developed, it will generally be preferable to a simulation model.
- Simulation models are often expensive and time-consuming to develop.
- The large volume of numbers produced by a simulation study or the persuasive impact of a realistic animation (see Sec. 3.4.2) often creates a tendency to place greater confidence in a study's results than is justified. If a model is not a "valid" representation of a system under study, the simulation results, no matter how impressive they appear, will provide little useful information about the actual system.

When deciding whether or not a simulation study is appropriate in a given situation, we can only advise that these advantages and drawbacks be kept in mind and that all other relevant facets of one's particular situation be brought to bear as well. Finally, it should be noted that in some studies both simulation and analytic models might be useful. In particular, simulation can be used to check the validity of assumptions needed in an analytic model. On the other hand, an analytic model can suggest reasonable alternatives to investigate in a simulation study.

Assuming that the decision has been prudently made to use the simulation tool, we have found that there are several pitfalls along the way to

successful completion of a simulation study [see also Solomon (1983, p. 10) and Law and McComas (1989)]:

- Failure to have a well-defined set of objectives at the beginning of the simulation study
- Inappropriate level of model detail
- Failure to communicate with management on a regular basis throughout the course of the simulation study
- Treating a simulation study as if it were primarily a complicated exercise in computer programming
- Failure to have people with operations-research and statistical training on the modeling team
- Obviously using commercial simulation software that may contain errors or whose complex macro statements may not be well documented and may not implement the modeling logic desired
- Reliance on simulators that make simulation accessible to “anyone” (see Sec. 3.3.1)
- Misuse of animation
- Failure to account correctly for sources of randomness in the actual system
- Using arbitrary distributions (e.g., normal or uniform) as input to the simulation
- Analyzing the output data from one simulation run using statistical formulas that assume independence
- Making a single replication of a particular system design and treating the output statistics as the “true answers”
- Comparing alternative system designs on the basis of one replication for each design
- Using wrong measures of performance

We will have more to say about what *to* do (rather than what *not* to do) concerning some of the above potential stumbling blocks in the later chapters of this book.

APPENDIX 1A

FIXED-INCREMENT TIME ADVANCE

As mentioned in Sec. 1.3.1, the second principal approach for advancing the simulation clock in a discrete-event simulation model is called *fixed-increment time advance*. With this approach, the simulation clock is advanced in incre-

ments of exactly Δt time units for some appropriate choice of Δt . After each update of the clock, a check is made to determine if any events should have occurred during the previous interval of length Δt . If one or more events were scheduled to have occurred during this interval, these events are considered to occur at the *end* of the interval and the system state (and statistical counters) are updated accordingly. The fixed-increment time-advance approach is illustrated in Fig. 1.93, where the curved arrows represent the advancing of the simulation clock and e_i ($i = 1, 2, \dots$) is the *actual* time of occurrence of the i th event of any type (*not* the i th value of the simulation clock). In the time interval $[0, \Delta t)$, an event occurs at time e_1 but is considered to occur at time Δt by the model. No events occur in the interval $[\Delta t, 2\Delta t)$, but the model checks to determine that this is the case. Events occur at the times e_2 and e_3 in the interval $[2\Delta t, 3\Delta t)$, but both events are considered to occur at time $3\Delta t$, etc. A set of rules must be built into the model to decide in what order to process events when two or more events are considered to occur at the same time by the model. Two disadvantages of fixed-increment time advance are the errors introduced by processing events at the end of the interval in which they occur and the necessity of deciding which event to process first when events that are not simultaneous in reality are treated as such by the model. These problems can be made less severe by making Δt smaller, but this increases the amount of checking for event occurrences that must be done and results in an increase in execution time. Because of these considerations, fixed-increment time advance is generally not used for discrete-event simulation models when the times between successive events can vary greatly.

The primary use of this approach appears to be for systems where it can reasonably be assumed that all events *actually* occur at one of the times $n\Delta t$ ($n = 0, 1, 2, \dots$) for an appropriately chosen Δt . For example, data in economic systems are often available only on an annual basis, and it is natural in a simulation model to advance the simulation clock in increments of 1 year. [See Naylor (1971) for a discussion of simulation of economic systems. See also Sec. 4.3 for discussion of an inventory system that can be simulated, without loss of accuracy, by fixed-increment time advance.]

It should be noted that fixed-increment time advance can be realized when using the next-event time-advance approach by artificially scheduling "events" to occur every Δt time units.

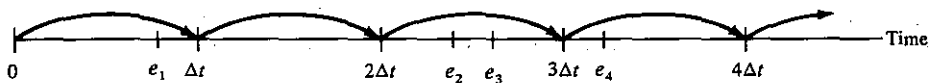


FIGURE 1.93
An illustration of fixed-increment time advance.

APPENDIX 1B

A PRIMER ON QUEUEING SYSTEMS

A *queueing system* consists of one or more servers that provide service of some kind to arriving customers. Customers who arrive to find all servers busy (generally) join one or more *queues* (or lines) in front of the servers, hence the name “queueing” system.

Historically, a large proportion of all discrete-event simulation studies have involved the modeling of a real-world queueing system, or at least some component of the system being simulated was a queueing system. Thus, we believe that it is important for the student of simulation to have at least a basic understanding of the components of a queueing system, standard notation for queueing systems, and measures of performance that are often used to indicate the quality of service being provided by a queueing system. Some examples of real-world queueing systems that have often been simulated are given in Table 1.4. For additional information on queueing systems in general, see Gross and Harris (1985) and Kleinrock (1975). Chandy and Sauer (1981) and Kleinrock (1976) are recommended for those interested in queueing models of computer systems.

1B.1 Components of a Queueing System

A queueing system is characterized by three components: arrival process, service mechanism, and queue discipline. Specifying the *arrival process* for a queueing system consists of describing how customers arrive to the system. Let A_i be the interarrival time between the arrivals of the $(i-1)$ st and i th customers (see Sec. 1.3). If $A_1, A_2, \dots$ are assumed to be IID random variables, we shall denote the *mean* (or expected) *interarrival time* by $E(A)$ and call $\lambda = 1/E(A)$ the *arrival rate* of customers.

The *service mechanism* for a queueing system is articulated by specifying the number of servers (denoted by s), whether each server has its own queue

TABLE 1.4
Examples of queueing systems

System	Servers	Customers
Bank	Tellers	Customers
Hospital	Doctors, nurses, beds	Patients
Computer system	Central processing unit, input/output devices	Jobs
Manufacturing line	Workers, machines	Items being manufactured
Airport	Runways, gates, security check-in stations	Airplanes, travelers
Communication system	Lines, circuits, operators	Calls, callers, messages

or there is one queue feeding all servers, and the probability distribution of customers' service times. Let S_i be the service time of the i th arriving customer. If $S_1, S_2, \dots$ are IID random variables, we shall denote the *mean service time* of a customer by $E(S)$ and call $\omega = 1/E(S)$ the *service rate* of a server.

The *queue discipline* of a queueing system refers to the rule that a server uses to choose the next customer from the queue (if any) when the server completes the service of the current customer. Commonly used queue disciplines include:

FIFO: Customers are served in a first-in, first-out manner.

LIFO: Customers are served in a last-in, first-out manner (see Prob. 2.17).

Priority: Customers are served in order of their importance (see Prob. 2.22) or on the basis of their service requirements (see Probs. 1.24, 2.20, and 2.21).

1B.2 Notation for Queueing Systems

Certain queueing systems occur so often in practice that standard notations have been developed for them. In particular, consider the queueing system shown in Fig. 1.94, which has the following characteristics:

1. s servers in parallel and one FIFO queue feeding all servers.
2. $A_1, A_2, \dots$ are IID random variables.
3. $S_1, S_2, \dots$ are IID random variables.
4. The A_i 's and S_i 's are independent.

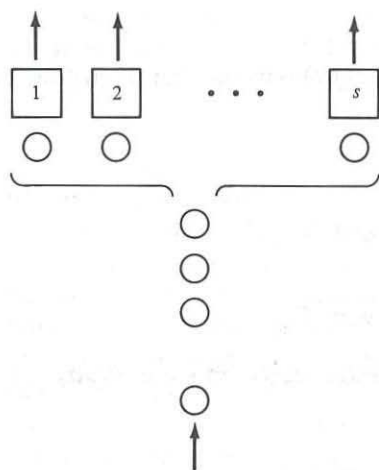


FIGURE 1.94
A $GI/G/s$ queue.

We call such a system a $GI/G/s$ queue, where GI (general independent) refers to the distribution of the A_i 's and G (general) refers to the distribution of the S_i 's. If specific distributions are given for the A_i 's and the S_i 's (as is always the case for simulation), symbols denoting these distributions are used in place of GI and G . The symbol M is used for the exponential distribution because of the Markovian, i.e., memoryless, property of the exponential distribution (see Prob. 4.26), the symbol E_k for a k -Erlang distribution (if X is a k -Erlang random variable, then $X = \sum_{i=1}^k Y_i$, where the Y_i 's are IID exponential random variables), and D for deterministic (or constant) times. Thus, a single-server queueing system with exponential interarrival times and service times and a FIFO queue discipline is called an $M/M/1$ queue.

For any $GI/G/s$ queue, we shall call the quantity $\rho = \lambda/(s\omega)$ the *utilization factor* of the queueing system ($s\omega$ is the service rate of the system when all servers are busy). It is a measure of how heavily the resources of a queueing system are utilized.

1B.3 Measures of Performance for Queueing Systems

There are many possible measures of performance for queueing systems. We now describe four such measures that are usually used in the mathematical study of queueing systems. The reader should not infer from our choices that these measures are necessarily the most relevant or important in practice (see Chap. 9 for further discussion). As a matter of fact, for some real-world systems these measures may not even be well defined; i.e., they may not exist.

Let

D_i = delay in queue of i th customer

$W_i = D_i + S_i$ = waiting time in system of i th customer

$Q(t)$ = number of customers in queue at time t

$L(t)$ = number of customers in system at time t [$Q(t)$ plus number of customers being served at time t]

Then the measures

$$d = \lim_{n \rightarrow \infty} \frac{\sum_{i=1}^n D_i}{n} \quad \text{w.p. 1}$$

$$\text{and} \quad w = \lim_{n \rightarrow \infty} \frac{\sum_{i=1}^n W_i}{n} \quad \text{w.p. 1}$$

(if they exist) are called the *steady-state average delay* and the *steady-state average waiting time*. Similarly, the measures

$$Q = \lim_{T \rightarrow \infty} \frac{\int_0^T Q(t) dt}{T} \quad \text{w.p. 1}$$

$$\text{and} \quad L = \lim_{T \rightarrow \infty} \frac{\int_0^T L(t) dt}{T} \quad \text{w.p. 1}$$

(if they exist) are called the *steady-state time-average number in queue* and the *steady-state time-average number in system*. Here and throughout this book, the qualifier “w.p. 1” (with probability 1) is given for mathematical correctness and has little practical significance. For example, suppose that $\sum_{i=1}^n D_i/n \rightarrow d$ as $n \rightarrow \infty$ (w.p. 1) for some queueing system. This means that if one performs a very large (an infinite) number of experiments, then in virtually every experiment $\sum_{i=1}^n D_i/n$ converges to the finite quantity d . Note that $\rho < 1$ is a necessary condition for d , w , Q , and L to exist for a $GI/G/s$ queue.

Among the most general and useful results for queueing systems are the *conservation equations*

$$Q = \lambda d \quad \text{and} \quad L = \lambda w$$

These equations hold for every queueing system for which d and w exist [see Stidham (1974)]. (Section 11.5 gives a simulation application of these relationships.) Another equation of considerable practical value is given by

$$w = d + E(S)$$

(see Sec. 1.4.7 and also Sec. 11.5 for further discussion).

It should be mentioned that the measures of performance discussed above can be analytically computed for $M/M/s$ queues ($s \geq 1$), $M/G/1$ queues for any distribution G , and for certain other queueing systems. In general, the interarrival-time distribution, the service-time distribution, or both must be exponential (or a variant of exponential, such as k -Erlang) for analytic solutions to be possible [see Gross and Harris (1985) or Kleinrock (1975, 1976)].

One interesting (and instructive) example of such an analytical solution is the steady-state average delay in queue for an $M/G/1$ queue, given by

$$d = \frac{\lambda \{ \text{Var}(S) + [E(S)]^2 \}}{2[1 - \lambda E(S)]}$$

where $\text{Var}(S)$ denotes the variance of the service-time distribution [see, for example, Ross (1989, p. 376) for a derivation of this formula]. Thus, we can see that if $E(S)$ is large, then congestion (here measured by d) will be larger; this is certainly to be expected. The formula also brings out the perhaps less obvious fact that congestion also increases if the *variability* of the service-time distribution is large, even if the mean service time stays the same; intuitively, this is because a highly variable service-time random variable will have a greater chance of taking on a large value (since it must be positive), which means that the (single) server will be tied up for a long time, causing the queue to build up.

APPENDIX 1C

NOTES ON THE COMPUTERS AND COMPILERS USED

The example simulation programs written in general-purpose languages in this and the next chapter have been run on several different computer systems and compilers, in an attempt to make them as general and portable as possible. There may still be some machine or compiler dependence, however, for example in input/output conventions. We have tried to obey standards for versions of the languages, where they exist.

All the FORTRAN programs shown in this book are in ANSI-Standard FORTRAN 77, with the exception of the INCLUDE statements found in the code and used to bring an external file into the source code at the point of their appearance. Moreover, these programs have all run in the following environments:

- IBM PC with IBM Professional FORTRAN (Version 1.00)
- IBM PS/2 Model 50Z with Microsoft FORTRAN (Version 4.01)
- Apple Macintosh SE with Absoft FORTRAN (Version 2.4)
- VAX 8650 running the VMS 5.2 operating system with VAX FORTRAN (Version 5.0)
- Encore Multimax 320 running the UMAX 4.3 operating system (a version of UNIX) with UMAX FORTRAN (f77)
- Cray-2 running the UNICOS 5.0.7 operating system (a version of UNIX System V) with Cray FORTRAN 77 (cft77, release 3.0)

(The Absoft FORTRAN compiler on the Macintosh SE has a problem with ENTRY points, necessitating changes in RAND and references to it, as detailed in App. 7A.) For the IBM PC Professional FORTRAN, VAX FORTRAN, UMAX FORTRAN, and on the Cray-2, no changes at all are necessary from the code shown in this book, i.e., the INCLUDE statements work as shown. For the IBM PS/2 with Microsoft FORTRAN, the form of the INCLUDE statements must be changed to

`$INCLUDE:'filename'`

where the filename is mm1.dcl or mm1alt.dcl, etc., and the \$ is in position 1. For Absoft FORTRAN on the Macintosh SE, the INCLUDE statements are the same as in the text except that the single quotes around the file name to be included must be removed.

The Pascal programs in this chapter have been run in the following environments:

- VAX 8650 running the VMS 5.2 operating system with VAX Pascal (Version 3.5)

- Cray-2 running the UNICOS 5.0.7 operating system (a version of UNIX System V) with Cray Pascal (4.0)

We did not run the Pascal programs on any microcomputers, since we did not have compilers available that support 32-bit integers, a requirement for the random-number generator of Fig. 7.6.

The C programs in this chapter have been run in the following environments:

- IBM PS/2 Model 50Z with Borland Turbo C (Version 1.5)
- Apple Macintosh IIcx with THINK C 4.0 (the names of the built-in functions `rand` and `time` in the ANSI library had to be changed to avoid conflicts with our use of these names)
- VAX 8650 running the VMS 5.2 operating system with VAX C (Version 2.4)
- Cray-2 running the UNICOS 5.0.7 operating system (a version of UNIX System V) with the Cray Standard C compiler (`scc`, release 1.0)

As noted in Sec. 1.4.6, we have used the ANSI-standard version of C, the most important feature of which is complete function prototyping. This prototyping could be removed from our programs to be run on compilers that do not support it.

The results did differ in some cases for a given model run in different languages, with different compilers, or on different machines, due to inaccuracies in floating-point operations. This can matter if, for example, at some point two events are scheduled to be very close together, and roundoff error could result in different sequencing. In particular, representing the simulation clock as a floating-point number, as we have done, can lead to variable results in many ways. In the inventory simulation of Sec. 1.5, for instance, there are actually nine separate simulation runs made, and a particular demand event [whose interdemand time was generated by a particular $U(0, 1)$ random number] occurred near the end of run 2 on one machine, but at the beginning of run 3 on another machine due to different floating-point roundoff errors in the simulation clock; from this point on, the results differed. The numerical output shown in all cases (in this and the following chapter) was produced on an IBM PC with IBM Professional FORTRAN.

PROBLEMS

- 1.1. Describe what you think would be the most effective way to study each of the following systems, in terms of the possibilities in Fig. 1.1, and discuss why:
 - (a) A small section of an existing factory
 - (b) A freeway interchange that has experienced severe congestion
 - (c) An emergency room in an existing hospital
 - (d) A pizza-delivery operation
 - (e) The shuttle-bus operation for a rental-car agency at an airport
 - (f) A battlefield-communication system

- 1.2. For each of the systems in Prob. 1.1, suppose that it has been decided to make a study via a simulation model. Discuss whether the simulation should be static or dynamic, deterministic or stochastic, and continuous or discrete.
- 1.3. For the single-server queueing system in Sec. 1.4, define $L(t)$ to be the *total* number of customers in the system at time t (including the queue and the customer in service at time t , if any).
 - (a) Is it true that $L(t) = Q(t) + 1$? Why or why not?
 - (b) For the same realization considered for the hand simulation in Sec. 1.4.2, make a plot of $L(t)$ vs. t (similar to Figs. 1.5 and 1.6) between times 0 and $T(6)$.
 - (c) From your plot in part (b), compute $\bar{L}(6)$ = the time-average number of customers in the system during the time interval $[0, T(6)]$. What is $\bar{L}(6)$ estimating?
 - (d) Augment Fig. 1.7 to indicate how $\bar{L}(6)$ is computed during the course of the simulation.
- 1.4. For the single-server queue of Sec. 1.4, suppose that we did not want to estimate the expected average delay in queue; the model's structure and parameters remain the same. Does this change the state variables? If so, how?
- 1.5. For the single-server queue of Sec. 1.4, let W_i = the *total* time in the system of the i th customer to finish service, which includes the time in queue plus the time in service of this customer. For the same realization considered for the hand simulation in Sec. 1.4.2, compute $\bar{W}(m)$ = the average time in system of the first m customers to exit the system, for $m = 5$; do this by augmenting Fig. 1.7 appropriately. How does this change the state variables, if at all?
- 1.6. From Fig. 1.5, it is clear that the maximum length of the queue was 3. Write a general expression for this quantity (for the n -delay stopping rule) and augment Fig. 1.7 so that it can be computed systematically during the simulation.
- 1.7. Modify the code for the single-server queue in Sec. 1.4.4, 1.4.5, or 1.4.6 to compute and write in addition the following measures of performance:
 - (a) The time-average number in the system (see Prob. 1.3)
 - (b) The average total time in the system (see Prob. 1.5)
 - (c) The maximum queue length (see Prob. 1.6)
 - (d) The maximum delay in queue
 - (e) The maximum time in the system
 - (f) The proportion of customers having a delay in queue in excess of 1 minute.
 Run this program using the random-number generator given in App. 7A.
- 1.8. The algorithm in Sec. 1.4.3 for generating an exponential random variate with mean β was to return $-\beta \ln U$, where U is a $U(0, 1)$ random variate. This algorithm could validly be changed to return $-\beta \ln(1 - U)$. Why?
- 1.9. Run the single-server queueing simulation of Sec. 1.4.4, 1.4.5, or 1.4.6 ten times by placing a loop around most of the main program, beginning just before the initialization and ending just after invoking the report generator. Discuss the results. (This is called *replicating* the simulation ten times independently.)
- 1.10. For the single-server queueing simulation of Sec. 1.4, suppose that the facility opens its doors at 9 A.M. (call this time 0) and closes its doors at 5 P.M., but operates until all customers present (in service or in queue) at 5 P.M. have been served. Change the code to reflect this stopping rule, and estimate the same performance measures as before.

- 1.11. For the single-server queueing system of Sec. 1.4, suppose that there is room in the queue for only two customers, and that a customer arriving to find that the queue is full just goes away (this is called *balking*). Simulate this system for a stopping rule of exactly 480 minutes, and estimate the same quantities as in Sec. 1.4, as well as the expected number of customers who balk.
- 1.12. Consider the inventory simulation of Sec. 1.5.
- For this model with these parameters, there can never be more than one order outstanding (i.e., previously ordered but not yet delivered) at a time. Why?
 - Describe specifically what changes would have to be made if the delivery lag were uniformly distributed between 0.5 and 6.0 months (rather than between 0.5 and 1.0 month); no other changes to the model are being considered. Should ordering decisions be based only on the inventory level $I(t)$?
- 1.13. Modify the inventory simulation of Sec. 1.5 so that it makes five replications of each (s, S) policy; see Prob. 1.9. Discuss the results. Which inventory policy is best? Are you sure?
- 1.14. A service facility consists of two servers in series (tandem), each with its own FIFO queue (see Fig. 1.95). A customer completing service at server 1 proceeds to server 2, while a customer completing service at server 2 leaves the facility. Assume that the interarrival times of customers to server 1 are IID exponential random variables with mean 1 minute. Service times of customers at server 1 are IID exponential random variables with mean 0.7 minute, and at server 2 are IID exponential random variables with mean 0.9 minute. Run the simulation for exactly 1000 minutes and estimate for each server the expected average delay in queue of a customer, the expected time-average number of customers in queue, and the expected utilization.
- 1.15. In Prob. 1.14, suppose that there is a travel time from the exit from server 1 to the arrival to queue 2 (or to server 2). Assume that this travel time is distributed uniformly between 0 and 2 minutes. Modify the simulation and rerun it under the same conditions to obtain the same performance measures. What is the required dimension (i.e., length) of the event list?
- 1.16. In Prob. 1.14, suppose that no queueing is allowed for server 2. That is, if a customer completing service at server 1 sees that server 2 is idle, she proceeds directly to server 2, as before. However, a customer completing service at server 1 when server 2 is busy with another customer must stay at server 1 until server 2 gets done; this is called *blocking*. While a customer is blocked from entering server 2, she receives no additional service from server 1, but prevents server 1 from taking the first customer, if any, from queue 1. Furthermore, "fresh" customers continue to arrive to queue 1 during a period of blocking. Compute the same six performance measures as in Prob. 1.14.
- 1.17. For the inventory system of Sec. 1.5, suppose that if the inventory level $I(t)$ at the beginning of a month is less than zero, the company places an *express order* to its

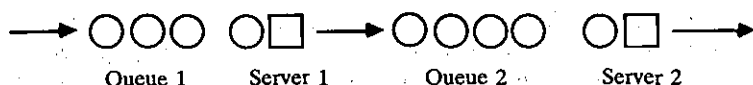


FIGURE 1.95

A tandem queueing system.

- supplier. (If $0 \leq I(t) < s$, the company still places a normal order.) An express order for Z items costs the company $48 + 4Z$ dollars, but the delivery lag is now uniformly distributed on $[0.25, 0.50]$ month. Run the simulation for all nine policies and estimate the expected average total cost per month, the expected proportion of time that there is a backlog, that is, $I(t) < 0$, and the expected number of express orders placed. Is express ordering worth it?
- 1.18. For the inventory simulation of Sec. 1.5, suppose that the inventory is *perishable*, having a shelf life distributed uniformly between 1.5 and 2.5 months. That is, if an item has a shelf life of ℓ months, then ℓ months after it is placed in inventory it spoils and is of no value to the company. (Note that different items in an order from the supplier will have different shelf lives.) The company discovers that an item is spoiled only upon examination before a sale. If an item is determined to be spoiled, it is discarded and the next item in the inventory is examined. Assume that items in the inventory are processed in a FIFO manner. Repeat the nine simulation runs and observe the same costs as before. Also compute the proportion of items taken out of the inventory that are discarded due to being spoiled.
- 1.19. Consider a service facility with s (where $s \geq 1$) parallel servers. Assume that interarrival times of customers are IID exponential random variables with mean $E(A)$ and that service times of customers (regardless of the server) are IID exponential random variables with mean $E(S)$. If a customer arrives and finds an idle server, the customer begins service immediately, choosing the leftmost (lowest-numbered) idle server if there are several available. Otherwise, the customer joins the tail of a *single* FIFO queue that supplies customers to all the servers. (This is called an $M/M/s$ queue; see App. 1B.) Write a general program to simulate this system that will estimate the expected average delay in queue, the expected time-average number in queue, and the expected utilization of each of the servers, based on a stopping rule of n delays having been completed. The quantities s , $E(A)$, $E(S)$, and n should be input parameters. Run the model for $s = 5$, $E(A) = 1$, $E(S) = 4$, and $n = 1000$.
- 1.20. Repeat Prob. 1.19, but now assume that an arriving customer finding more than one idle server chooses among them with equal probability. For example, if $s = 5$ and a customer arrives to find servers 1, 3, 4, and 5 idle, he chooses each of these servers with probability 0.25.
- 1.21. Customers arrive to a bank consisting of three tellers in parallel.
- If there is a single FIFO queue feeding all tellers, what is the required dimension (i.e., length) of the event list for a simulation model of this system?
 - If each teller has his own FIFO queue and if a customer can *jockey* (i.e., jump) from one queue to another (see Sec. 2.6 for the jockeying rules), what is the required dimension of the event list? Assume that jockeying takes no time.
 - Repeat part (b) if jockeying takes 3 seconds.
- Assume in all three parts that no events are required to terminate the simulation.
- 1.22. A manufacturing system contains m machines, each subject to randomly occurring breakdowns. A machine runs for an amount of time that is an exponential random variable with mean 8 hours before breaking down. There are s (where s is a fixed, positive integer) repairmen to fix broken machines, and it takes one repairman an exponential amount of time with mean 2 hours to complete the repair of one machine; no more than one repairman can be assigned to work on a

broken machine even if there are other idle repairmen. If more than s machines are broken down at a given time, they form a FIFO "repair" queue and wait for the first available repairman. Further, a repairman works on a broken machine until it is fixed, regardless of what else is happening in the system. Assume that it costs the system \$50 for each hour that each machine is broken down and \$10 an hour to employ each repairman. (The repairmen are paid an hourly wage regardless of whether they are actually working.) Assume that $m = 5$, but write general code to accommodate a value of m as high as 20 by changing an input parameter. Simulate the system for exactly 800 hours for each of the employment policies $s = 1, 2, \dots, 5$ to determine which policy results in the smallest expected average cost per hour. Assume that at time 0 all machines have just been "freshly" repaired.

- 1.23. For the facility of Prob. 1.10, suppose that the server normally takes a 30-minute lunch break at the first time after 12 noon that the facility is empty. If, however, the server has not gone to lunch by 1 P.M., the server will go after completing the customer in service at 1 P.M. (Assume in this case that all customers in the queue at 1 P.M. will wait until the server returns.) If a customer arrives while the server is at lunch, the customer *may* leave immediately without being served; this is called *balking*. Assume that whether such a customer balks depends on the amount of time remaining before the server's return. (The server posts his time of return from lunch.) In particular, a customer who arrives during lunch will balk with the following probabilities:

Time remaining before server's return (minutes)	Probability of a customer's balking
[20, 30)	0.75
[10, 20)	0.50
[0, 10)	0.25

(The random-integer-generation method discussed in Sec. 1.5.2 can be used to determine whether a customer balks. For a simpler approach, see Sec. 8.4.1.) Run the simulation and estimate the same measures of performance as before. (Note that the server is not busy when at lunch and that the time-average number in queue is computed including data from the lunch break.) In addition, estimate the expected number of customers who balk.

- 1.24. For the single-server queueing facility of Sec. 1.4, suppose that a customer's service time is known at the instant of arrival. Upon completing service to a customer, the server chooses from the queue (if any) the customer with the smallest service time. Run the simulation until 1000 customers have completed their delays and estimate the expected average delay in queue, the expected time-average number in queue, and the expected proportion of customers whose delay in queue is greater than 1 minute. (This priority queue discipline is called *shortest job first*.)
- 1.25. For the tandem queue of Prob. 1.14, suppose that with probability 0.2, a customer completing service at server 2 is *dissatisfied* with her overall service and must be completely served over again (at least once) by both servers. Define the delay in queue of a customer (in a particular queue) to be the total delay in that queue for all of that customer's passes through the facility. Simulate the facility

for each of the following cases (estimate the same measures as before):

- (a) Dissatisfied customers join the tail of queue 1.
- (b) Dissatisfied customers join the head of queue 1.

- 1.26. A service facility consists of two type A servers and one type B server (not necessarily in the psychological sense). Assume that customers arrive at the facility with interarrival times that are IID exponential random variables with a mean of 1 minute. Upon arrival, a customer is determined to be either a type 1 customer or a type 2 customer, with respective probabilities of 0.75 and 0.25. A type 1 customer can be served by any server but will choose a type A server if one is available. Service times for type 1 customers are IID exponential random variables with a mean of 0.8 minute, regardless of the type of server. Type 1 customers who find all servers busy join a single FIFO queue *for type 1 customers*. A type 2 customer requires service from *both* a type A server and the type B server *simultaneously*. Service times for type 2 customers are uniformly distributed between 0.5 and 0.7 minute. Type 2 customers who arrive to find both type A servers busy *or* the type B server busy join a single FIFO queue *for type 2 customers*. Upon completion of service of *any* customer, preference is given to a type 2 customer if one is present and if both a type A and the type B server are then idle. Otherwise, preference is given to a type 1 customer. Simulate the facility for exactly 1000 minutes and estimate the expected average delay in queue and the expected time-average number in queue for each type of customer. Also estimate the expected proportion of time that each server spends on each type of customer.
- 1.27. A supermarket has two checkout stations, regular and express, with a single checker per station; see Fig. 1.96. Regular customers have exponential interarrival times with mean 2.1 minutes and have exponential service times with mean 2.0 minutes. Express customers have exponential interarrival times with mean 1.1 minutes and exponential service times with mean 0.9 minute. The arrival processes of the two types of customers are independent of each other. A regular customer arriving to find at least one checker idle begins service immediately, choosing the regular checker if both are idle; regular customers arriving to find both checkers busy join the end of the regular queue. Similarly, an express customer arriving to find an idle checker goes right into service, choosing the express checker if both are idle; express customers arriving to find both checkers busy join the end of the express queue, even if it is longer than the regular queue. When either checker finishes serving a customer, he takes the next customer from his queue, if any, and if his queue is empty but the other one is not, he takes the first customer from the other queue. If both queues are empty the checker becomes idle. Note that the mean service time of a customer is determined by the

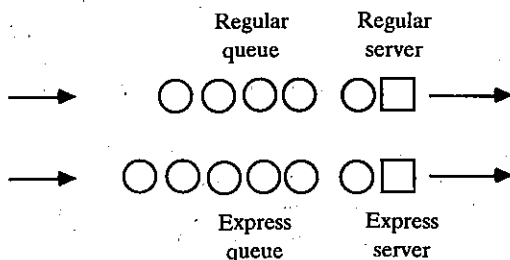


FIGURE 1.96

A supermarket checkout operation.

customer type, and not by whether the checker is the regular or express one. Initially, the system is empty and idle and the simulation is to run for exactly 8 hours. Compute the average delay in each queue, the time-average number in each queue, and the utilization of each checker. What recommendations would you have for further study or improvement of this system? (On June 21, 1983, the *Cleveland Plain Dealer*, in a story entitled "Fast Checkout Wins over Low Food Prices," reported that "Supermarket shoppers think fast checkout counters are more important than attractive prices, according to a survey [by] the Food Marketing Institute. . . . The biggest group of shoppers, 39 percent, replied 'fast checkouts,' . . . and 28 percent said good or low prices. . . [reflecting] growing irritation at having to stand in line to pay the cashier.")

- 1.28. A one-pump gas station is always open and has two types of customers. A police car arrives every 30 minutes (exactly), with the first police car arriving at time 15 minutes. Regular (nonpolice) cars have exponential interarrival times with mean 5.6 minutes, with the first regular car arriving at time 0. Service times at the pump for all cars are exponential with mean 4.8 minutes. A car arriving to find the pump idle goes right into service, and regular cars arriving to find the pump busy join the end of a single queue. A police car arriving to find the pump busy, however, goes to the front of the line, ahead of any regular cars in line: [If there are already other police cars at the front of the line, assume that an arriving police car gets in line ahead of them as well. (How could this happen?)] Initially the system is empty and idle, and the simulation is to run until exactly 500 cars (of any type) have completed their delays in queue. Estimate the expected average delay in queue for each type of car separately, the expected time-average number of cars (of either type) in queue, and the expected utilization of the pump.
- 1.29. Of interest in telephony are models of the following type. Between two large cities, A and B, are a fixed number, n , of long-distance lines or circuits. Each line can operate in either direction (i.e., can carry calls originating in A or B) but can carry only one call at a time; see Fig. 1.97. If a person in A or B wants to place a call to the other city and a line is open (i.e., idle), the call goes through immediately on one of the open lines. If all n lines are busy, the person gets a recording saying that she must hang up and try later; there are no facilities for queueing for the next open line, so these *blocked* callers just go away. The times between attempted calls from A to B are exponential with mean 10 seconds, and the times between attempted calls from B to A are exponential with mean 12 seconds. The length of a conversation is exponential with mean 4 minutes, regardless of the city of origin. Initially all lines are open, and the simulation is to run for 12 hours; compute the time-average number of lines that are busy, the time-average proportion of lines that are busy, the total number of attempted calls (from either city), the number of calls that are blocked, and the proportion

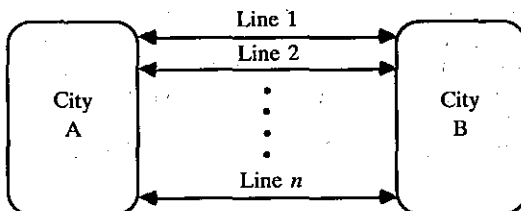


FIGURE 1.97

A long-distance telephone system.

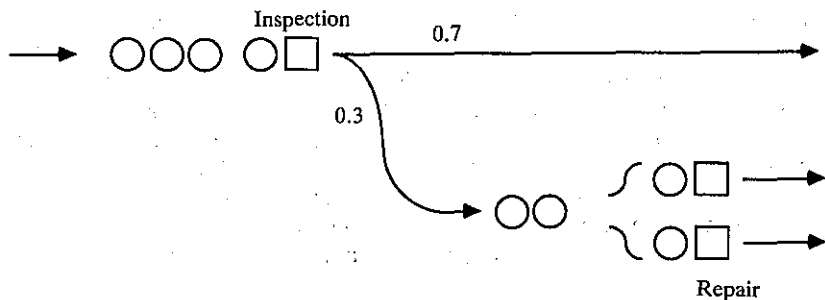


FIGURE 1.98

A bus maintenance depot.

- of calls that are blocked. Determine approximately how many lines would be needed so that no more than 5 percent of the attempted calls will be blocked.
- 1.30. City busses arrive to the maintenance facility with exponential interarrival times with mean 2 hours. The facility consists of a single inspection station and two identical repair stations; see Fig. 1.98. Every bus is inspected, and inspection times are distributed uniformly between 15 minutes and 1.05 hours; the inspection station is fed by a single FIFO queue. Historically, 30 percent of the busses have been found during inspection to need some repair. The two parallel repair stations are fed by a single FIFO queue, and repairs are distributed uniformly between 2.1 hours and 4.5 hours. Run the simulation for 160 hours and compute the average delay in each queue, the average length of each queue, the utilization of the inspection station, and the utilization of the repair station (defined to be half of the time-average number of busy repair stations, since there are two stations). Replicate the simulation five times. Suppose that the arrival rate of busses quadrupled, i.e., the mean interarrival time decreased to 30 minutes. Would the facility be able to handle it? Can you answer this question without simulation?

REFERENCES

- Banks, J., and J. S. Carson: *Discrete-Event System Simulation*, Prentice-Hall, Englewood Cliffs, N.J. (1984).
- Braun, M.: *Differential Equations and Their Applications*, Applied Mathematical Sciences, Vol. 15, Springer-Verlag, New York (1975).
- Chandrasekaran, U., and S. Sheppard: Discrete Event Distributed Simulation—A Survey, *Proc. Conference on Methodology and Validation*, Orlando, Fla., pp. 32–37 (1987).
- Chandy, K. M., and J. Misra: Distributed Simulation: A Case Study in Design and Verification of Distributed Programs, *IEEE Trans. Software Eng.*, SE-5: 440–452 (1979).
- Chandy, K.M., and J. Misra: Asynchronous Distributed Simulation via a Sequence of Parallel Computations, *Commun. Assoc. Comput. Mach.*, 24: 198–206 (1981).
- Chandy, K.M., and J. Misra: Distributed Deadlock Detection, *Assoc. Comput. Mach. Trans. Computer Systems*, 1: 144–156 (1983).
- Chandy, K. M., and C. H. Sauer: *Computer Systems Performance Analysis*, Prentice-Hall, Englewood Cliffs, N.J. (1981).

- Comfort, J. C.: The Simulation of a Master-Slave Event-Set Processor, *Simulation*, 42: 117-124 (1984).
- Davis, G. B., and T. R. Hoffmann: *FORTRAN 77: A Structured, Disciplined Style*, 3d ed., McGraw-Hill, New York (1988).
- Fayek, A.-M. M.: *Introduction to Combined Discrete-Continuous Simulation Using PC SIMSCRIPT II.5*, CACI Products Company, La Jolla, Calif. (1988).
- Fishman, G. S.: *Principles of Discrete Event Simulation*, John Wiley, New York (1978).
- Forgionne, G. A.: Corporate Management Science Activities: An Update, *Interfaces*, 13:3: 20-23 (1983).
- Gordon, G.: *System Simulation*, 2d ed., Prentice-Hall, Englewood Cliffs, N.J. (1978).
- Grogono, P.: *Programming in Pascal*, 2d ed., Addison-Wesley, Reading, Mass. (1984).
- Gross, D., and C. M. Harris: *Fundamentals of Queueing Theory*, 2d ed., John Wiley, New York (1985).
- ♦ Halton, J. H.: A Retrospective and Prospective Survey of the Monte Carlo Method, *SIAM Rev.*, 12: 1-63 (1970).
- ♦ Hammersley, J. M., and D. C. Handscomb: *Monte Carlo Methods*, Methuen, London (1964).
- Harpell, J. L., M. S. Lane, and A. H. Mansour: Operations Research in Practice: A Longitudinal Study, *Interfaces*, 19:3: 65-74 (1989).
- Heidelberger, P.: Discrete Event Simulations and Parallel Processing: Statistical Properties, *SIAM J. Statist. Comput.*, 9: 1114-1132 (1988).
- Jefferson, D. R.: Virtual Time, *Assoc. Comput. Mach. Trans. Programming Languages and Systems*, 7: 404-425 (1985).
- Jensen, K., and N. Wirth (revised by A. B. Mickel and J. F. Miner): *Pascal User Manual and Report*, 3d ed., Springer-Verlag, New York (1985).
- Kernighan, B. W., and D. M. Ritchie: *The C Programming Language*, 2d ed., Prentice-Hall, Englewood Cliffs, N.J. (1988).
- Kleinrock, L.: *Queueing Systems, Vol. I, Theory*, John Wiley, New York (1975).
- Kleinrock, L.: *Queueing Systems, Vol. II, Computer Applications*, John Wiley, New York (1976).
- Koffman, E. B., and F. L. Friedman: *Problem Solving and Structured Programming in FORTRAN 77*, 3d ed., Addison-Wesley, Reading, Mass. (1987).
- Lavenberg, S., R. Muntz, and B. Samadi: Performance Analysis of a Rollback Method for Distributed Simulation, *Performance '83* (A. K. Agrawala and S. K. Tripathi, eds.): 117-132 (1983).
- Law, A. M., and M. G. McComas: Pitfalls to Avoid in the Simulation of Manufacturing Systems, *Ind. Eng.*, 21: 28-31 (May 1989).
- Law, A. M., and M. G. McComas: Secrets of Successful Simulation Studies, *Ind. Eng.*, 22: 47-48, 51-53, 72 (May 1990).
- Misra, J.: Distributed Discrete-Event Simulation, *Computing Surveys*, 18: 39-65 (1986).
- Morgan, B. J. T.: *Elements of Simulation*, Chapman & Hall, London (1984).
- Naylor, T. H.: *Computer Simulation Experiments with Models of Economic Systems*, John Wiley, New York (1971).
- Pegden, C. D.: *Introduction to SIMAN* (January 1989 version), Systems Modeling Corp., Sewickley, Pa. (1989).
- Pratt, C. A.: Catalog of Simulation Software, *Simulation*, 49: 165-181 (1987).
- Pritsker, A. A. B.: *Introduction to Simulation and SLAM II*, 3d ed., Systems Publishing Corp., West Lafayette, Ind. (1986).
- Rasmussen, J. J., and T. George: After 25 Years: A Survey of Operations Research Alumni, Case Western Reserve University, *Interfaces*, 8:3: 48-52 (1978).
- Ross, S. M.: *Introduction to Probability Models*, 4th ed., Academic Press, San Diego, Calif. (1989).
- ♦ Rubinstein, R. Y.: *Simulation and the Monte Carlo Method*, John Wiley, New York (1981).
- Sargent, R. G.: Event Graph Modelling for Simulation with an Application to Flexible Manufacturing Systems, *Management Sci.*, 34: 1231-1251 (1988).

- Schmidt, J. W., and R. E. Taylor: *Simulation and Analysis of Industrial Systems*, Richard D. Irwin, Homewood, Ill. (1970).
- Schruben, L.: Simulation Modeling with Event Graphs, *Commun. Assoc. Comput. Mach.*, 26: 957-963 (1983).
- Shannon, R. E.: *Systems Simulation: The Art and Science*, Prentice-Hall, Englewood Cliffs, N.J. (1975).
- Shannon, R. E., S. S. Long, and B. P. Buckles: Operations Research Methodologies in Industrial Engineering, *AIIE Trans.*, 12: 364-367 (1980).
- Sheppard, S., R. E. Young, U. Chandrasekaran, and M. Krishnamurthi: Three Mechanisms for Distributing Simulation, *Proc. 12th Conference of the NSF Production Research and Technology Program*, Madison, Wis., pp. 67-70 (1985).
- Solomon, S. L.: *Simulation of Waiting-Line Systems*, Prentice-Hall, Englewood Cliffs, N.J. (1983).
- Som, T. K., and R. G. Sargent: A Formal Development of Event Graphs as an Aid to Structured and Efficient Simulation Programs, *ORSA J. Comput.*, 1: 107-125 (1989).
- Stidham, S.: A Last Word on $L = \lambda w$, *Operations Res.*, 22: 417-421 (1974).
- Swart, W., and L. Donno: Simulation Modeling Improves Operations, Planning, and Productivity for Fast Food Restaurants, *Interfaces*, 11:6: 35-47 (1981).
- Thomas, G., and J. DaCosta: A Sample Survey of Corporate Operations Research, *Interfaces*, 9:4: 102-111 (1979).

CHAPTER 2

MODELING COMPLEX SYSTEMS



Recommended sections for a first reading: 2.1 through 2.5

2.1 INTRODUCTION

In Chap. 1 we looked at simulation modeling in general, and then modeled and coded two specific systems. Those systems were very simple, and it was possible to program them directly in a general-purpose language, without using any special simulation software or support programs (other than a random-number generator). Most real-world systems, however, are quite complex, and coding them without supporting software can be a difficult and time-consuming task.

In this chapter we first discuss an activity that takes place in most simulations, *list processing*. A group of FORTRAN support routines, SIMLIB, is then introduced, which takes care of some standard list-processing tasks as well as several other common simulation chores, such as processing the event list, accumulating statistics, generating random numbers and observations from a few distributions, and writing out results. SIMLIB is then used in four example simulations, the first of which is just the single-server queueing system from Sec. 1.4 (included to illustrate the use of SIMLIB on a familiar model); the last three examples are of somewhat greater complexity.